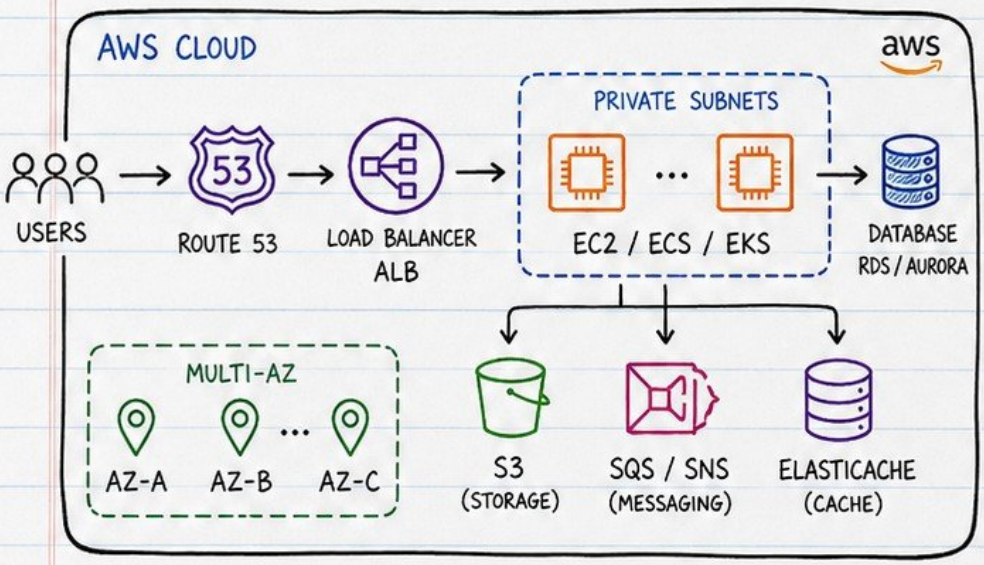
VERIQTA

EDITION 1.0
VERIQTA SERIES

AWS SOLUTIONS ARCHITECT PRODUCTION HANDBOOK

Building Enterprise AWS Systems That Scale, Recover, and Survive Production



- SECURE BY DESIGN
- BUILT FOR SCALE
- DESIGNED FOR RELIABILITY
- ENGINEERED FOR RESILIENCE
- OPTIMIZED FOR COST + VALUE

WHAT'S INSIDE

- ☑ Production-Grade Architectures
- ☑ Design Principles & Trade-offs
- ☑ Reliability & Disaster Recovery
- ☑ Security & Compliance
- ☑ Performance & Scaling Strategies
- ☑ Cost Optimization Techniques
- ☑ Real-World Patterns & Pitfalls
- ☑ Enterprise Reference Architectures

FOR ENGINEERS WHO:

- Design Systems
- Run Production
- Solve Problems
- Think in Trade-offs
- Own Outcomes

THE PRODUCTION ARCHITECT MINDSET

An architect does not just choose services.
An architect designs systems that deliver business value, handle scale, survive failure, and remain cost effective.

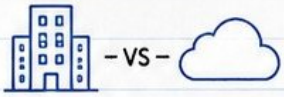
THINKING IN SYSTEMS

- See the big picture
- Understand how components interact
- Optimize the system, not the service
- Think end-to-end
- Design for scale

DESIGNING FOR FAILURE

- Failure will happen
- Assume a component, AZ, or Region can fail
- Build for recovery, not just for uptime
- Test your failure assumptions

BUSINESS VS TECHNICAL REQUIREMENTS



BUSINESS (Why)

- What problem are we solving?
- What is the impact?
- What is success?
- What are the constraints?

TECHNICAL (How)

- How will we build it?
- Which services?
- How will it scale?
- How will we secure it?

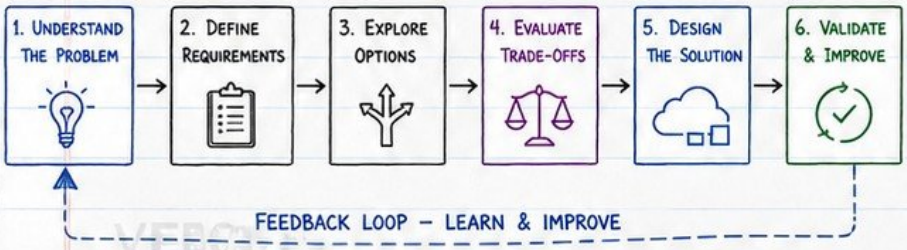
TRADE-OFFS

- No architecture is perfect
- Every decision has a cost
- Balance: Cost, Performance, Reliability, Security, and Speed
- Choose what matters most for the business

ARCHITECTURE PRINCIPLES

- Guiding rules for good architecture
- Keep it simple (as simple as possible)
- Build loose coupling
- Automate everything
- Measure everything
- Improve continuously

ARCHITECTURAL THINKING FLOW



ASK YOURSELF THESE QUESTIONS

- ☑ What is the real business problem?
- ☑ Who are the users?
- ☑ What happens if this fails?
- ☑ How will we detect issues?
- ☑ How will we recover?
- ☑ How will this scale?
- ☑ What will this cost?
- ☑ Is this secure by design?
- ☑ Can we operate and maintain it?

CORE ARCHITECTURE PRINCIPLES (WELL-ARCHITECTED PILLARS)

OPERATIONAL EXCELLENCE	Automate operations, monitor systems, learn from failures, and improve continuously.
SECURITY	Protect data, control access, encrypt everything, and build with a Zero Trust mindset.
RELIABILITY	Design for failure, test recovery, and ensure consistent performance.
PERFORMANCE EFFICIENCY	Use the right resources, scale efficiently, and remove bottlenecks.
COST OPTIMIZATION	Optimize cost without sacrificing value. Measure, right-size, and eliminate waste.
SUSTAINABILITY	Build efficiently, reduce waste, and design systems that last.

PRODUCTION NOTE

The cloud does not remove responsibility.
It moves responsibility to architectural design and operational excellence.

BAD ARCHITECTURE FAILS IN PRODUCTION.
GOOD ARCHITECTURE FAILS GRACEFULLY.

SENIOR ENGINEER TIP

Always document your decisions, assumptions, and trade-offs.
Future you (or your team) will thank you.

GATHERING BUSINESS REQUIREMENTS

Great architecture starts with deep understanding of the business.
The right questions up front prevent wrong solutions later.

FUNCTIONAL REQUIREMENTS

- What should the system do?
- Core features
- User roles & permissions
- Workflow & business processes
- Data inputs and outputs
- Integrations with other systems

NON-FUNCTIONAL REQUIREMENTS

- Performance
- Scalability
- Security
- Reliability
- Maintainability
- Usability
- Observability
- Portability

AVAILABILITY TARGETS

- Uptime requirements (e.g., 99.9%, 99.99%)
- RTO (Recovery Time Objective)
- RPO (Recovery Point Objective)
- Maintenance windows
- Multi-AZ or Multi-Region needs

BUDGET CONSTRAINTS

- Project budget
- Monthly/Annual run-cost limits
- Cost sensitivity (High / Medium / Low)
- Cost optimization expectations
- Preference for managed services vs self-managed

COMPLIANCE REQUIREMENTS

- Regulatory standards (e.g., GDPR, HIPAA, PCI-DSS, SOC 2)
- Data residency requirements
- Audit & logging requirements
- Data classification
- Retention policies

GROWTH PROJECTIONS

- Current user base
- Expected growth (users, data, transactions)
- Growth rate (monthly/yearly)
- Padit loads & seasonality
- Future features on the roadmap

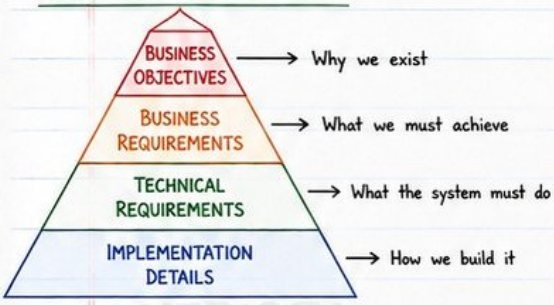
KEY BUSINESS QUESTIONS TO ASK

- ☑ What problem are we solving?
- ☑ Who are the users?
- ☑ What does success look like?
- ☑ What happens if the system is down?
- ☑ What are the biggest risks to the business?
- ☑ What are the must-have vs nice-to-have?
- ☑ What are our constraints (time, budget, people)?
- ☑ How will this system generate or save money?
- ☑ What systems must we integrate with?
- ☑ What legal or compliance rules must we follow?

STAKEHOLDERS & THEIR FOCUS

STAKEHOLDER	WHAT THEY CARE ABOUT	WHAT YOU NEED FROM THEM
EXECUTIVES	Business value, risk, cost, time-to-market	Goals, priorities, success metrics
PRODUCT OWNERS	Features, user needs, roadmap	Functional requirements, priorities, trade-offs
SECURITY / COMPLIANCE	Risk, data protection, compliance	Policies, standards, constraints
OPERATIONS / IT	Reliability, monitoring, supportability	Operational requirements, runbook expectations
FINANCE	Budget, ROI, ongoing costs	Budget limits, cost targets, approval process

REQUIREMENTS HIERARCHY



EXAMPLE (E-COMMERCE PLATFORM)

- Users: 5M today → 25M in 2 years
- Peak: 10x normal traffic (Black Friday)
- Availability: 99.95%
- RTO: 1 hour | RPO: 15 minutes
- Budget: $50,000/month
- Compliance: PCI-DSS, GDPR
- Data: Must stay in us-east-1
- Integrations: Payment Gateway, ERP, Shipping Provider
- Goal: Fast, secure, scalable, and cost-efficient

PRODUCTION NOTE

Most architecture failures happen when requirements are unclear or incomplete.

CLARITY HERE = STABILITY IN PRODUCTION.

SENIOR ENGINEER TIP

Spend more time here than on any tool or service.
Great architecture is 80% discovery, 20% design.

- ☑ Assumptions kill architectures.
- ☑ Document everything.
- ☑ Validate with stakeholders early and often.

BETTER QUESTIONS
BETTER ARCHITECTURE
BETTER OUTCOMES

AWS GLOBAL INFRASTRUCTURE

AWS Global Infrastructure is the foundation of every resilient, scalable, and highly available architecture.

1. REGIONS

- A Region is a separate geographic area.
- Each Region is isolated from others.
- Choose Regions close to your users.
- Example: us-east-1 (N. Virginia)
- Use multiple Regions for DR and compliance.

2. AVAILABILITY ZONES (AZs)

- Each Region has multiple, physically separate AZs.
- Connected with low-latency, high-bandwidth network.
- Designed to be isolated from failures in other AZs.
- Never deploy critical workloads in a single AZ.
- Spread across AZs for high availability.

3. EDGE LOCATIONS

- 400+ Points of Presence (PoPs) worldwide.
- Content is cached close to users using CloudFront.
- Improves performance and reduces latency.
- Used for CDN, DDoS protection, and edge security.

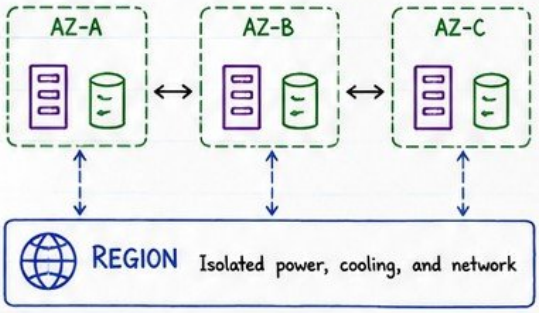
4. LOCAL ZONES

- Extend AWS compute, storage, and database services closer to end users.
- Ideal for latency-sensitive applications.
- Each Local Zone is connected to a parent Region.
- Example: game servers, streaming, real-time analytics.

5. WAVELENGTH

- Brings AWS services to the 5G edge.
- Ultra-low latency for mobile applications.
- Enables real-time use cases like AR/VR, IoT, gaming.
- Deployed in telecom provider data centers.

6. FAULT ISOLATION



FAULT ISOLATION PRINCIPLES

- ☑ AZs are isolated from each other.
- ☑ Failure in one AZ does not impact others.
- ☑ Each AZ has independent power, cooling, and network.
- ☑ Services are designed for failure.
- ☑ Design with failure as the default assumption.

HOW TRAFFIC FLOWS GLOBALLY

KEY TAKEAWAYS

- ☑ Pick the right Region.
- ☑ Design across multiple AZs.
- ☑ Use Edge for performance.
- ☑ Use Local Zones for low latency.
- ☑ Use Wavelength for 5G edge.
- ☑ Design for failure from day one.

CHOOSING THE RIGHT CONSTRUCT

CONSTRUCT	PRIMARY PURPOSE	LATENCY	USE WHEN
REGION	Isolation, compliance, DR	High (cross-region)	You need isolation or DR
AZ	High availability	Very Low	You need HA within ia Region
EDGE LOCATIONS	Performance, CDN, security	Very Low (at edge)	Static/dynamic content delivery
LOCAL ZONES	Low-latency compute	Low	Users are near a metro area
WAVELENGTH	5G edge applications	Ultra Low	Mobile, IoT, AR/VR, real-time

PRODUCTION NOTE

Never build with a single point of failure.

Your architecture should continue serving traffic even when an AZ, Local Zone, or network segment is down.

Resiliency is a design decision, not a feature.

CHOOSING THE RIGHT COMPUTE

There is no best compute service—only the best fit for your workload, team, and operational model.

1. EC2

- Full control over OS, runtime, and software.
- Best for lift-and-shift, legacy apps, and custom workloads.
- You manage scaling, patching, and HA.
- Highest flexibility, highest responsibility.

2. ECS (FARGATE OR EC2)

- Run containerized apps without managing servers (Fargate) or with EC2 control.
- Simple, serverless (on Fargate) container orchestration.
- Great for microservices and web applications.

3. EKS

- Managed Kubernetes service.
- Best for complex, large-scale, and cloud-native apps.
- Huge ecosystem and portability.
- More operational overhead than ECS.

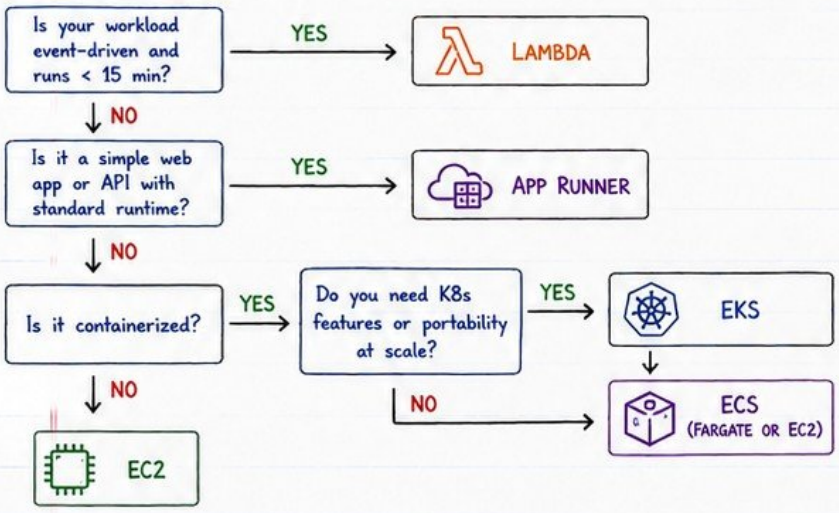
4. LAMBDA

- Serverless functions (event-driven).
- Pay per request and duration.
- Best for stateless, short-running tasks.
- Not ideal for long-running or persistent processes.

5. APP RUNNER

- Fully managed platform for containerized web apps and APIs.
- Built-in autoscaling, HTTPS, and custom domains.
- Simplest way to run web apps on AWS.

6. DECISION FRAMEWORK



SENIOR ENGINEER TIP

Start with the simplest service that meets your requirements.

You can always move to a more powerful option, but moving from complex to simple is harder.

SIMPLICITY FIRST.
OPERATE WHAT YOU NEED, NOT WHAT YOU CAN.

THINK ABOUT

- ☑ Team skills and operational maturity
- ☑ Time to market
- ☑ Scaling needs
- ☑ Cost model
- ☑ Compliance and security needs
- ☑ Observability and tooling

7. COMPUTE COMPARISON

CRITERIA	EC2	ECS (FARGATE)	EKS	LAMBDA	APP RUNNER
MANAGEMENT	You manage	Serverless (AWS manages compute)	You manage K8s control plane (AWS manages control plane)	Fully serverless	Fully managed
SCALING	Manual / Auto Scaling	Service Auto Scaling	K8s / Cluster Autoscaler	Automatic	Automatic
BEST FOR	Lift-and-shift, legacy, custom apps	Microservices, web apps	Complex apps, multi-tenant, cloud-native	Event-driven, APIs, tasks	Web apps, APIs, simple containers
COST MODEL	Pay for instances	Pay per vCPU/Memory used	Pay for nodes + usage	Pay per request + duration	Pay per vCPU/Memory + requests
FLEXIBILITY	Highest	High	Very High	Low (by design)	Medium
OPERATIONS	High	Low	High	Very Low	Very Low

PRODUCTION NOTE

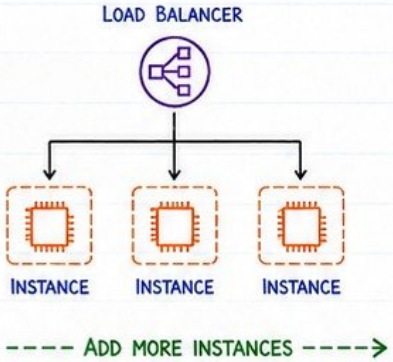
- Match the compute to the workload, not the other way around.
- Don't over-engineer. Start small.
- Revisit your choice as requirements evolve.
- Monitor cost and performance continuously.
- Design for portability and avoid vendor lock-in where possible.

COMPUTE SCALING STRATEGIES

Scaling keeps your applications fast, available, and cost-effective as demand changes.

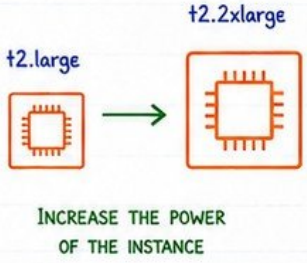
1. HORIZONTAL SCALING (SCALE OUT)

- Add more instances.
- Distribute traffic across instances.
- Best for handling variable workloads.
- Improves availability and fault tolerance.
- Stateless design recommended.



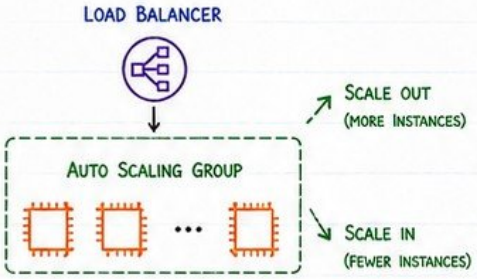
2. VERTICAL SCALING (SCALE UP)

- Increase the size of existing instance.
- More CPU, RAM, and storage.
- Limited by instance family limits.
- May require downtime for some workloads.



3. AUTO SCALING

- Automatically adjusts capacity based on demand.
- Maintains availability and optimizes cost.
- Works across EC2, ECS, and EKS.



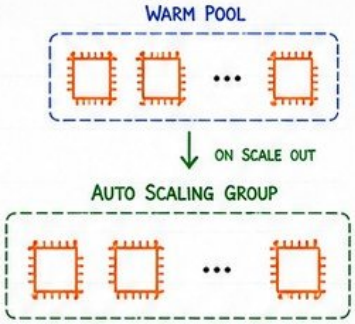
4. SCALING POLICIES

- Define when and how scaling actions occur.
- Based on metrics, schedules, or events.
- Ensure performance while controlling cost.

POLICY TYPE	DESCRIPTION
Target Tracking	Maintain a metric at a target value
Step Scaling	Scale based on metric thresholds
Simple Scaling	Add or remove a fixed number of instances
Scheduled Scaling	Scale on a predictable schedule
Predictive Scaling	Use ML to forecast demand and scale

5. WARM POOLS

- Keep instances pre-initialized and ready.
- Reduces launch time during scale-out.
- Ideal for applications with sudden traffic spikes.
- Saves cost compared to keeping extra instances running.



SENIOR ENGINEER TIP

- Prefer horizontal scaling for resilience.
- Design stateless services whenever possible.
- Use Auto Scaling with Target Tracking for stable systems.
- Right-size instances and use metrics that reflect real user experience (not just CPU).
- Warm Pools are a game-changer for spiky workloads.
- Test scaling in staging before production.

SCALING METRICS EXAMPLES

METRIC	BEST USED FOR
CPU Utilization	General purpose workloads
Memory Utilization	Memory intensive workloads
Request Count / RPS	Web applications and APIs
Queue Depth (SQS)	Background workers
Custom Metrics	Application-specific signals

SCALING CONSIDERATIONS

- ☑ Application architecture (stateful vs stateless)
- ☑ Startup time of instances
- ☑ Database and downstream capacity
- ☑ Network bandwidth and limits
- ☑ Cost impact of scaling actions
- ☑ Monitoring and alerting in place
- ☑ Testing scaling behavior under load

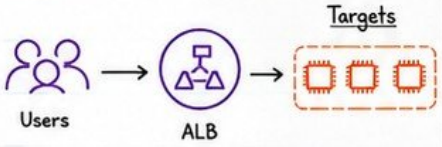
PRODUCTION NOTE

Scaling is not just about adding capacity. It is about delivering the right capacity at the right time with the right cost. A well-designed scaling strategy improves reliability, user experience, and operational efficiency. Plan, measure, test, and iterate.

LOAD BALANCING ARCHITECTURE

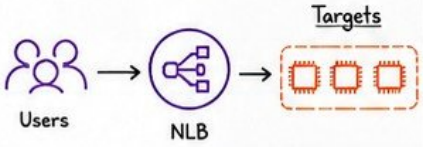
Distribute traffic. Improve availability. Protect your systems.
Load balancers are the front door of your architecture.

1. APPLICATION LOAD BALANCER (ALB)



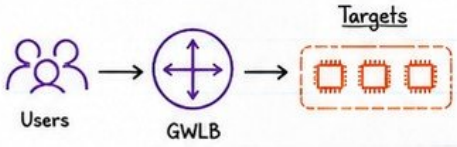
- Layer 7 load balancing (HTTP/HTTPS)
- Routes based on host, path, headers
- Supports advanced routing rules
- Ideal for HTTP/S web applications and APIs
- Works with EC2, ECS, EKS, Lambda (via IP)

2. NETWORK LOAD BALANCER (NLB)



- Layer 4 load balancing (TCP/UDP/TLS)
- Ultra-low latency and high throughput
- Static IP (per AZ), preserves source IP
- Ideal for high-performance, non-HTTP apps
- Works with EC2, ECS, IP targets

3. GATEWAY LOAD BALANCER (GWLB)



- Layer 3/4 load balancing for appliances
- Routes traffic to security and inspection appliances
- Transparent to applications
- Ideal for firewalls, IDS/IPS, intrusion detection
- Works with IP targets in Auto Scaling groups

4. HEALTH CHECKS

COMPONENT	DETAILS
Purpose	Verify target instances are healthy
Protocol	HTTP/HTTPS, TCP, or custom
Interval	How often checks are sent (e.g., 30s)
Timeout	How long to wait for a response (e.g., 5s)
Healthy Threshold	Consecutive successes to mark healthy
Unhealthy Threshold	Consecutive failures to mark unhealthy
Action	Help LB route only to healthy targets

Best Practice

- ☑ Tune intervals and thresholds based on application behavior.
- ☑ Avoid flapping by using appropriate thresholds.

5. STICKY SESSIONS

METHOD	DESCRIPTION
Purpose	Send same user to the same target for the session
ALB	Uses cookie (lb_cookie) or application-based stickiness
NLB	Uses source IP for stickiness
GWLB	Not applicable (stateful appliances handle session)
Use Cases	Cart, user sessions, workflows

Best Practice

- ☑ Use only when needed.
- ☑ Ensure targets are stateful or session-aware.

KEY TAKEAWAYS

- ☑ Choose the right LB for the right job.
- ☑ Health checks ensure only healthy targets receive traffic.
- ☑ Stickiness helps maintain session context when needed.
- ☑ Monitor LB metrics and tune for performance and availability.
- ☑ Balance cost, performance, and complexity.

LOAD BALANCER COMPARISON

FEATURE	ALB	NLB	GWLB
Layer	Layer 7	Layer 4	Layer 3/4
Protocols	HTTP, HTTPS, gRPC, WebSocket	TCP, UDP, TLS	GENEVE (for appliances)
Routing	Content-based (host, path, header)	Connection-based	Traffic to appliances
IP Address	Dynamic (per AZ)	Static (per AZ)	Dynamic (per AZ)
Use Case	Web apps, APIs, microservices	High performance, TCP services	Security appliances, inspection
Stickiness	Yes (cookie / app-based)	Yes (source IP)	No
SSL Termination	Yes	No (pass-through)	No
Best For	Layer 7 application routing	High throughput, low latency	Network and security appliances

SENIOR ENGINEER TIP

Use ALB for application routing and advanced features. Use NLB for performance and static IP. Use GWLB for centralized network security and inspection.

PRODUCTION NOTE

Load balancers are not just performance tools--they are reliability, security, and availability enablers. Design for scale, monitor continuously, and test failover scenarios regularly.

LB KEY METRICS

- ☑ Request Count / Latency
- ☑ Healthy / Unhealthy Targets
- ☑ Target Response Time
- ☑ 5xx Errors / Throttles

PRODUCTION VPC DESIGN

ENGINEERING EXCELLENCE VERIQTA

A well-designed VPC is the foundation of a secure, scalable, and highly available AWS architecture.

1. CIDR PLANNING

- Use private IP address space (RFC1918).
- Plan for current needs and future growth.
- Avoid overlapping CIDRs.
- Use /16 per VPC as a best practice.

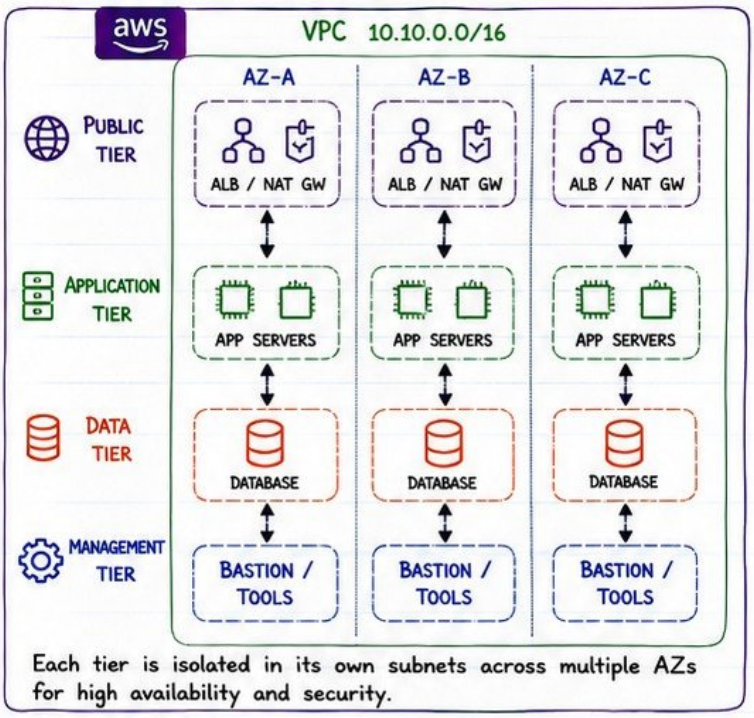
CIDR BLOCK	USABLE IPS
/16	65,534
/17	32,766
/18	16,382
/19	8,190
/20	4,094
/24	251

EXAMPLE CIDR PLAN (10.10.0.0/16)

TIER	CIDR BLOCK	AZ-A	AZ-B	AZ-C
Public Subnets	10.10.0.0/20	10.10.0.0/24	10.10.16.0/24	10.10.32.0/24
Private App Subnets	10.10.48.0/20	10.10.48.0/24	10.10.64.0/24	10.10.80.0/24
Private Data Subnets	10.10.96.0/20	10.10.96.0/24	10.10.112.0/24	10.10.128.0/24
Database Subnets	10.10.144.0/20	10.10.144.0/24	10.10.160.0/24	10.10.176.0/24
Management Subnets	10.10.192.0/20	10.10.192.0/24	10.10.208.0/24	10.10.224.0/24

TIP: Leave room to grow. You can't resize a VPC CIDR later.

2. MULTI-TIER ARCHITECTURE



Each tier is isolated in its own subnets across multiple AZs for high availability and security.

3. PUBLIC VS PRIVATE SUBNETS

PUBLIC SUBNETS

- Have a route to an Internet Gateway.
- Used for resources that need internet access (ALB, NAT Gateway, Bastion, etc).
- Assign Public IPs only when necessary.

PRIVATE SUBNETS

- No direct route to the Internet.
- Access internet via NAT Gateway (for outbound only).
- Used for application servers and databases.

Keep your workload private. Expose only what must be public.

4. ROUTE TABLES

ROUTE TABLE	ASSOCIATED SUBNETS	ROUTES (EXAMPLES)	TARGET
PUBLIC RT	Public Subnets	10.10.0.0/16	local
		0.0.0.0/0	igw-1234abcd
PRIVATE APP RT	Private App Subnets	10.10.0.0/16	local
		0.0.0.0/0	nat-1234abcd
PRIVATE DATA RT	Private Data Subnets	10.10.0.0/16	local
			–
MANAGEMENT RT	Management Subnets	10.10.0.0/16	local
		0.0.0.0/0	nat-1234abcd

BEST PRACTICE

Use separate route tables for each tier.
Associate only the subnets that need those routes.

PRODUCTION VPC DESIGN CHECKLIST

- ☑ CIDR block sized for growth
- ☑ Subnets distributed across 3+ AZs
- ☑ Public subnets for internet-facing components
- ☑ Private app subnets for workloads
- ☑ Private data subnets for databases
- ☑ Management subnets for admin access
- ☑ Route tables follow least privilege
- ☑ NAT Gateways in multiple AZs
- ☑ VPC Endpoints for AWS services
- ☑ Flow logs enabled
- ☑ Security groups and NACLs in place
- ☑ Monitoring and alarms configured

PRODUCTION NOTE

A great VPC design reduces risk, improves performance, and makes your architecture easier to operate at any scale.

VERIQTA

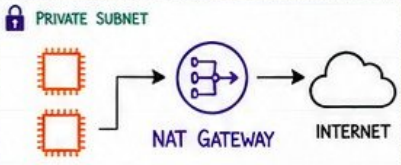
ENTERPRISE NETWORKING

ENGINEERING VERIQTA EXCELLENCE

A secure, scalable, and well-connected network is the foundation of every enterprise architecture on AWS.

1. NAT GATEWAY

- Allows instances in private subnets to access the internet for outbound connections.
- No inbound connections initiated from the internet.
- Highly available when deployed in multiple AZs.



ROUTE TABLE	
DESTINATION	TARGET
0.0.0.0/0	nat-0abc123def456
10.0.0.0/16	local

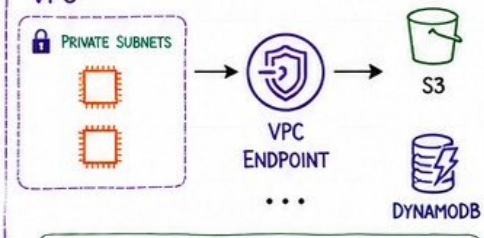
2. INTERNET GATEWAY

- Enables communication between your VPC and the internet.
- Attached to the VPC (one per VPC).
- Used by resources in public subnets.

ROUTE TABLE	
DESTINATION	TARGET
0.0.0.0/0	igw-1234abcd
10.0.0.0/16	local

3. VPC ENDPOINTS

- Private connections from your VPC to AWS services without using the internet.
- Improves security and reduces data transfer costs.
- Types: Gateway Endpoints (S3, DynamoDB) Interface Endpoints (PrivateLink).

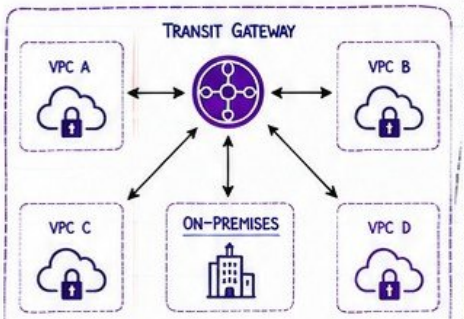


BENEFITS

- ☑ Traffic stays on AWS network
- ☑ Enhanced security (no internet exposure)
- ☑ Lower latency and data transfer cost

4. TRANSIT GATEWAY

- Hub-and-spoke networking to connect multiple VPCs and on-premises networks.
- Simplifies network management at scale.
- Centralized routing and inspection.

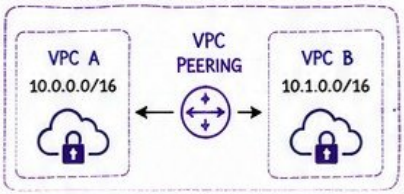


USE CASES

- ☑ Multi-account / multi-VPC environments
- ☑ Hybrid cloud connectivity
- ☑ Centralized security inspection and logging

5. PEERING

- Connects two VPCs privately using the AWS backbone.
- No overlap in CIDR blocks.
- Non-transitive (A peered with B, B not automatically with C).



USE CASES

- ☑ Application-to-application communication
- ☑ Shared services across VPCs
- ☑ Simple and cost-effective connectivity

NOTE

Peering does not support transitive routing. Use Transit Gateway for hub-and-spoke or multi-VPC scaling.

COMPARISON AT A GLANCE

SERVICE	PURPOSE	INTERNET	SCOPE
NAT Gateway	Outbound internet for private subnets	Yes	Per AZ
Internet Gateway	Inbound/Outbound internet access	Yes	Per VPC
VPC Endpoints	Private access to AWS services	No	Per VPC
Transit Gateway	Connect multiple VPCs & on-prem	No	Global
Peering	Connect two VPCs privately	No	Per Connection

SENIOR ENGINEER TIP

- ☑ Use NAT Gateway, not NAT Instance, for high availability and scale.
- ☑ Deploy NAT Gateway in each AZ where private subnets exist.
- ☑ Prefer VPC Endpoints for S3, DynamoDB, CloudWatch, ECR, and more.
- ☑ Use Transit Gateway for enterprise hub-and-spoke at scale.
- ☑ Document all networking and routing for troubleshooting and audits.

VERIQTA

NETWORK DESIGN BEST PRACTICES

Plan IP addressing and CIDR blocks early. Avoid overlaps.	Keep private subnets private. No public IPs unless necessary.	Use least privilege security groups and NACLs.
Use separate subnets per AZ for high availability.	Enable VPC Flow Logs for visibility and troubleshooting.	Monitor and alert on network metrics and anomalies.

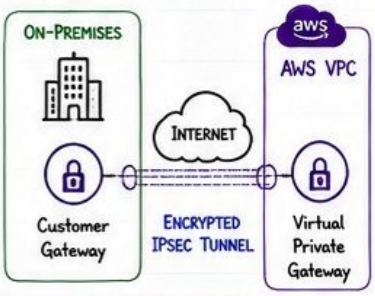
HYBRID CLOUD CONNECTIVITY

Connect your on-premises infrastructure to AWS securely, reliably, and at scale.

1. VPN CONNECTION

- Site-to-Site VPN connects your on-premises network to your VPC over the internet.
- Encrypted tunnels using IPsec.
- Cost-effective for low bandwidth and backup connectivity.

SITE-TO-SITE VPN ARCHITECTURE

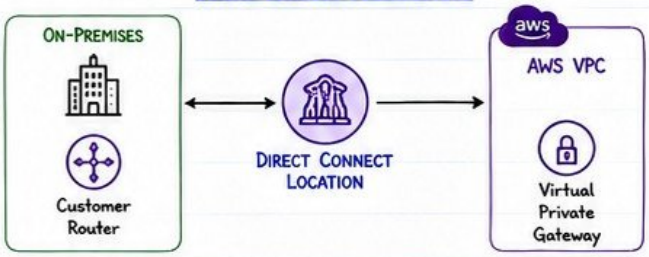


VPN CONSIDERATIONS	
FEATURE	DETAILS
Bandwidth	Up to 1.25 Gbps per VPN tunnel
Availability	Use two tunnels (one per AZ) for HA
Best For	Low to medium bandwidth, backup
Cost	Pay for data transfer out + VPN

2. AWS DIRECT CONNECT

- Dedicated network connection from your data center to AWS.
- Consistent latency, higher bandwidth, and greater reliability.
- Reduces internet dependency and improves security.

DIRECT CONNECT ARCHITECTURE



DIRECT CONNECT CONSIDERATIONS	
FEATURE	DETAILS
Bandwidth	50 Mbps to 100 Gbps
Availability	Use connections in multiple locations
Best For	High bandwidth, stable, critical workloads
Cost	DX connection + data transfer out

3. DNS FOR HYBRID ENVIRONMENTS

- Use Route 53 Resolver for hybrid DNS resolution.
- Ensure secure, reliable name resolution across environments.
- Avoid DNS latency and resolution failures.

HYBRID DNS ARCHITECTURE

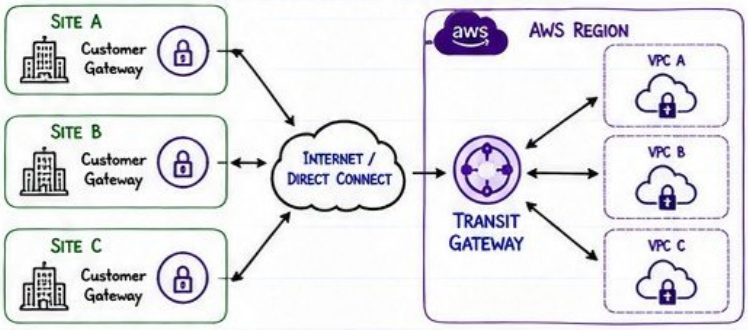
BEST PRACTICES

- ☑ Use conditional forwarders for on-prem domains.
- ☑ Enable DNS query logging for visibility.
- ☑ Use private hosted zones for internal domains.
- ☑ Ensure high availability with endpoints across AZs.

4. MULTI-SITE NETWORKING

- Connect multiple on-premises sites to AWS.
- Centralize connectivity using Transit Gateway.
- Simplify routing and improve scalability.

MULTI-SITE CONNECTIVITY WITH TRANSIT GATEWAY



BENEFITS

- ☑ Centralized routing and management
- ☑ Consistent network policies
- ☑ Scalable to dozens of sites and VPCs
- ☑ Reduces operational complexity

KEY TAKEAWAYS

- ☑ Use VPN for cost-effective, low-bandwidth or backup connectivity.
- ☑ Use Direct Connect for high bandwidth, consistent performance, and reliability.
- ☑ Use Route 53 Resolver for secure and efficient DNS across environments.
- ☑ Use Transit Gateway to scale connectivity across multiple sites and VPCs.
- ☑ Design for high availability, security, and future growth.

HYBRID CONNECTIVITY CHECKLIST

- ☑ Choose VPN or Direct Connect based on needs
- ☑ Use redundant tunnels or connections
- ☑ Implement Route 53 Resolver endpoints
- ☑ Use private IP addressing across links
- ☑ Enable monitoring and alerting
- ☑ Test failover and recovery regularly
- ☑ Document network architecture
- ☑ Review security and compliance

STORAGE DECISION FRAMEWORK

Choosing the right storage service improves performance, reduces cost, and simplifies operations.

1. STORAGE SERVICES OVERVIEW

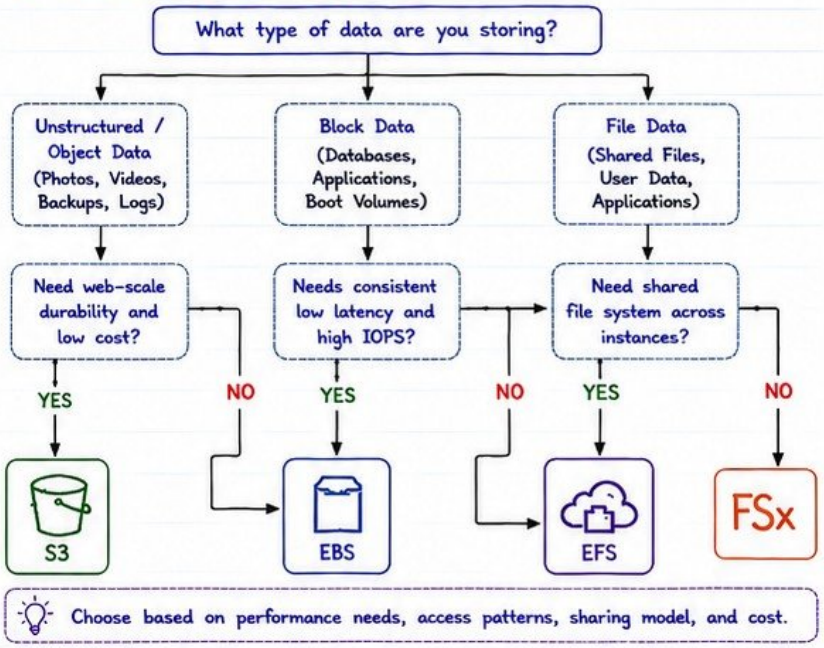
S3 – Simple Storage Service
- Highly durable object storage
- Virtually unlimited scale
- Ideal for backups, archives, static content, data lakes, and more

EBS – Elastic Block Store
- Block storage for EC2 instances
- Low latency, high performance
- Ideal for databases, boot volumes, and transactional workloads

EFS – Elastic File System
- Managed NFS file system
- Shared access across multiple EC2 instances
- Ideal for lift-and-shift, content management, home directories

FSx – Fully Managed File Systems
- Native integrations (Windows, Lustre, NetApp)
- High performance, feature rich
- Ideal for enterprise apps, HPC, and specialized workloads

2. STORAGE DECISION FRAMEWORK



3. STORAGE SERVICES AT A GLANCE

SERVICE	S3	EBS	EFS	FSx
TYPE	Object Storage	Block Storage	File Storage (NFS)	Managed File Systems
ACCESS	REST API / SDK	Attached to EC2	NFS v4.1	Protocol Specific (SMB, Lustre, NFS, iSCSI)
SCALABILITY	Virtually Unlimited	Up to 16 TiB per volume	Petabyte scale	Up to Petabytes
LATENCY	Milliseconds	Low (sub-millisecond)	Low	Low (varies by FS type)
DURABILITY	11 9's (99.999999999%)	99.999%	99.9999999999%	99.9% – 99.999999999%
THROUGHPUT	High	High (depends on volume type)	High	Very High
USE CASES	Backups, Archives, Static Websites, Data Lakes	Databases, Boot Volumes, ERP, Transactions	Home Directories, Content Management, Lift-and-Shift	Enterprise Apps, HPC, Media Workflows
PRICING MODEL	Per GB stored	Per GB-month + IOPS/Throughput	Per GB-month + Throughput	Per GB-month + Capacity/Throughput

4. STORAGE TRADE-OFFS

FACTOR	S3	EBS	EFS	FSx
Performance	★★★☆☆	★★★★★	★★★☆☆	★★★★★
Latency	Higher	Lowest	Low	Low – Very Low
Sharing	N/A	Single Instance	Multi-Instance	Multi-Instance
Durability	Excellent	Good	Excellent	Excellent
Cost	Lowest	Medium – High	Medium	High
Best For	Massive scale, low cost	High performance block workloads	Shared file use cases	Specialized, high performance

5. BEST PRACTICES

- ☑ Use S3 Lifecycle policies to transition and expire data.
- ☑ Choose the right EBS volume type for workload needs.
- ☑ Use EFS for shared file access across multiple instances.
- ☑ Use FSx when you need advanced features or native application integration.
- ☑ Encrypt data at rest and in transit.
- ☑ Implement backups and cross-region replication.
- ☑ Monitor storage usage and performance continuously.
- ☑ Review and optimize storage costs regularly.

SENIOR ENGINEER TIP

Start with the simplest service that meets your needs. You can always migrate or refactor later. Over-engineering storage is costly!

DATABASE SELECTION FRAMEWORK

Choose the right database for the right workload to optimize performance, cost, and scalability.

1. DATABASE OPTIONS OVERVIEW

RDS – RELATIONAL DATABASE SERVICE
- Managed relational databases (MySQL, PostgreSQL, MariaDB, Oracle, SQL Server)
- Easy to use, maintain, and scale
- Ideal for traditional relational applications

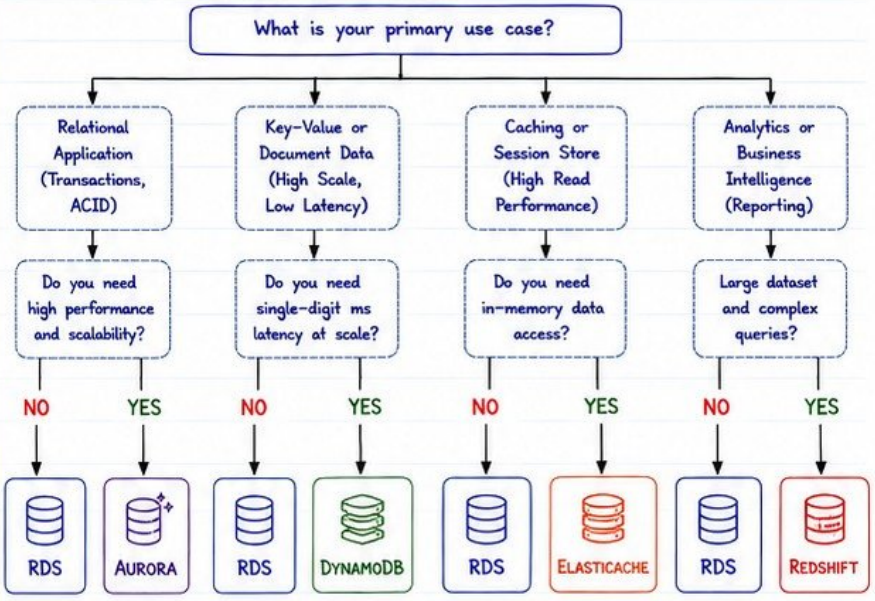
AURORA – AWS NATIVE RELATIONAL
- MySQL and PostgreSQL compatible
- High performance and availability
- Auto scaling storage and serverless options
- Ideal for high throughput applications

DYNAMODB – NOSQL KEY-VALUE & DOCUMENT
- Fully managed NoSQL database
- Single-digit millisecond latency at any scale
- Ideal for high scale, low latency applications
- Key-value and document data model

ELASTICACHE – IN-MEMORY DATA STORE
- Managed Redis or Memcached
- Improves application performance
- Ideal for caching, sessions, leaderboards, queues, and real-time analytics

REDSHIFT – DATA WAREHOUSE
- Petabyte-scale data warehouse
- Optimized for analytics and BI
- Columnar storage for fast queries
- Ideal for large scale reporting & analytics

2. DATABASE SELECTION FRAMEWORK



Tip: Start simple. You can migrate or replicate data as your needs evolve.

3. DATABASE COMPARISON AT A GLANCE

FEATURE	RDS	AURORA	DYNAMODB	ELASTICACHE	REDSHIFT
Database Type	Relational (SQL)	Relational (SQL)	NoSQL (Key-Value / Document)	In-Memory Cache	Data Warehouse
Engines Supported	MySQL, PostgreSQL, MariaDB, Oracle, SQL Server	MySQL, PostgreSQL	N/A	Redis, Memcached	N/A
Scaling	Vertical (Scale Up) Read Replicas	Auto Scaling Storage Serverless v2	Horizontal (On-Demand) Virtually Unlimited	Horizontal (Shards) Multi-AZ	Horizontal (Clusters) Multi-AZ
Performance	Good	Excellent (Up to 5x RDS)	Excellent (Single-digit ms)	Excellent (Microseconds)	Excellent for Analytics
Use Cases	Web Apps, CMS, ERP, CRM, Traditional Apps	High Throughput Apps, SaaS, E-commerce, IoT	Gaming, IoT, Mobile Apps, Session Store, Shopping Cart	Caching, Sessions, Leaderboards, Queues	BI, Reporting, Data Mining, ETL, Analytics
Durability	99.99%	99.99% (6 copies)	99.999999999% (11 9's)	Data in Memory (Durable with Multi-AZ)	99.9%
Pricing Model	Per Instance + Storage	Per Capacity Unit + Storage	Pay per Request (On-Demand)	Per Node (Hourly)	Per Node Hour + Storage

4. DATABASE BEST PRACTICES

- ☑ Choose the simplest database that meets your requirements.
- ☑ Use Multi-AZ for high availability and disaster recovery.
- ☑ Enable automated backups and test restoration regularly.
- ☑ Encrypt data at rest and in transit.
- ☑ Monitor performance using CloudWatch and Performance Insights.
- ☑ Use read replicas or caching to offload read traffic.
- ☑ Review and optimize queries and indexes regularly.
- ☑ Apply least privilege access using IAM and security groups.
- ☑ Plan capacity and scaling based on growth projections.

5. COMMON USE CASE MAPPING

USE CASE	RECOMMENDED SERVICE
E-commerce Platform	Aurora (for transactions) + ElastiCache + DynamoDB (for sessions)
Mobile App Backend	DynamoDB + ElastiCache
Real-time Analytics Dashboard	Redshift + S3 + QuickSight
Content Management System	RDS (MySQL / PostgreSQL)
IoT Data Ingestion	DynamoDB + Kinesis + Redshift
Caching Layer	ElastiCache (Redis)
Data Warehousing	Redshift

SENIOR ENGINEER NOTE: Database selection impacts your entire architecture. Consider performance, consistency, scalability, cost, and operational complexity before committing. Re-evaluate as your workload evolves.

CACHING ARCHITECTURE

Caching improves performance, reduces latency, decreases cost, and offloads backend systems.

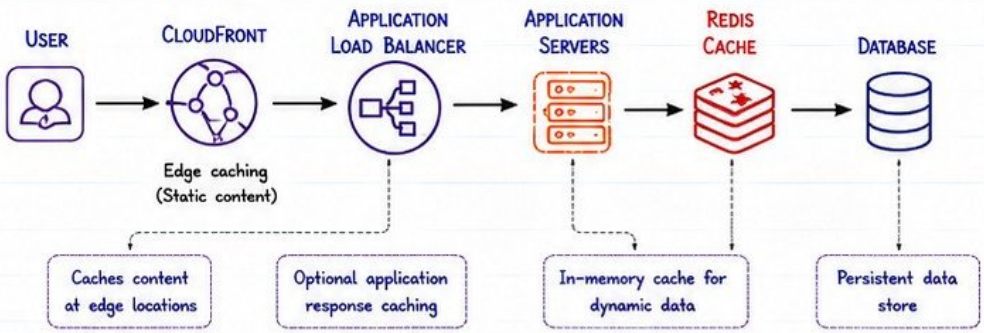
1. CACHING OVERVIEW

Caching stores frequently accessed data closer to users or applications to reduce response time and backend load.

BENEFITS

- ☑ Faster response time
- ☑ Reduced database load
- ☑ Lower bandwidth usage
- ☑ Improved scalability
- ☑ Cost optimization

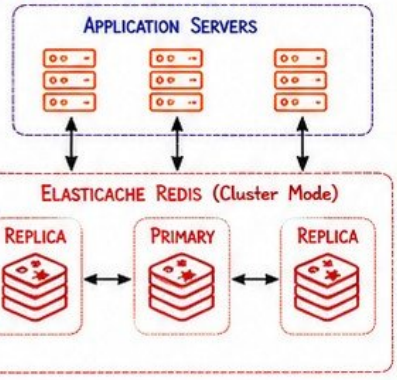
2. CACHING ARCHITECTURE LAYERS



Use multiple caching layers together for maximum performance.

3. REDIS (ELASTICACHE) ARCHITECTURE

- In-memory data store
- Microsecond latency
- Supports data structures
- High throughput
- Replication & failover

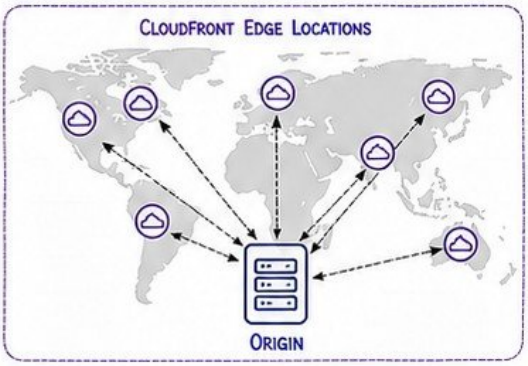


USE CASES

- ☑ Session store
- ☑ Shopping carts
- ☑ Leaderboards
- ☑ Rate limiting
- ☑ Real-time analytics
- ☑ API response caching

4. CLOUDFRONT (CONTENT DELIVERY NETWORK)

- Caches content at edge locations globally
- Reduces latency for static and dynamic content
- Integrates with S3, ALB, and custom origins
- Supports HTTPS, compression, and Edge Functions



5. APPLICATION CACHING PATTERNS

PATTERN	DESCRIPTION	EXAMPLE
Cache-Aside	Application checks cache first, then database on miss.	User profiles, product details
Read-Through	Cache handles data loading on cache misses.	Enterprise apps, CMS systems
Write-Through	Writes go to cache and database simultaneously.	User settings, configuration
Write-Behind	Writes go to cache first, then async to database.	Analytics events, logs
Refresh-Ahead	Cache refreshes before expiration to avoid misses.	Trending data, leaderboards

6. CACHE INVALIDATION STRATEGIES

STRATEGY	DESCRIPTION	BEST FOR
Time To Live (TTL)	Data expires after a set time automatically.	News, feeds, reference data
Event-Based Invalidation	Invalidate cache when data changes.	User updates, inventory
Versioning	Change version number to invalidate old cache.	APIs, static assets, configurations
Manual Invalidation	Explicitly clear cache when needed.	Admin actions, critical updates
Soft Expiration	Serve stale data while refreshing in background.	High traffic apps, APIs

7. CACHING BEST PRACTICES

- ☑ Cache only what is frequently accessed.
- ☑ Use appropriate TTLs based on data volatility.
- ☑ Implement cache hierarchy (browser → CDN → Application → Database).
- ☑ Monitor cache hit/miss ratio and adjust.
- ☑ Use compression and Gzip for better performance.
- ☑ Secure cache data and restrict access.

8. COMMON PITFALLS

- ☑ Caching too much invalid data.
- ☑ Not setting proper TTLs.
- ☑ Ignoring cache warming for critical data.
- ☑ Cache stampede (thundering herd) during expiration.
- ☑ No monitoring or alerts for cache performance.

SENIOR ENGINEER TIP

Design caching with consistency, performance, and cost in mind.
The right cache strategy can make or break your application at scale.

IDENTITY ARCHITECTURE

Secure access. Control permissions.
Protect your AWS environment.

1. IDENTITY ARCHITECTURE OVERVIEW

AWS Identity and Access Management (IAM) lets you securely control access to AWS resources.

IAM PRINCIPLES

- ☑ Verify explicitly
- ☑ Use least privilege
- ☑ Enable MFA
- ☑ Centralize management
- ☑ Monitor and audit

2. IDENTITY ARCHITECTURE FLOW

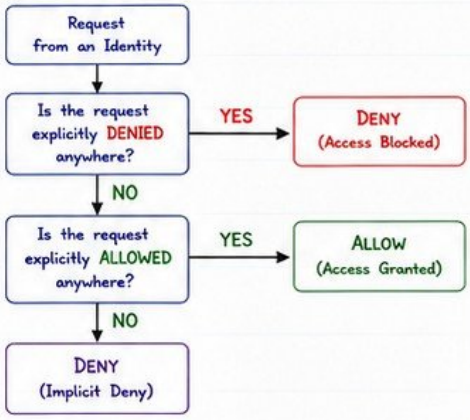
Identities	Authentication	Authorization	AWS Resources	Monitoring
Users Groups Roles Applications	Password MFA Federation SSO	Policies Permissions Conditions Boundaries	EC2, S3, RDS, Lambda, VPC and more	CloudTrail Access Analyzer CloudWatch

The right identity architecture ensures the right people have the right access for the right reason.

3. CORE IAM COMPONENTS

	USERS	• Represent individuals • Have long-term credentials • Assigned to groups or roles
	GROUPS	• Collection of users • Permissions managed at group level • Simplifies administration
	ROLES	• Assumed by users, services, or applications • Short-term credentials • Enable cross-account access
	POLICIES	• JSON documents that define permissions • Attached to users, groups, or roles • Allow or deny actions on resources
	PERMISSION BOUNDARIES	• Set the maximum permissions an identity can have • Prevents escalation beyond defined boundary • Acts as a safety guardrail
	MFA	• Adds extra layer of security • Required for console and sensitive actions • Protects against credential compromise

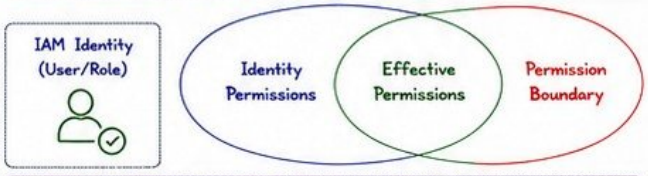
4. IAM POLICY EVALUATION LOGIC



KEY TAKEAWAY

Explicit Deny always overrides Allow.
If no Allow is found, access is denied by default.

5. PERMISSION BOUNDARIES EXPLAINED



The effective permissions are the intersection of Identity permissions and Permission Boundary.

6. LEAST PRIVILEGE BEST PRACTICES

- ☑ Grant only the permissions required to perform a task.
- ☑ Avoid wildcard (*) permissions.
- ☑ Use conditions (aws:SourceIp, aws:MultiFactorAuthPresent, etc.).
- ☑ Use roles for temporary access, not long-term credentials.
- ☑ Review and audit permissions regularly.
- ☑ Enable MFA for all users.
- ☑ Use permission boundaries to prevent privilege escalation.
- ☑ Remove unused users, roles, and access keys.

LEAST PRIVILEGE = STRONG SECURITY

7. IAM POLICY EXAMPLE (S3 READ-ONLY)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["s3:ListBucket", "s3:GetObject"],
      "Resource": [
        "arn:aws:s3:::my-bucket",
        "arn:aws:s3:::my-bucket/*"
      ]
    }
  ]
}
```

THIS POLICY ALLOWS:

- ☑ List the bucket
- ☑ Read objects in the bucket

DOES NOT ALLOW:

- ☒ Write objects
- ☒ Delete objects

8. ACCESS LEVELS GUIDELINES

ACCESS LEVEL	DESCRIPTION	EXAMPLES	WHEN TO USE
ADMIN ACCESS	Full access to manage resources	AdministratorAccess	Break-glass access Only for admins
POWER USER	Manage most resources	PowerUserAccess	DevOps, senior engineers
READ-ONLY	View and monitor resources	ReadOnlyAccess	Auditors, analysts, reporting tools
SERVICE ROLES	Access for AWS services	EC2Role, LambdaRole	Enable services to work securely

SENIOR ENGINEER TIP: Design identity with security in mind. Start with no access, add only what is needed, and continuously review. Great architecture is secure by default.

Secrets and Key Management

Protect sensitive data and encryption keys using AWS managed services and best practices.

1. Secrets and Key Management Overview

Securely store, rotate, and control access to secrets, API keys, certificates, and encryption keys across your AWS environment.

AWS Secrets Manager

- Store, rotate, and manage secrets
- Automatic rotation for supported services
- Fine-grained access control
- Audit with CloudTrail

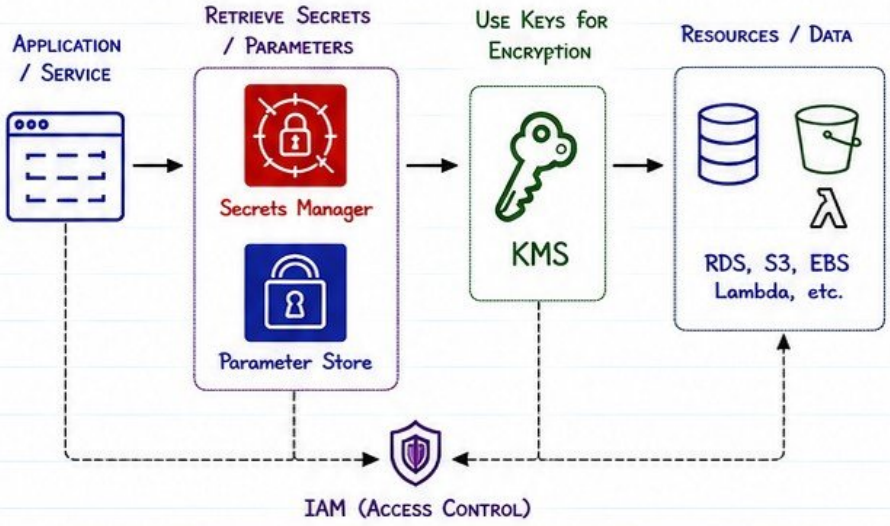
AWS Systems Manager Parameter Store

- Store configuration and secrets
- Hierarchical structure
- SecureString for encrypted values
- Integration with IAM and KMS

AWS Key Management Service (KMS)

- Create and manage encryption keys
- Encrypt and decrypt data
- Key rotation and key policies
- Integrated with AWS services

2. Secrets and Keys Architecture



Applications never hardcode secrets or keys.
They retrieve securely at runtime with least privilege.

3. Service Comparison

Feature	Secrets Manager	Parameter Store	KMS
Primary Use	Secrets (DB creds, API keys, tokens)	Configuration & parameters	Encryption keys
Data Type	Secrets (text/binary)	String / SecureString	Keys (AES, RSA, ECC)
Encryption	KMS encrypted (by default)	SecureString uses KMS	Uses KMS keys
Automatic Rotation	Yes (for supported services)	No	Key rotation (yearly)
Versioning	Yes	Yes	Key versions
Cost	$ per secret + API calls	Standard: Free Advanced: $ per param	$ per key + API calls
Integration	RDS, Redshift, Lambda, DocumentDB, etc.	EC2, Lambda, AppConfig, CloudFormation, etc.	Integrated with all AWS services

4. Encryption Strategy Framework

1	Classify Data	Identify sensitive data (PII, credentials, financial, intellectual property).
2	Choose Encryption Type	At rest, in transit, or both.
3	Select KMS Keys	Use customer managed keys (CMK) for sensitive workloads.
4	Encrypt Data	Use KMS APIs or service integrations.
5	Manage Access	Apply least privilege IAM policies and key policies.
6	Monitor and Audit	Enable CloudTrail, CloudWatch, and AWS Config.
7	Rotate and Review	Rotate keys and secrets regularly and review access.

Encryption Principle
Encrypt everything, everywhere, all the time.

5. Common Use Cases

Database Credentials	Store and rotate database credentials for RDS, Aurora, DocumentDB.
API Keys and Tokens	Store API keys, tokens, and OAuth credentials safely.
Application Configuration	Store environment variables, feature flags, and configuration values.
SSL/TLS Certificates	Store private keys and certificates for secure communications.
Encryption Keys	Manage keys for S3, EBS, RDS, Lambda, and custom applications.

6. Best Practices

- [x] Use Secrets Manager for secrets that need rotation.
- [x] Use Parameter Store for configuration and non-rotating secrets.
- [x] Always use customer managed KMS keys for sensitive data.
- [x] Apply least privilege IAM policies.
- [x] Enable CloudTrail logging for audit and compliance.
- [x] Rotate secrets and keys regularly.
- [x] Avoid storing secrets in code, environment variables, or plain text.
- [x] Use encryption in transit (TLS) and at rest.
- [x] Tag secrets, parameters, and keys for better management.

Senior Engineer Tip

Good security is invisible.
Secure your secrets and keys today so incidents never happen tomorrow.

Application Integration

Build reliable, scalable, and decoupled applications using AWS messaging, events, and workflows.

1. Integration Services Overview

Amazon SQS (Simple Queue Service)

- Fully managed message queue service
- Decouples and scales microservices
- Standard and FIFO queues
- At-least-once delivery

Amazon SNS (Simple Notification Service)

- Publish/subscribe messaging service
- Fan-out messages to multiple subscribers
- Push, Email, SMS, HTTP, SQS, Lambda
- Decouples publishers from subscribers

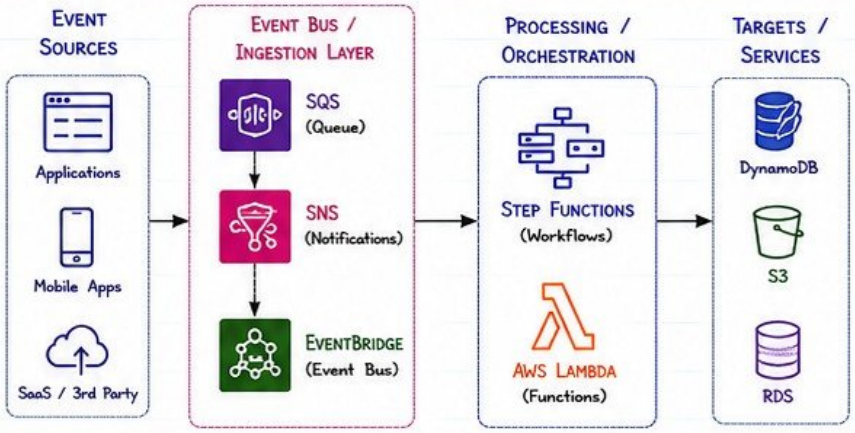
Amazon EventBridge

- Serverless event bus for applications
- Route events to targets at scale
- SaaS integrations and custom buses
- Event filtering and transformation

AWS Step Functions

- Orchestrate workflows with serverless states
- Visual workflow designer
- Built-in retries, error handling, timeouts
- Integrates with 200+ AWS services

2. Event-Driven Architecture Pattern



Benefits of Event-Driven Architecture

- ☑ Decouples producers and consumers
- ☑ Scales automatically with demand
- ☑ Improves resilience and fault isolation
- ☑ Enables real-time processing
- ☑ Reduces latency and improves agility
- ☑ Lower operational overhead

3. Service Comparison

Feature	SQS	SNS	EventBridge	Step Functions
Type	Message Queue	Pub/Sub Messaging	Event Bus	Workflow Orchestration
Communication	Point-to-Point	One-to-Many (Fan-out)	One-to-Many (Routing)	State Machine Execution
Delivery	At-least-once	At-least-once	At-least-once	At-least-once (per state)
Ordering	Standard: Best-effort FIFO: Exactly-once	Best-effort	Best-effort	Controlled by workflow
Retention	Up to 14 days	—	24 hours (default)	Execution history: 90 days
Integration	Pull-based	Push-based	Event-driven	Orchestration / Control
Best For	Decoupling & buffering	Broadcast notifications	Event routing & Integration	Complex workflows
Pricing	Per request + data	Per request + delivery	Per event	Per state transition

4. Integration Flow Example

1. Application generates an event.
2. Event sent to SQS queue.
3. SNS publishes notification to multiple subscribers.
4. EventBridge routes event to appropriate targets.
5. Step Functions orchestrates multi-step workflow.
6. Services process the event and store results.

5. Common Use Cases

Microservice Communication

Use SQS to decouple services and handle spikes.

Broadcast Notifications

Use SNS to notify users via email, SMS, or push.

Event Routing & Integration

Use EventBridge to integrate SaaS and AWS events.

Business Workflows

Use Step Functions for approvals, ETL, and orchestration.

Real-Time Data Pipelines

Combine services to build real-time event-driven pipelines.

6. Best Practices

- ☑ Design for decoupling and loose coupling.
- ☑ Use appropriate service for the use case.
- ☑ Implement retries and dead-letter queues.
- ☑ Use idempotent consumers.
- ☑ Monitor and alert on failed messages.
- ☑ Keep messages small and focused.
- ☑ Use EventBridge for centralized event routing.
- ☑ Use Step Functions for complex business workflows.
- ☑ Tag resources and follow naming standards.

Key Takeaway

Event-driven architecture enables scalable, reliable, and resilient systems by decoupling components and reacting to events in real time.

VERIQTA

Senior Engineer Tip: Design for failure. Use queues, retries, and DLQs. Events are your contract. Keep them consistent and versioned.

VERIQTA

DNS Architecture

Scalable, highly available, and cost-effective DNS management with Amazon Route 53.

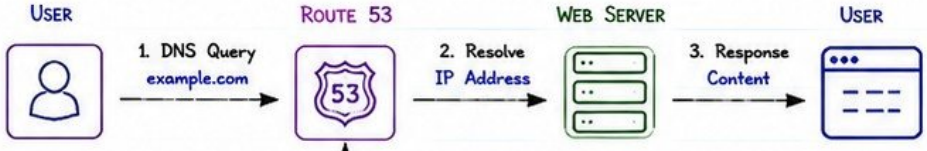
1. DNS Architecture Overview

DNS translates domain names to IP addresses. Amazon Route 53 is a highly available and scalable DNS web service.

Amazon Route 53

- ☑ Highly available and scalable
- ☑ 100% SLA for DNS queries
- ☑ Global anycast network
- ☑ Integrated with AWS services
- ☑ Health checks and routing policies
- ☑ Domain registration and management

2. DNS Resolution Flow



Route 53 Global Anycast Network

DNS queries are routed to the nearest edge location for fast and reliable responses.

DNS is the first step in every user request. Make it fast, reliable, and resilient.

3. Route 53 Components

Hosted Zones	Containers for DNS records that define how to route traffic for a domain.
Health Checks	Monitor the health of your resources and endpoints.
Traffic Policies	Define how Route 53 routes DNS queries based on conditions.
Domain Registration	Register and manage domain names directly in Route 53.
Query Logging	Log DNS queries for monitoring and analysis.

4. Routing Policies

Policy	Description	Use Case	Diagram
Simple	Routes traffic to a single resource.	Single resource, basic setups	
Failover	Routes to primary resource. If unhealthy, routes to secondary.	High availability, disaster recovery	
Latency	Routes to the lowest latency endpoint based on user location.	Global apps, performance optimization	
Geo Location	Routes based on the geographic location of the user.	Content localization, compliance	
Geo Proximity	Routes based on geographic location and resource health.	Global traffic management with failover	

5. Hosted Zone Types

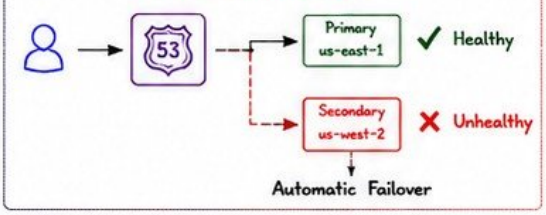
Public Hosted Zone

- ☑ Resolves queries on the internet
- ☑ Accessible to public users
- ☑ Used for public websites and APIs
- ☑ Example: example.com

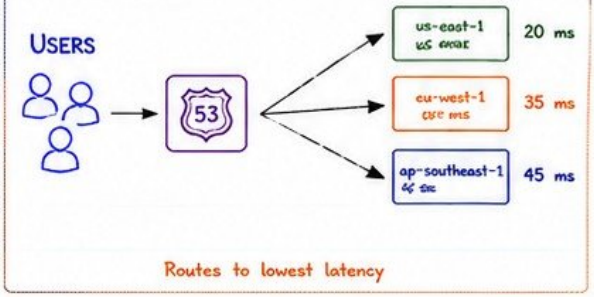
Private Hosted Zone

- ☑ Resolves queries within VPC only
- ☑ Not accessible on the internet
- ☑ Used for internal applications
- ☑ Example: internal.example.com

Failover Routing Example



6. Failover and Latency Examples



Key Takeaways

- ☑ Use Route 53 for highly available DNS.
- ☑ Choose the right routing policy for your use case.
- ☑ Monitor health checks regularly.
- ☑ Plan for failure from the start.

Senior Engineer Tip: Your DNS architecture is critical to user experience and system reliability. Design for performance, resilience, and simplicity.

Content Delivery

Deliver content to users globally with low latency, high availability, and maximum performance using Amazon CloudFront.

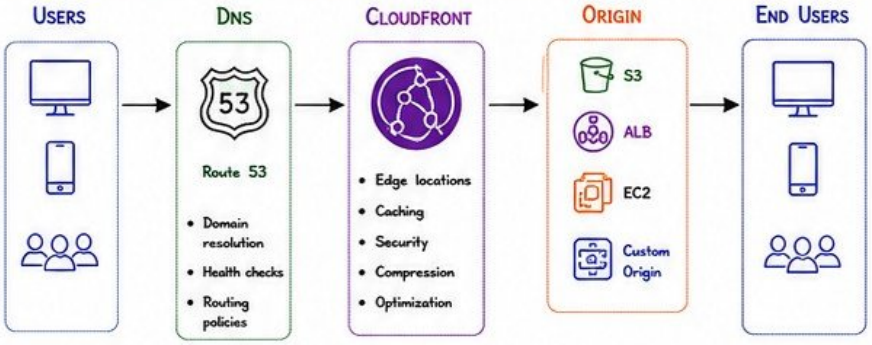
1. Content Delivery Overview

Amazon CloudFront is a global content delivery network (CDN) that accelerates the delivery of your static and dynamic content, APIs, and streaming media.

Key Benefits

- Low latency and high transfer speeds
- Global edge locations
- DDoS protection and security
- Scales automatically
- Cost-effective performance

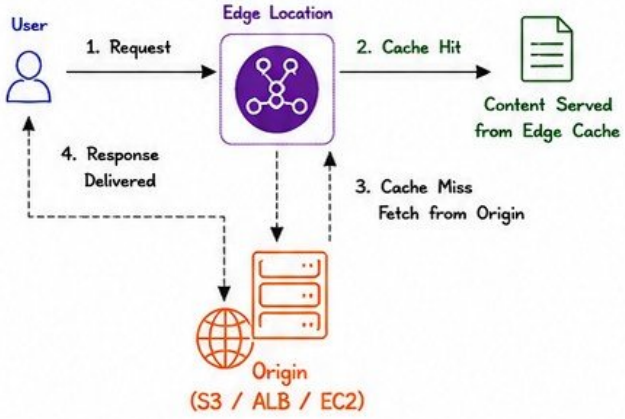
2. CloudFront Architecture



3. CloudFront Components

Distribution	The CloudFront distribution is the main entity that defines how content is delivered.
Edge Locations	CloudFront has 400+ edge locations worldwide to cache and deliver content.
Cache	Content is cached at edge locations to reduce latency and origin load.
Behavior	Defines how requests are handled based on path patterns, methods, and headers.
Origin	The source of the content (S3, ALB, EC2, or custom server).
SSL/TLS	Secure content delivery using ACM certificates with HTTPS.

4. Edge Caching Explained



Cache Rules

- Content cached based on behavior settings and TTL.
- Cache reduces origin load and improves performance.
- Invalidate or expire content to refresh cache.

5. Origin Design Options

Origin Type	Use Case	Considerations
S3 Origin	Static websites, files, images, videos	Best for static content, highly scalable
ALB Origin	Dynamic web applications, APIs	Supports load balancing, health checks
EC2 Origin	Custom applications, legacy systems	Full control, manage instances
Custom Origin	On-premises or 3rd-party applications	Use when other origin types not applicable

6. Performance Optimization

- ☑ Enable compression (Gzip / Brotli).
- ☑ Use appropriate TTL values for caching.
- ☑ Optimize images and assets before upload.
- ☑ Use HTTP/2 and HTTP/3 for faster delivery.
- ☑ Enable TLS 1.2 or higher.
- ☑ Use Origin Shield to reduce origin load.
- ☑ Configure cache key settings carefully.
- ☑ Minimize redirects and unnecessary requests.
- ☑ Use separate distributions for different use cases.
- ☑ Monitor performance with CloudWatch and real-user metrics.

Tip

The right caching strategy can reduce latency by 50% or more and significantly lower your costs.

7. Content Delivery Best Practices

- ☑ Use CloudFront with S3 for static content.
- ☑ Implement custom error pages for better user experience.
- ☑ Enable WAF and geo-restrictions for security.
- ☑ Use signed URLs or signed cookies for private content.
- ☑ Invalidate cache after content updates.
- ☑ Monitor metrics and set up alarms.

8. Common Use Cases

Static Websites	Deliver fast and secure static websites globally.
Video Streaming	Stream video content with low latency and high availability.
API Acceleration	Accelerate API responses and reduce server load.
Software Downloads	Deliver large files and updates quickly and reliably.

Key Takeaways

- CloudFront brings content closer to users.
- Edge caching improves speed and reduces origin costs.
- Proper origin and cache design is critical.
- Monitor, optimize, and secure your distributions.

Senior Engineer Tip: Design for performance, security, and cost. Use caching and optimization to ensure users get your content fast, every time, everywhere.

High Availability Patterns

Design resilient applications that remain available during failures, maintenance, and disasters.

1. High Availability Overview

High availability ensures your application is accessible with minimal or no downtime by eliminating single points of failure and using redundancy.

Core Principles

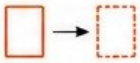
- Eliminate single points of failure
- Distribute across multiple Availability Zones
- Monitor health and fail fast
- Design for recovery and self-healing
- Test failover regularly

2. High Availability Patterns Overview

Multi-AZ

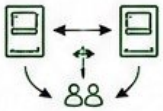
Distribute resources across multiple Availability Zones to protect against AZ failures.

Active-Passive



One environment handles traffic (active) while the other stays on standby (passive).

Active-Active



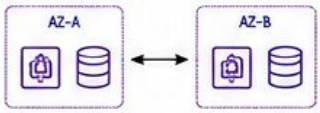
Multiple environments serve live traffic simultaneously for maximum availability and scalability.

Stateless Design

Build stateless applications so any instance can serve any request.

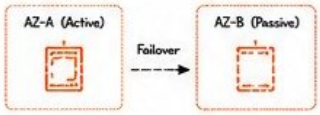
3. Pattern Diagrams

Multi-AZ



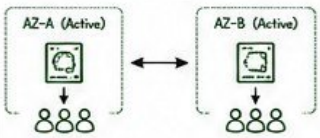
- Resources in multiple AZs
- Automatic failover
- Load balancer across AZs
- Data replicated

Active-Passive



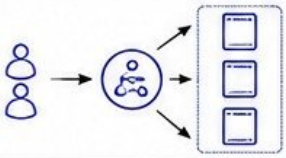
- One active environment
- Standby in another AZ
- Manual or automatic failover
- Used for critical systems

Active-Active



- Both environments live
- Traffic distributed
- Global load balancing
- Higher cost, higher availability

Stateless



- No session or state on server
- Externalize state to databases/caches
- Easy scaling and failover

4. Pattern Comparison

Pattern	Availability	Complexity	Cost	Use Cases
Multi-AZ	High	Medium	Medium	Most applications, databases, web services
Active-Passive	High (during failover)	Medium	Low–Medium	Disaster recovery, critical workloads
Active-Active	Very High	High	High	Global apps, low latency, high traffic
Stateless Design	High	Medium	Low–Medium	Web apps, APIs, microservices

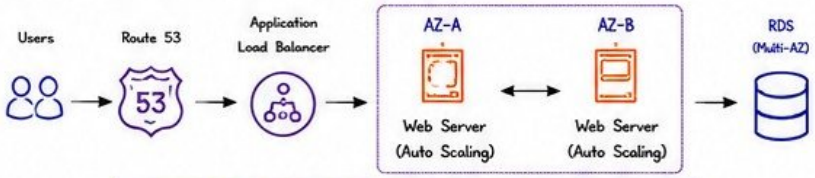
5. Key Components for High Availability

Load Balancer	Distributes traffic across healthy instances in multiple AZs.
Health Checks	Continuously monitor instances and remove unhealthy targets automatically.
Automated Failover	Auto scaling, Route 53 health checks, and failover routing ensure quick recovery.
Backup and Replication	Replicate data across AZs and regions for durability and availability.

6. Design Considerations

- ☑ Use at least two Availability Zones.
- ☑ Distribute compute, database, and storage.
- ☑ Avoid cross-AZ data transfer for performance-critical paths.
- ☑ Use Auto Scaling groups across multiple AZs.
- ☑ Store state in external services (RDS, DynamoDB, ElastiCache, S3).
- ☑ Implement connection draining during deployments.
- ☑ Test failure scenarios and validate recovery.
- ☑ Monitor and alert on failures.

7. Example: Multi-AZ Web Application



If AZ-A fails, traffic automatically routes to AZ-B. Application remains available with no downtime.

8. Best Practices

- ☑ Design for failure, not just for deployment.
- ☑ Automate everything: scaling, failover, alerts.
- ☑ Keep components loosely coupled.
- ☑ Prefer managed services over self-managed.
- ☑ Continuously test and improve your HA strategy.

TIP

High availability is a design choice, not a feature. Plan for it from day one.

9. Common Use Cases

Web Applications
Ensure websites remain available 24/7.

E-Commerce
Handle traffic spikes and prevent downtime.

Financial Systems
Protect critical transactions and data.

Media & Streaming
Deliver content globally with high reliability.

Senior Engineer Tip: High availability is not zero downtime, it is minimizing downtime. Measure, monitor, and continuously improve your resilience.

Disaster Recovery

Prepare for the worst, recover fast, and keep your business running.

1. Disaster Recovery Overview

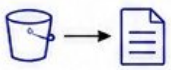
Disaster Recovery (DR) ensures your business can continue operating after a major disruption such as region outage, natural disaster, or security incident.

DR Goals

- Protect critical data and systems
- Minimize downtime and data loss
- Restore services quickly and reliably
- Maintain business continuity

2. Disaster Recovery Strategies

Backup and Restore



Restore data and infrastructure after disruption. Lowest cost, highest RTO.

Use when: Workloads can tolerate longer recovery times.

Pilot Light

Minimal core resources running in DR region. Scaled up when needed.

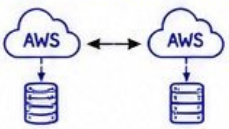
Use when: Workloads need faster recovery with lower cost.

Warm Standby

Scaled-down version of production running in DR region.

Use when: Workloads need quick recovery with moderate cost.

Multi-Region (Active-Active)



Fully active in multiple regions. Traffic served from multiple locations.

Use when: Workloads require highest availability and lowest RTO.

RPO = How much data you can afford to lose (Recovery Point Objective)
RTO = How fast you must recover (Recovery Time Objective)

3. Strategy Comparison

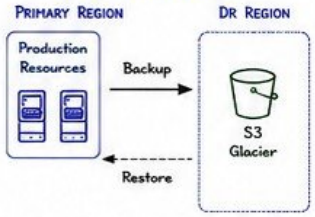
Strategy	RPO	RTO	Cost	Complexity	Use Cases
Backup and Restore	Hours to Days	Hours to Days	Low	Low	Non-critical workloads, archives, dev/test
Pilot Light	Minutes to Hours	Minutes to Hours	Low-Medium	Medium	Production workloads with moderate RTO
Warm Standby	Minutes	Minutes	Medium-High	Medium-High	Critical workloads needing fast recovery
Multi-Region (Active-Active)	Seconds	Seconds	High	High	Mission-critical applications, global apps

4. Recovery Workflow

1	Detect	Monitor and detect incident or failure.
2	Assess	Determine impact, priorities, and recovery strategy.
3	Activate	Trigger DR plan and failover or restore.
4	Validate	Verify systems, data integrity, and connectivity.
5	Recover	Resume normal operations and communicate.

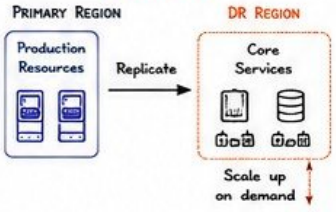
5. Architecture Examples

Backup and Restore



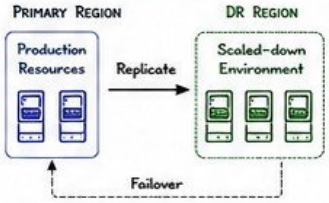
Data backed up to S3/Glacier and restored on recovery.

Pilot Light



Core components running in DR. Scale when needed.

Warm Standby



Reduced-scale environment ready for fast failover.

Multi-Region (Active-Active)

Traffic distributed across regions for high availability and low latency.

6. Key Components for DR

- **AWS Backup** — Centralized backup management and automation.
- **S3 Cross-Region Replication** — Replicate data across regions automatically.
- **Amazon Route 53** — Global DNS routing and health checks.
- **AWS CloudWatch** — Monitoring, alarms, and incident detection.
- **AWS Systems Manager** — Automate recovery with runbooks and automation.

Best Practices

- [x] Define and document DR plan.
- [x] Test DR plan regularly.
- [x] Automate backups and failover.
- [x] Monitor replication and health.
- [x] Keep runbooks up to date.
- [x] Communicate roles and responsibilities.
- [x] Review and improve continuously.

7. DR Readiness Checklist

- [x] Identify critical workloads and data.
- [x] Define RPO and RTO for each workload.
- [x] Choose the right DR strategy.
- [x] Implement automated backups and replication.
- [x] Configure health checks and monitoring.
- [x] Test failover and recovery regularly.
- [x] Validate data integrity after recovery.
- [x] Document runbooks and contacts.
- [x] Train teams and conduct disaster drills.
- [x] Review and update DR plan.

Tip

A good DR plan is tested, not just written. What you test you can trust in production.

Senior Engineer Tip: Disaster recovery is not just about technology. It is about process, people, and practice. Plan for failure, test often, and recover with confidence.

Well-Architected Framework

Build secure, reliable, efficient, and sustainable workloads using proven architectural best practices.

1. Well-Architected Overview

The AWS Well-Architected Framework helps you design and operate reliable, secure, efficient, cost-effective, and sustainable systems in the cloud.

Why it Matters

- ☑ Consistent approach to architecture
- ☑ Improves operational performance
- ☑ Reduces risk and technical debt
- ☑ Drives business value
- ☑ Supports continuous improvement

2. The Six Pillars

Operational Excellence	Security	Reliability	Performance Efficiency	Cost Optimization	Sustainability
Run and monitor systems to deliver business value and continually improve.	Protect systems, data, and assets while enabling business agility.	Ensure systems work as intended and recover quickly from failures.	Use computing resources efficiently to meet system requirements.	Avoid unnecessary costs and achieve maximum business value from cloud investments.	Minimize environmental impact and promote long-term ecological balance.

Use all six pillars together to make balanced architectural decisions that deliver long-term value.

3. Pillar Summary

Pillar	Key Objective
Operational Excellence	Run reliable operations, automate, and improve continuously.
Security	Protect data and systems, manage identity, and reduce risk.
Reliability	Design for availability, fault tolerance, and disaster recovery.
Performance Efficiency	Use the right resources and architectures for performance.
Cost Optimization	Reduce costs and maximize the value of cloud spending.
Sustainability	Reduce energy usage and adopt sustainable cloud practices.

4. Pillar Interactions

How They Work Together

- Strong security supports reliability and trust.
- Operational excellence drives efficiency and reduces cost.
- Performance efficiency improves user experience and lowers cost.
- Sustainability reduces cost and supports compliance.
- Balanced decisions create long-term business value.

5. Pillar Best Practices

Operational Excellence	Security	Reliability	Performance Efficiency	Cost Optimization	Sustainability
☑ Automate everything	☑ Apply least privilege	☑ Design for failure	☑ Choose the right resources	☑ Right-size resources	☑ Use efficient services and regions
☑ Monitor and alert	☑ Encrypt data in transit and at rest	☑ Use multi-AZ architectures	☑ Use caching and CDN	☑ Use reserved capacity	☑ Reduce over-provisioning
☑ Use IaC and version control	☑ Enable MFA	☑ Backup and test recovery	☑ Design scalable architectures	☑ Automate shutdown of unused resources	☑ Optimize storage lifecycle
☑ Document and share knowledge	☑ Use managed security services	☑ Monitor health	☑ Optimize code and queries	☑ Monitor cost and usage	☑ Monitor and reduce energy usage
☑ Continuously improve processes	☑ Review and audit regularly	☑ Automate failover	☑ Test performance regularly	☑ Review and optimize continuously	☑ Plan for long-term sustainability

6. Architectural Decision Guide

Operational Excellence	Can we operate this system easily and improve it over time?
Security	Is this system secure by design? Are risks identified and mitigated?
Reliability	Will this system remain available and recover quickly from failures?
Performance Efficiency	Does this design meet performance needs using optimal resources?
Cost Optimization	Is this the most cost-effective solution for the value provided?
Sustainability	Does this solution minimize environmental impact over the long term?

7. Continuous Improvement Cycle

Tip

Well-Architected is not a one-time review.

It is an ongoing journey of learning, measuring, and improving.

Key Takeaway

Use the Well-Architected Framework as your guide to design and operate systems that are secure, reliable, efficient, cost-effective, and sustainable—today and tomorrow.

Senior Engineer Tip: Great architects balance all six pillars. Optimize for business value, not just technology.

Security Architecture

Design secure, resilient, and trusted systems by applying proven security principles and patterns.

1. Security Architecture Overview

A strong security architecture protects data, systems, and workloads by applying layered controls, least privilege, continuous monitoring, and proactive defense.

Security Objectives

- ☑ Protect confidentiality, integrity, and availability
- ☑ Prevent threats and reduce attack surface
- ☑ Detect and respond to security events
- ☑ Ensure compliance and governance
- ☑ Enable business agility securely

2. Security Principles

Least Privilege	Defense in Depth	Zero Trust	Security by Design	Continuous Improvement
Grant only the minimum access necessary for users and services.	Use multiple layers of controls to protect resources at every level.	Never trust, always verify. Validate every request.	Build security into every layer of the architecture from the start.	Continuously monitor, assess, and improve security posture.

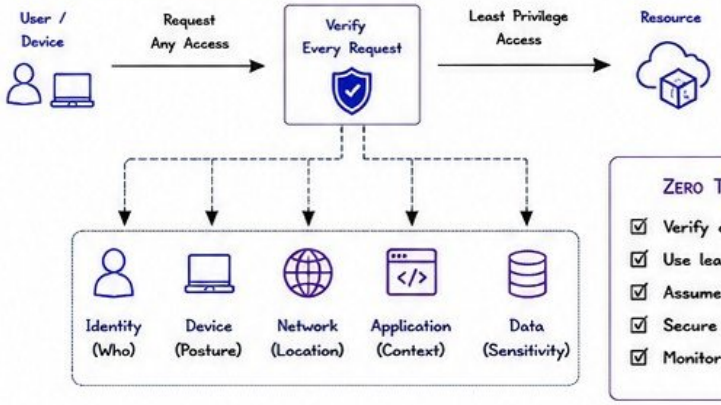
Security is not a product, it is an architecture and a process.

3. Defense in Depth (Layered Security)

Layer 1 Perimeter Security	AWS WAF, Shield, Route 53, CloudFront, Security Groups
Layer 2 Network Security	VPC, Subnets, NACLs, Network Segmentation, Private Endpoints
Layer 3 Workload Security	IAM Roles, Security Groups, Container Security, EKS Policies
Layer 4 Data Security	Encryption, KMS, Secrets Manager, Backups, Data Classification
Layer 5 Monitoring & Response	CloudTrail, CloudWatch, GuardDuty, Security Hub, SIEM, Incident Response

Compromise one layer, other layers still protect your assets.

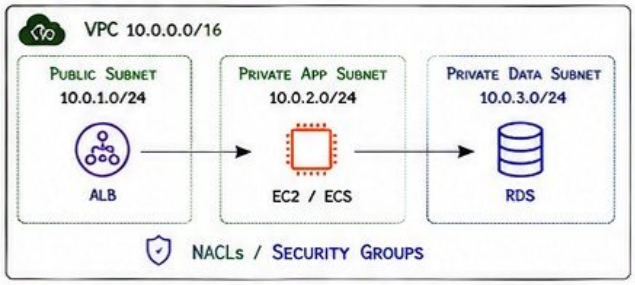
4. Zero Trust Architecture



Zero Trust Principles

- ☑ Verify explicitly
- ☑ Use least privilege access
- ☑ Assume breach
- ☑ Secure end-to-end
- ☑ Monitor continuously

5. Network Segmentation



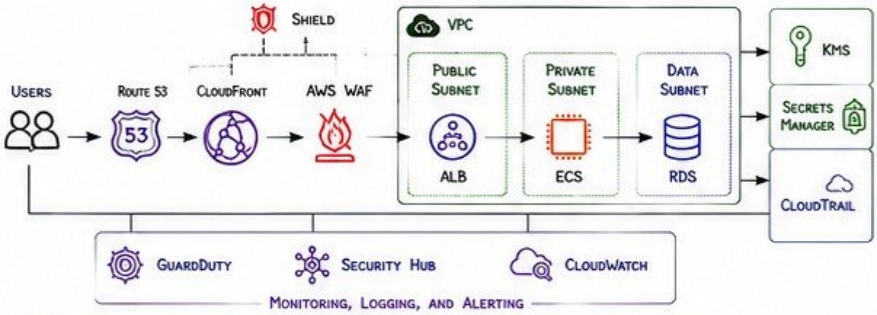
Benefits

- ☑ Reduce attack surface
- ☑ Isolate critical workloads
- ☑ Control traffic flow
- ☑ Limit lateral movement
- ☑ Improve compliance

6. Key Security Services

Service	Purpose
IAM	Manage identities and access
AWS WAF	Protect web applications from common attacks
Security Groups	Control inbound and outbound traffic
Network ACLs	Subnet-level traffic filtering
KMS	Manage encryption keys
Secrets Manager	Store and rotate secrets securely
GuardDuty	Detect threats using ML and anomaly detection
CloudTrail	Log and monitor API activity
Config	Track configuration changes and compliance
Security Hub	Centralized security posture management

7. Security Architecture Diagram (Example)



8. Security Best Practices

- ☑ Enable MFA for all users
- ☑ Use IAM roles instead of keys
- ☑ Encrypt data in transit and at rest
- ☑ Keep systems and dependencies updated
- ☑ Enable logging and monitoring everywhere
- ☑ Automate security checks and compliance
- ☑ Review access regularly
- ☑ Test incident response plans

Key Takeaway

A secure architecture is built on principles, not products. Layer your defenses, trust nothing by default, and monitor everything.

Senior Engineer Tip: Good security architecture does not stop attackers. It detects them early, limits damage, and helps you recover fast.

Logging Strategy

Centralized, secure, and actionable logging for visibility, compliance, and troubleshooting.

1. Logging Strategy Overview

A logging strategy ensures that all important events, activities, and changes are captured, stored securely, and made available for analysis and audit.

Logging Objectives

- ☑ Gain complete visibility into your environment
- ☑ Accelerate troubleshooting and incident response
- ☑ Meet compliance and audit requirements
- ☑ Detect threats and suspicious activities
- ☑ Support metrics, monitoring, and analytics
- ☑ Retain logs securely for the right duration

2. Key AWS Logging Services

CloudWatch Logs

Collects, monitors, and stores logs from AWS resources, applications, and custom sources.

Enables log insights and real-time analysis.

CloudTrail

Records API activity and account changes across your AWS account for audit, security, and governance.

VPC Flow Logs

Captures IP traffic flow information for network interfaces in your VPC.

Central Logging

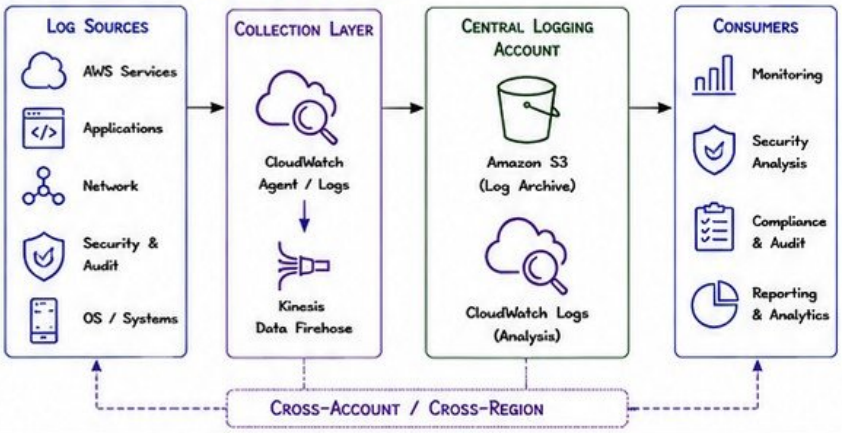
Aggregates logs from multiple accounts and regions into a central location for analysis and retention.

Collect the right logs, store them securely, and make them actionable.

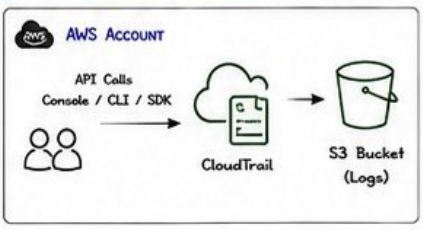
3. Log Sources Examples

Source Type	Examples
AWS Services	EC2, RDS, S3, Lambda, ECS, EKS, ALB, API Gateway, CloudFront
Applications	Application logs, error logs, access logs, audit logs
Network	VPC Flow Logs, Route 53 logs, Load Balancer access logs
Security & Audit	CloudTrail, GuardDuty, IAM Access Analyzer, Config
Operating System	Linux / Windows system logs, auth logs, syslog

4. Central Logging Architecture

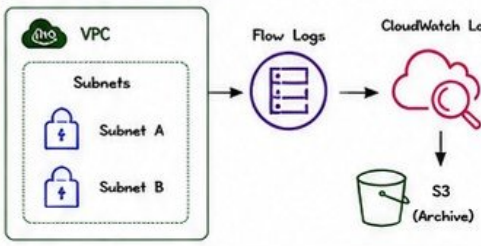


5. CloudTrail Logging



- ☑ Records management events and data events.
- ☑ Helps with security analysis and compliance.
- ☑ Enable in all regions.
- ☑ Store logs in a dedicated S3 bucket.

6. VPC Flow Logs



- ☑ Capture metadata about IP traffic.
- ☑ Helps with network monitoring and security.
- ☑ Store in CloudWatch Logs or S3.
- ☑ Analyze with Insights or Athena.

7. Log Retention Strategy

Log Type	Recommended Retention	Storage	Purpose
Application Logs	30 – 90 days	CloudWatch Logs	Troubleshooting
Security / Audit Logs	1 – 7 years	S3 (Glacier)	Compliance / Audit
VPC Flow Logs	30 – 90 days	S3 / CW Logs	Network Monitoring
CloudTrail Logs	1 – 7 years	S3 (Glacier)	Audit / Forensics
System Logs	30 – 180 days	CloudWatch Logs	Operations

Retain logs based on compliance requirements and business needs.

8. Logging Best Practices

- ☑ Centralize logs from all accounts and regions.
- ☑ Use consistent log formats (JSON recommended).
- ☑ Encrypt logs in transit and at rest.
- ☑ Restrict access using IAM least privilege.
- ☑ Use log retention policies and lifecycle rules.
- ☑ Monitor log delivery and set up alerts.
- ☑ Use CloudWatch Logs Insights for fast queries.
- ☑ Integrate logs with SIEM for advanced analysis.
- ☑ Regularly review and archive old logs.

Key Takeaway

A strong logging strategy gives you visibility, accelerates response, supports compliance, and turns data into actionable insights.

Senior Engineer Tip: If it's not logged, it didn't happen. Log important events, protect your logs, and use them to improve security, and performance.

VERIQTA

AWS Solutions Architect Production Handbook

Monitoring Architecture

Proactively monitor your systems, detect issues early, and deliver reliable, high-performance applications.

1. Monitoring Architecture Overview

Monitoring provides visibility into the health, performance, and availability of your applications and infrastructure. A well-designed monitoring architecture enables you to detect, understand, and respond to issues quickly.

Monitoring Goals

- Detect issues early
- Understand system behavior
- Improve performance
- Ensure availability
- Drive continuous improvement

2. Monitoring Components

Metrics	Alarms	Dashboards	SLOs
Numerical data collected over time to track performance and health.	Alerts triggered when metrics breach defined thresholds.	Visual views of metrics and system health in one place.	Service Level Objectives define reliability targets for your services.

Good monitoring turns data into insights and insights into action.

3. Metrics

Metrics are the foundation of monitoring. They provide quantitative data about your systems and applications.

Common Metrics

- ☑ CPU Utilization
- ☑ Memory Usage
- ☑ Disk I/O and Space
- ☑ Network In/Out
- ☑ Request Count
- ☑ Error Count
- ☑ Latency (Response Time)
- ☑ Throughput
- ☑ Queue Depth
- ☑ Database Connections

Best Practices

- ☑ Collect only what you need.
- ☑ Use meaningful names and units.
- ☑ Set baseline expectations.
- ☑ Monitor at the right granularity.

4. Alarms

Alarms notify you when something is wrong so you can take action before it impacts users.

Alarm States

OK
Metric is within the expected range.

Alarm
Metric is outside the threshold.

Insufficient Data
Not enough data to evaluate.

Best Practices

- ☑ Set meaningful thresholds.
- ☑ Reduce alarm noise.
- ☑ Use composite alarms.
- ☑ Route to the right people.

5. Dashboards

Dashboards provide a centralized, real-time view of system health and performance.

Best Practices

- ☑ Keep dashboards simple and focused.
- ☑ Show the most important metrics first.
- ☑ Use clear labels and consistent units.
- ☑ Create dashboards for different audiences.
- ☑ Review and update dashboards regularly.

Dashboard Examples

- Executive Overview
- Application Health
- Infrastructure Health
- Business Metrics
- Cost Overview

6. Service Level Objectives (SLOs)

SLOs define the reliability and performance targets for your services from the user's perspective.

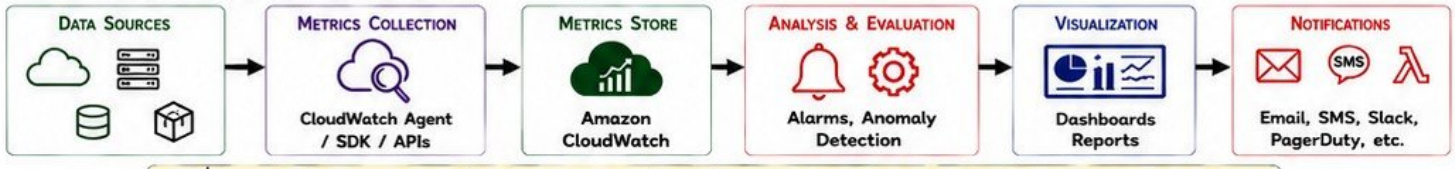
SLO Examples

- 99.9% API availability
- 95% of requests < 200ms
- Error rate < 0.1%
- Successful transactions > 99%

Best Practices

- ☑ Define SLOs based on user experience.
- ☑ Use real data to set achievable targets.
- ☑ Review and adjust regularly.
- ☑ Align SLOs with business priorities.

7. Monitoring Architecture Flow



Collect → Analyze → Visualize → Alert → Act → Improve

Senior Engineer Tip: Monitoring is not just about collecting data. It's about understanding your systems, acting on insights, and continuously improving reliability.

OBSERVABILITY

See what is happening, understand why it is happening, and act fast to deliver reliable systems.

1. OBSERVABILITY OVERVIEW

Observability goes beyond monitoring. It helps you understand the internal state of your systems by analyzing their outputs.

Three Pillars of Observability:
METRICS | LOGS | TRACES

OBSERVABILITY GOALS

- ☑ Understand system behavior
- ☑ Detect issues early
- ☑ Reduce mean time to resolution (MTTR)
- ☑ Improve user experience
- ☑ Build reliable, scalable systems

2. OBSERVABILITY PILLARS

METRICS	LOGS	TRACES
Numeric data measured over time to track performance and health.	Detailed events that record what happened in your system.	Request flows that show how a request travels across services.

Observability drives insights.
Insights drive faster, better decisions.
Better decisions drive better systems.

3. TRACING

Tracing shows the end-to-end journey of a request as it flows through multiple services.

KEY BENEFITS

- ☑ Understand request flow
- ☑ Identify latency bottlenecks
- ☑ Pinpoint failing components
- ☑ Analyze dependencies
- ☑ Improve performance

TRACE STRUCTURE

TRACE (Whole Request)
↓
SPANS (Operations)
↓
TAGS (Metadata)
↓
LOGS (Events)

4. DISTRIBUTED LOGGING

Distributed logging collects logs from all services and centralizes them for search, analysis, and alerting.

BEST PRACTICES

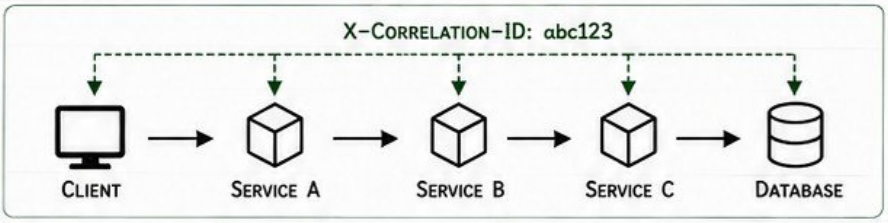
- ☑ Use structured logs (JSON)
- ☑ Include timestamps (UTC)
- ☑ Include log level
- ☑ Include service name
- ☑ Include correlation ID
- ☑ Centralize logs
- ☑ Set retention policies

LOG LEVELS

ERROR	Critical issues
WARN	Warning conditions
INFO	General information
DEBUG	Detailed debugging
TRACE	Very detailed traces

5. CORRELATION IDS

Correlation IDs connect logs, metrics, and traces across services for a single request, making debugging faster and easier.



BEST PRACTICES

- ☑ Generate ID at the edge (API Gateway / Load Balancer)
- ☑ Pass ID in headers (X-Correlation-ID)
- ☑ Log the ID in every service
- ☑ Use ID in traces and metrics

EXAMPLE LOG ENTRY

```
{
  "timestamp": "2025-08-02T12:30:45Z",
  "level": "INFO",
  "service": "payment-service",
  "message": "Payment processed successfully",
  "correlation_id": "abc123"
}
```

KEY TAKEAWAY

Observability is not just about collecting data. It is about connecting data to understand systems deeply and act with confidence.

SENIOR ENGINEER TIP: You can't fix what you can't see. Build observability in from day one. The best time to add it is before you need it.

CI/CD Architecture

Automate your delivery pipeline to build, test, secure, and deploy software faster and more reliably.

1. CI/CD Architecture Overview

CI/CD automates the software delivery lifecycle from code commit to production. It improves code quality, accelerates delivery, reduces risk, and enables continuous feedback.

CI/CD Goals

- ☑ Automate build, test, and deployment
- ☑ Deliver features faster and safer
- ☑ Ensure quality and security at every stage
- ☑ Enable repeatable and reliable releases
- ☑ Minimize manual steps and errors

2. CI/CD Pipeline Stages

1. Source	2. Build	3. Test	4. Deploy	5. Monitor
Code commit and version control	Compile code, resolve dependencies, and build artifacts	Run automated tests (unit, integration, security, etc.)	Deploy to test, staging, and production environments	Monitor applications and collect feedback

Automate everything, Monitor everything. Improve continuously.

3. AWS CodePipeline Architecture

Source	Build	Test	Deploy	Monitor
AWS CodeCommit	AWS CodeBuild	AWS CodeBuild / Third-party	AWS CodeDeploy	Amazon CloudWatch

Key Benefits

- ☑ Fully managed CI/CD service
- ☑ Native AWS integrations
- ☑ Secure and scalable
- ☑ Supports multiple accounts and regions

Best Practices

- ☑ Store artifacts in S3
- ☑ Use IAM roles with least privilege
- ☑ Enable manual approvals for production
- ☑ Integrate with CloudWatch for visibility

4. GitHub Actions Architecture

Code	Workflow	Jobs	Artifacts	Deploy
GitHub Repository	GitHub Actions	Runners (GitHub / Self-hosted)	Store & Share Artifacts	Deploy to Environments

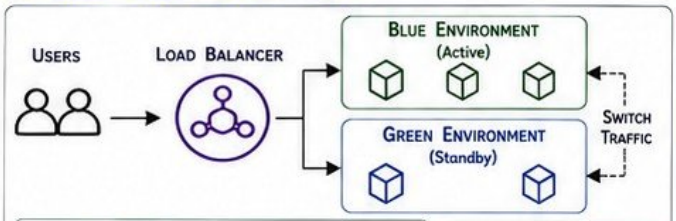
Key Benefits

- ☑ Highly flexible and powerful
- ☑ Large marketplace of actions
- ☑ Works with any cloud or platform
- ☑ Cost-effective

Best Practices

- ☑ Use reusable workflows
- ☑ Pin action versions
- ☑ Store secrets securely
- ☑ Use environments with approvals

5. Blue-Green Deployment



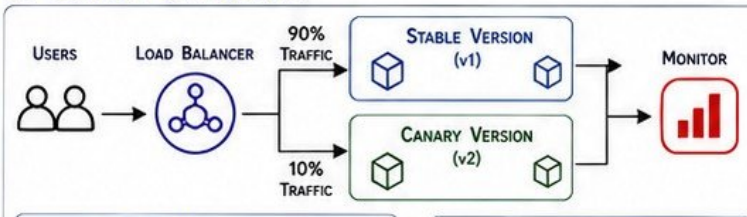
How it Works

- ☑ 1. Deploy new version to Green environment.
- ☑ 2. Test the new version.
- ☑ 3. Switch traffic from Blue to Green.
- ☑ 4. Keep Blue as standby or rollback if needed.

Benefits

- ☑ Zero downtime
- ☑ Easy rollback
- ☑ Full production environment testing

6. Canary Deployment



How it Works

- ☑ 1. Deploy new version to a small subset of users.
- ☑ 2. Monitor metrics and user behavior.
- ☑ 3. Gradually increase traffic if healthy.
- ☑ 4. Rollback quickly if issues detected.

Benefits

- ☑ Lower risk
- ☑ Early issue detection
- ☑ Data-driven rollout
- ☑ Better user experience

Senior Engineer Tip: Great CI/CD pipelines are fast, reliable, and secure. Automate everything you can. Measure everything that matters.

VERIQTA

VERIQTA

AWS Solutions Architect Production Handbook

Infrastructure as Code

Define, provision, and manage infrastructure through code for consistency, repeatability, and scalability.

1. Infrastructure As Code Overview

Infrastructure as Code (IaC) lets you manage and provision cloud resources using code instead of manual processes. It improves consistency, speeds up deployments, and reduces human error.

Key Benefits

- ☑ Consistency across environments
- ☑ Repeatable and reliable deployments
- ☑ Version controlled infrastructure
- ☑ Faster provisioning and updates
- ☑ Better collaboration and auditability
- ☑ Reduced manual errors

2. IaC Principles

Declarative	Idempotent	Version Controlled	Modular	Automated
Define the desired state. The tool figures out how to achieve it.	Apply code multiple times with the same result.	Store IaC in Git repositories for tracking, review, and rollback.	Reuse components with modules and templates.	Automate provisioning, updates, and teardowns.

Code it. Review it. Test it. Deploy it.
Infrastructure should follow the same standards as application code.

3. AWS CloudFormation

AWS CloudFormation is AWS-native IaC service that uses JSON or YAML templates to model and provision AWS resources.

Key Features

- ☑ AWS-native integration
- ☑ Stack creation and management
- ☑ Change sets for safe updates
- ☑ Rollback on failure
- ☑ Drift detection
- ☑ Nested stacks and modules

Template Example (YAML)

```yaml
Resources:
  MyBucket:
    Type: AWS::S3::Bucket
    Properties:
      BucketName: veriqta-demo-bucket
      VersioningConfiguration:
        Status: Enabled
```

4. Terraform

Terraform is an open-source IaC tool that supports multi-cloud (AWS, Azure, GCP, etc.) using its own HashiCorp Configuration Language (HCL).

Key Features

- ☑ Multi-cloud support
- ☑ Human-readable configuration
- ☑ Plan, apply, destroy workflow
- ☑ State management
- ☑ Drift detection
- ☑ Extensive provider ecosystem

Config Example (HCL)

```hcl
resource "aws_s3_bucket" "example" {
  bucket = "veriqta-demo-bucket"

  versioning {
    enabled = true
  }
}
```

5. Drift Detection

Drift occurs when the real-world infrastructure differs from the defined code. Drift detection helps you identify and fix unwanted changes.

Drift Detection Workflow

Why Drift Happens

- Manual changes in the console
- Emergency hotfixes
- Changes by other teams
- Forgotten temporary resources

Best Practices

- ☑ Enable drift detection regularly
- ☑ Integrate with CI/CD pipelines
- ☑ Alert on drift detection
- ☑ Enforce changes through IaC only
- ☑ Document and approve exceptions

Tools Supporting Drift Detection

AWS CloudFormation	Built-in drift detection for stacks
Terraform	terraform plan shows drift
Custom Solutions	AWS Config, Cloud Control API, or scripts

Keep your infrastructure aligned with your code. Detect drift early and fix it before it causes problems.

Senior Engineer Tip: Treat your infrastructure code like production application code. Code review, testing, versioning, and CI/CD are non-negotiable.

VERIQTA

VERIQTA
Multi-Account Strategy

Organize, secure, and scale your AWS environment using multiple accounts with centralized governance.

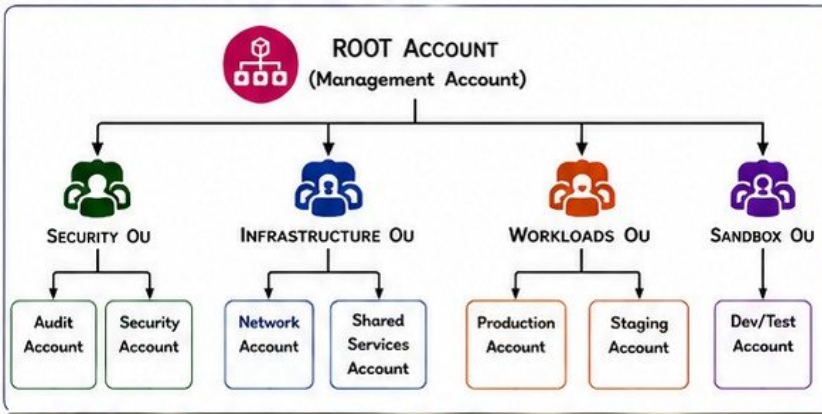
1. Multi-Account Strategy Overview

A multi-account strategy helps you isolate workloads, enforce security boundaries, simplify billing, and improve governance at scale. AWS Organizations is the foundation for managing multiple AWS accounts.

Key Benefits

- ☑ Strong security isolation
- ☑ Centralized governance and control
- ☑ Simplified billing and cost management
- ☑ Scalability for teams and workloads
- ☑ Compliance and audit readiness

2. AWS Organizations Structure



Use Organizational Units (OUs) to group accounts by purpose, apply policies, and manage access at scale.

3. Service Control Policies (SCPs)

SCPs allow you to set guardrails for your accounts. They define the maximum permissions an account can have within an OU.

Common SCP Use Cases

- ☑ Deny regions
- ☑ Restrict root user access
- ☑ Deny creation of IAM users
- ☑ Restrict services (e.g., disable EC2)
- ☑ Enforce tagging policies
- ☑ Prevent IAM policy changes

SCP Example (Deny Region)

```
{
  "Effect": "Deny",
  "Action": "*",
  "Resource": "*",
  "Condition": {
    "StringNotEquals": {
      "aws:RequestedRegion": [
        "us-east-1", "eu-west-1"
      ]
    }
  }
}
```

4. Landing Zone Architecture

A Landing Zone is a pre-configured, secure, and scalable AWS environment that enables teams to deploy workloads quickly while following best practices.

Landing Zone Components

Identity	Networking	Security	Governance	FinOps
Centralized identity with AWS SSO	Hub-and-spoke VPC design with shared services	Guardrails, SCPs, logging, monitoring, and alerts	Account factory, tagging, and policies	Centralized billing, budgets, and cost visibility

Use AWS Control Tower, AWS Landing Zone Accelerator, or your custom automation to implement Landing Zones.

5. Multi-Account Best Practices

Security

- Enable MFA on all users
- Use least privilege access
- Centralize logging with CloudTrail and AWS Config
- Use SCPs to enforce guardrails

Governance

- Use AWS Organizations
- Tag everything
- Apply consistent policies
- Use AWS SSO for centralized access
- Automate account provisioning

Cost Management

- Use consolidated billing
- Set budgets and alerts
- Use Cost Explorer
- Regular cost reviews
- Optimize resources continuously

Operational Excellence

- Automate with IaC
- Use CI/CD pipelines
- Monitor and alert
- Document everything
- Test disaster recovery regularly

Scalability

- Design for growth
- Use OUs for structure
- Automate account creation
- Enable self-service with guardrails

Senior Engineer Tip: Treat your organization like a product. Build strong guardrails, automate everything, and empower teams to move fast without breaking security.

VERIQTA

VERIQTA
Cost Optimization

Optimize your AWS costs without sacrificing performance, reliability, or security.

1. Cost Optimization Overview

Cost optimization is a continuous process of analyzing, right-sizing, and using the right pricing models and services to reduce waste and maximize value.

Cost Optimization Goals

- ☑ Reduce unnecessary spending
- ☑ Increase resource utilization
- ☑ Use the right pricing model
- ☑ Automate cost savings
- ☑ Improve visibility and accountability

2. AWS Cost Optimization Framework

1. Inform	2. Optimize	3. Operate	4. Monitor	5. Evolve
Gain visibility into your costs and usage.	Identify opportunities to eliminate waste.	Implement cost controls and best practices.	Continuously monitor usage, costs, and anomalies.	Iterate and improve as your needs change.

Optimize continuously. Small savings today lead to significant impact over time.

3. Savings Plans

Savings Plans offer significant discounts in exchange for a commitment to a consistent amount of usage ($/hour) for 1 or 3 years.

Key Benefits

- ☑ Up to 72% savings
- ☑ Flexible across instance families, sizes, and regions
- ☑ Automatically applies to usage
- ☑ Low commitment management

Best Practices

- ☑ Use Compute Savings Plans for flexibility
- ☑ Match commitment to baseline usage
- ☑ Monitor and adjust regularly

4. Reserved Instances (RIs)

Reserved Instances provide deep discounts (up to 72%) in exchange for committing to a specific instance type in a specific region for 1 or 3 years.

Key Benefits

- ☑ Up to 72% savings
- ☑ Predictable pricing
- ☑ Capacity reservation
- ☑ Convertible RIs for flexibility

Best Practices

- ☑ Use for steady-state, predictable workloads
- ☑ Choose Convertible RIs when possible
- ☑ Review before each renewal

5. Spot Instances

Spot Instances let you take advantage of unused EC2 capacity at up to 90% off On-Demand prices.

Key Benefits

- ☑ Up to 90% savings
- ☑ Ideal for fault-tolerant and flexible workloads
- ☑ Great for batch, CI/CD, and big data

Best Practices

- ☑ Use Auto Scaling with Spot capacity
- ☑ Set interruption handling strategy
- ☑ Diversify instance types and AZs

6. Storage Lifecycle

Store data in the right storage class at the right time to reduce costs significantly.

Storage Class	Use Case	Cost Savings
S3 Standard	Frequently accessed data	–
S3 IA (Infrequent Access)	Less frequent access	Up to 47%
S3 Glacier Instant Retrieval	Rare access, ms retrieval	Up to 68%
S3 Glacier Flexible Retrieval	Rare access, minutes	Up to 77%
S3 Glacier Deep Archive	Long-term archive	Up to 95%

7. Cost Optimization Best Practices

- Right-size resources
- Remove unused resources
- Use right pricing model
- Automate scheduling (stop/start)
- Set budgets and alerts
- Review costs regularly

Key Takeaway

The cheapest resource is the one you don't use.

Measure, optimize, automate, and repeat.

Senior Engineer Tip: Make cost optimization everyone's responsibility. Build visibility, ownership, and accountability into your process.

VERIQTA
Performance Optimization

Optimize your AWS workloads for speed, efficiency, and cost by understanding and improving performance.

1. Performance Optimization Overview

Performance optimization is the process of identifying resource constraints, eliminating bottlenecks, and designing systems that deliver maximum throughput with minimum latency at the lowest cost.

Performance Optimization Goals

- ☑ Maximize throughput and minimize latency
- ☑ Eliminate bottlenecks
- ☑ Right size resources
- ☑ Improve user experience
- ☑ Reduce infrastructure costs
- ☑ Maintain reliability and scalability

2. Performance Optimization Framework

1. Measure	2. Analyze	3. Optimize	4. Validate	5. Monitor
Collect metrics and baseline current performance.	Find bottlenecks and root causes using data.	Apply the right optimizations and right-size resources.	Test and validate improvements with metrics.	Continuously monitor and tune as workloads grow.

Measure first. Change second.
Always validate improvements with real data.

3. Bottleneck Analysis

Bottleneck analysis helps identify the single resource or component that limits overall system performance.

Common Bottlenecks

- ☑ CPU saturation
- ☑ Memory pressure
- ☑ Disk I/O
- ☑ Network bandwidth
- ☑ Database queries
- ☑ Application thread locks
- ☑ Third-party dependencies

Analysis Steps

1. Monitor key metrics
2. Identify symptoms
3. Isolate the bottleneck
4. Validate the root cause
5. Implement and verify fix

4. Right Sizing

Right sizing ensures you use the right type and size of resources to meet demand without over-provisioning or under-provisioning.

Right Sizing Principles

- ☑ Match resources to workload demand
- ☑ Use metrics-driven decisions
- ☑ Consider burst and peak usage
- ☑ Review regularly
- ☑ Avoid over-provisioning

Right Sizing Actors

- ☑ EC2 instances
- ☑ RDS instances
- ☑ EBS volumes
- ☑ Auto Scaling groups
- ☑ Lambda memory
- ☑ ElastiCache nodes

5. Throughput Planning

Throughput planning ensures your system can handle current and future load while maintaining performance and availability.

Planning Considerations

- ☑ Estimate current and future load
- ☑ Identify peak and average traffic
- ☑ Plan for horizontal scaling
- ☑ Design for failover and redundancy
- ☑ Account for third-party limits

Throughput Capacity Formula

Capacity = (Requests per second) x (Average processing time per request) x (Safety factor)

Where:

- Safety factor = 1.2 to 2.0
- Add 20% - 100% buffer for spikes

6. Performance Optimization Best Practices

Monitor everything that matters	Optimize data and queries	Use caching to reduce latency	Optimize application code	Design for scalability from day one	Balance performance and cost

Key Takeaway

High performance is a result of good architecture, the right resources, and continuous optimization.

Senior Engineer Tip: Performance problems are normal. Not monitoring is the problem. Build visibility, analyze deep, and optimize continuously.

VERIQTA

VERIQTA

AWS Solutions Architect Production Handbook

Microservices Architecture

Build loosely coupled, independently deployable services that scale and evolve faster.

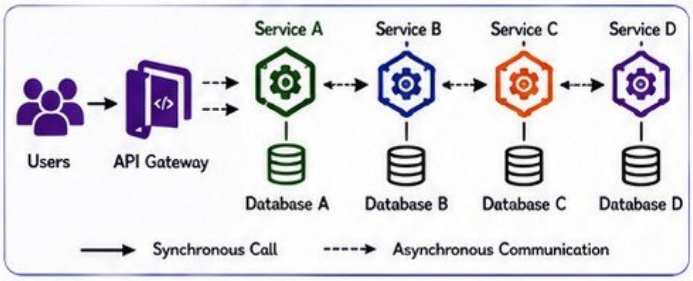
1. Microservices Overview

Microservices architecture breaks applications into small, independent services that own their data and communicate over well-defined APIs. This approach improves scalability, resilience, and team autonomy.

Key Benefits

- ☑ Independent deployment and scaling
- ☑ Technology diversity
- ☑ Fault isolation and resilience
- ☑ Faster delivery and innovation
- ☑ Better team autonomy and ownership

3. Architecture Overview



4. Key Components

Service	API	Data Store	Communication	Discovery	Configuration
Independent business capability	Defines contracts between services	Owns its data and schema	Sync (REST) Asynchronous (Messaging)	Finds and registers services	Externalizes settings and secrets

5. Communication Patterns

Synchronous (Request/Response)

- ☑ Simple and immediate interaction
- ☑ Uses REST or gRPC
- ☑ Lower latency but tight coupling

Asynchronous (Event-Driven)

- ☑ Decoupled and resilient
- ☑ Uses message brokers (e.g., SQS, SNS)
- ☑ Better for long-running tasks

6. Data Management

Database per Service

- Each service owns its own data
- Avoid shared databases
- Ensures data autonomy

Data Consistency

- Use eventual consistency
- Design for idempotency
- Use saga patterns for transactions

7. Deployment and Scaling

Independent Deployment
Deploy and update services independently

Auto Scaling
Scale services based on demand

Fault Isolation
Failure in one service does not affect others

8. Service Discovery

Services discover each other dynamically.

- Use AWS Cloud Map
- Health checks ensure availability
- Supports dynamic environments

9. Observability

- Distributed tracing with AWS X-Ray
- Centralized logging with CloudWatch Logs
- Metrics and alerts for each service

10. Security

- Authenticate and authorize with IAM and JWT
- Use HTTPS and TLS for all service communication
- Apply least privilege access and network segmentation

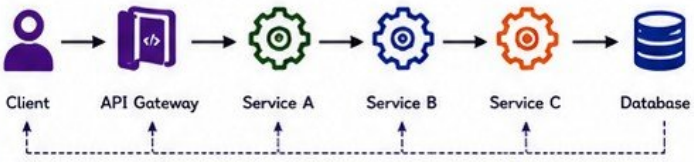
11. Resilience Patterns

- Circuit Breaker to handle failures
- Retries with exponential backoff
- Timeouts and bulkheads
- Graceful degradation

12. Best Practices

- ☑ Keep services small and focused
- ☑ Design for failure and recovery
- ☑ Automate testing and deployment
- ☑ Monitor and optimize continuously

Example Flow



Key Takeaway

Microservices enable agility, scalability, and resilience by breaking monolithic applications into small, independent, and focused services.

Senior Engineer Tip: Start with a clear domain design, automate everything, and monitor continuously.

VERIQTA
Container Architecture

Package applications and dependencies consistently for portable, scalable, and resilient deployments.

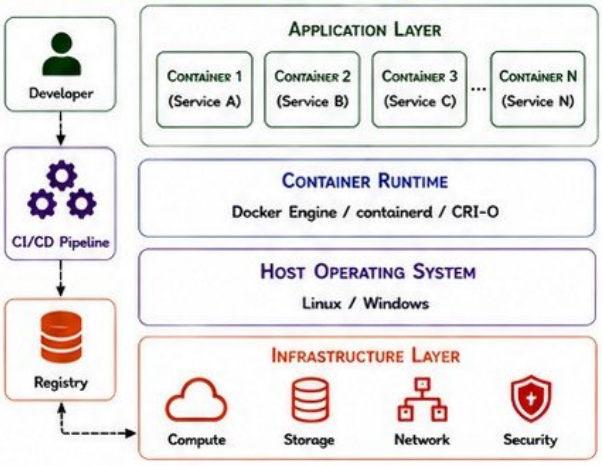
1. Overview

Container architecture uses containers to package applications with all their dependencies, libraries, and runtime components, ensuring consistency across environments from development to production.

2. Key Benefits

- ☑ Consistency across environments
- ☑ Fast and lightweight deployments
- ☑ Efficient resource utilization
- ☑ Isolation and security
- ☑ High portability
- ☑ Easy scalability and automation

3. Container Architecture Overview



4. Container Principles

- ☑ **Immutability:** Containers are built once and never changed.
- ☑ **Isolation:** Each container runs in its own isolated environment.
- ☑ **Portability:** Run anywhere the container runtime is available.
- ☑ **Efficiency:** Containers share the host OS kernel, using fewer resources.
- ☑ **Scalability:** Easily scale containers horizontally.
- ☑ **Declarative:** Define desired state and automate deployments.

5. Container Components

Image	Container	Registry	Network	Volume
A lightweight, read-only template with application code, libraries, and dependencies.	A running instance of an image with its own filesystem, network, and process space.	Stores and distributes container images. Example: Amazon ECR, Docker Hub.	Provides connectivity between containers and external systems.	Persistent storage separated from the container lifecycle.

6. Container Lifecycle

Build	Push	Pull	Run	Remove
Create image from Dockerfile.	Push image to registry.	Pull image to host.	Run container from image.	Stop and remove container.

Automate the lifecycle with CI/CD pipelines and Infrastructure as Code.

7. Container Orchestration (Example: Amazon EKS)

- ☑ Schedule containers across a cluster of nodes.
- ☑ Ensure high availability and self-healing.
- ☑ Manage scaling, rolling updates, and rollbacks.
- ☑ Provide service discovery and load balancing.
- ☑ Integrate with IAM, VPC, and security controls.

8. Best Practices

Security	Image Hygiene	Resource Limits	Logging & Monitoring	Automate Everything
Scan images, use least privilege, and keep images up to date.	Use minimal base images and remove unnecessary packages.	Set CPU and memory limits to prevent resource exhaustion.	Centralize logs and monitor container health.	Use IaC, CI/CD and GitOps for reproducible deployments.

Key Takeaway

Containers provide a consistent, portable, and efficient way to build and run applications. Combine containers with orchestration for true production scale.

VERIQTA

Senior Engineer Tip: Design for automation, observability, and resilience from day one.

VERIQTA

AWS Solutions Architect Production Handbook

Serverless Architecture

Build and run applications without managing servers.
Focus on code. AWS manages the rest.

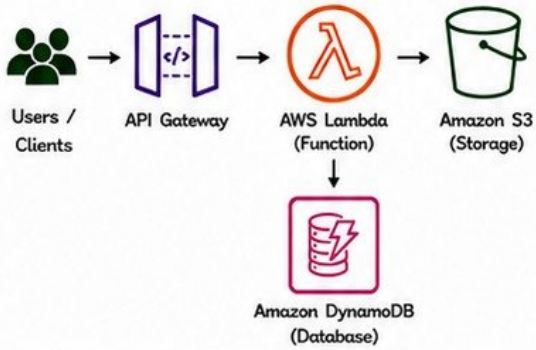
1. Overview

Serverless architecture allows you to build and run applications without provisioning or managing servers. It automatically scales, is highly available, and you pay only for what you use.

2. Key Benefits

- ☑ No server management
- ☑ Automatic scaling
- ☑ High availability and built-in resilience
- ☑ Pay-per-use pricing
- ☑ Faster time to market
- ☑ Focus on business logic, not infrastructure

3. Serverless Architecture Overview



Event-driven, scalable, and fully managed components working together.

4. Serverless Principles

- ☑ Event-Driven: Functions run in response to events.
- ☑ Stateless: Each function execution is independent.
- ☑ Automatic Scaling: Scales to zero and scales out as needed.
- ☑ Pay-per-Use: You pay only for actual requests and compute time.
- ☑ Managed Services: Leverage fully managed AWS services.
- ☑ Loose Coupling: Components communicate via events and APIs.
- ☑ Resilient by Design: Built-in redundancy and fault tolerance.

5. Core Serverless Services

AWS Lambda	Amazon API Gateway	Amazon S3	Amazon DynamoDB	Amazon EventBridge
Run code without provisioning servers.	Securely expose APIs to trigger functions.	Durable storage for files, logs, and static content.	Serverless NoSQL database with high performance.	Event bus for building event-driven apps.

6. Common Use Cases

Web & Mobile Backends	File Processing & Transcoding	Email & SMS Notifications	Data Processing & Analytics	Scheduled Jobs & Automation

Use serverless when workloads are event-driven, unpredictable, or highly variable.

7. Serverless Architecture Patterns

API-Driven Applications	Use API Gateway + Lambda to build scalable RESTful APIs.
Event-Driven Processing	S3 / EventBridge events trigger Lambda functions for processing.
Data Aggregation & Analytics	Ingest data with Lambda, store in S3, query with Athena.
Orchestration with Step Functions	Coordinate multiple serverless tasks with workflows.
Real-time Applications	DynamoDB Streams + Lambda for real-time updates and notifications.

8. Best Practices

Keep functions small and focused	Use managed services whenever possible	Secure functions with least privilege IAM roles	Monitor with CloudWatch and X-Ray	Optimize for cost and performance

Key Takeaway

Serverless removes operational burden, reduces costs, and accelerates development by letting you focus on building features while AWS handles scaling, availability, and infrastructure.

VERIQTA

Senior Engineer Tip: Design for idempotency, observability, and performance. Test for cold starts and optimize execution time.

VERIQTA

AWS Solutions Architect Production Handbook

VERIQTA DATA LAKE ARCHITECTURE

ENGINEERING VERIQTA EXCELLENCE

Store all your data in its raw form. Enable analytics, ML, and reporting at scale with performance and governance.

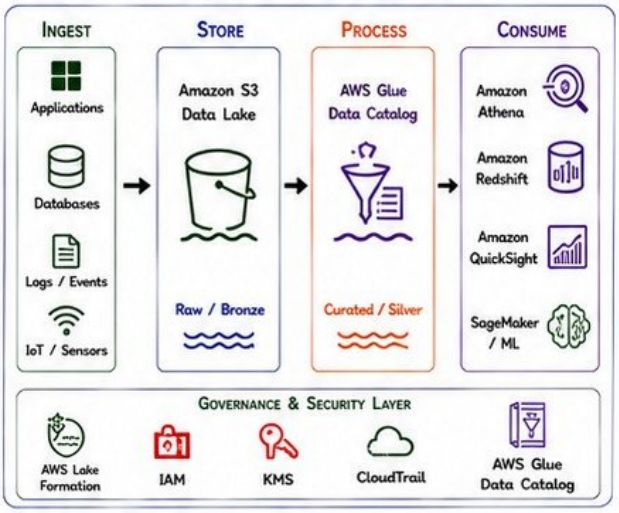
1. Overview

A data lake architecture allows you to ingest, store, and manage large amounts of structured, semi-structured, and unstructured data at any scale. It provides a single repository for all data to support diverse analytics and machine learning workloads.

2. Key Benefits

- ☑ Store all data in raw format
- ☑ Handle structured, semi-structured, and unstructured data
- ☑ High scalability and durability
- ☑ Cost-effective storage
- ☑ Support advanced analytics and ML
- ☑ Decouple storage from compute

3. Data Lake Architecture Overview



4. Data Lake Principles

- ☑ Store data once, use many times.
- ☑ Keep data in raw format in the lake.
- ☑ Separate storage from compute.
- ☑ Use metadata to discover and manage data.
- ☑ Apply governance, security, and access control.
- ☑ Enable multiple engines to process the same data.
- ☑ Optimize for cost, performance, and scalability.

5. Data Lake Layers

Bronze (Raw)

Raw data ingested as-is from source systems.

Immutable and append-only.

Silver (Curated)

Cleaned, enriched, and transformed data for analysis.

Conformed and quality-checked.

Gold (Consumption)

Business-level aggregated data for reporting, dashboards, and machine learning.

6. Common Data Sources

Relational Databases	Files / CSV / JSON	Cloud Applications	IoT / Devices	Logs / Events	3rd Party APIs

Store everything first, decide how to use it later.
The lake is the foundation for all analytics.

7. Data Lake Best Practices

Security First
Encrypt data, use least privilege, and enforce fine-grained access control.

Metadata Management
Use Glue Data Catalog to organize and discover data.

Cost Optimization
Use S3 lifecycle, partitioning, compression, and right storage classes.

Data Quality
Validate, profile, and monitor data quality continuously.

Scalability
Design for massive scale and high availability by default.

Automation
Automate ingestion, cataloging, and data pipelines with Glue / CI/CD.

Monitoring
Monitor data pipelines, usage, costs, and performance.

Governance
Use Lake Formation for data governance, lineage, and compliance.

8. Typical Data Lake Workflow

Key Takeaway

A data lake centralizes all data in raw format, enabling limitless analytics, machine learning, and reporting. Governed, secure, and well-architected lakes unlock the true value of data.

VERIQTA

Senior Engineer Tip: Design your data lake like a product: well-governed, well-documented, observable, and cost-optimized.

VERIQTA
Analytics Architecture

Turn data into insights at scale. Build modern analytics systems for reporting, dashboards, BI, and machine learning.

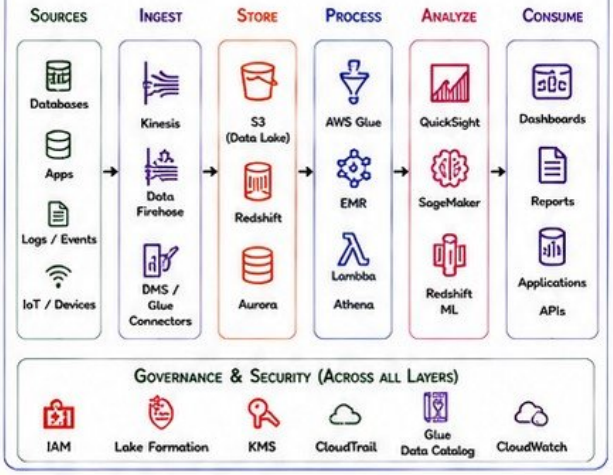
1. Overview

Analytics architecture collects, processes, stores, and analyzes data to deliver actionable insights. It supports dashboards, reporting, business intelligence, ad-hoc queries, and advanced analytics at scale.

2. Key Benefits

- ☑ Data-driven decision making
- ☑ Scalable and cost-effective
- ☑ Real-time and near real-time insights
- ☑ Self-service analytics for teams
- ☑ Supports ML and predictive analytics
- ☑ High performance and concurrency

3. Analytics Architecture Overview



4. Architecture Principles

- ☑ Decouple storage from compute.
- ☑ Store raw data once, use many times.
- ☑ Use open formats (Parquet, ORC) for efficiency.
- ☑ Enable metadata-driven discovery and governance.
- ☑ Design for scalability, security, and cost optimization.
- ☑ Deliver trusted data with data quality and lineage.

5. Analytics Components

Data Lake	Data Warehouse	Data Catalog	Query Engine	Visualization
Amazon S3	Amazon Redshift	AWS Glue Data Catalog	Amazon Athena	Amazon QuickSight
Central repository for raw and processed data.	High performance analytics and SQL queries.	Metadata store for discovery, lineage, and governance.	Serverless SQL queries on data in S3.	BI dashboards and interactive visualizations.

6. Analytics Workflow

1. **Collect** — Data from multiple sources
2. **Ingest** — Stream or batch data into the lake
3. **Store** — Store raw data in S3 (Data Lake)
4. **Process** — Clean, transform, and enrich data
5. **Analyze** — Run queries, generate insights
6. **Consume** — Visualize, share, and act on data

Separation of storage and compute gives you flexibility, performance, and cost control.

7. Analytics Patterns

Batch Analytics	Process large volumes of data on a schedule using Glue, EMR, or Redshift.
Real-time Analytics	Use Kinesis + Lambda + Redshift Streaming Ingestion for near real-time insights.
Self-service BI	Enable business users with curated datasets and QuickSight dashboards.
Advanced Analytics / ML	Use SageMaker, Redshift ML, and feature stores to build predictive models.

8. Best Practices

Secure Data	Data Quality	Cost Optimization	Performance	Automation
Encrypt data at rest and in transit. Use least privilege access.	Validate, clean, and monitor data quality continuously.	Use lifecycle policies, compression, partitioning, and right-sized compute.	Partition data, use columnar formats and optimize queries.	Automate with IaC, CI/CD, and scheduled workflows.

Key Takeaway

A well-designed analytics architecture delivers trusted, scalable, and cost-effective insights.
It empowers your organization to make better decisions faster.

VERIQTA

Senior Engineer Tip: Start simple, govern early, automate everything, and continuously optimize for value.

VERIQTA

AWS Solutions Architect Production Handbook

AI and Machine Learning Architecture

Design scalable, secure, and cost-effective AI/ML systems.
From data to models to intelligent applications.

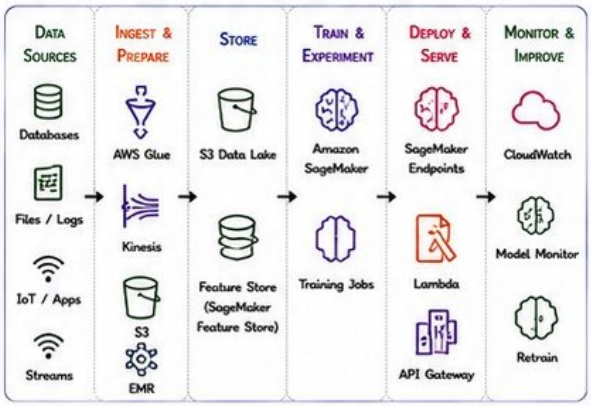
1. Overview

AI/ML architecture enables organizations to build, train, deploy, and operate intelligent models at scale. It integrates data engineering, model development, infrastructure, and MLOps for reliable machine learning in production.

2. Key Benefits

- ☑ Automate predictions and decisions
- ☑ Improve accuracy and efficiency
- ☑ Extract insights from large datasets
- ☑ Scale models with high availability
- ☑ Accelerate innovation and time to value
- ☑ Reduce operational cost with automation

3. AI/ML Architecture Overview



End-to-end lifecycle from raw data to production AI with continuous improvement.

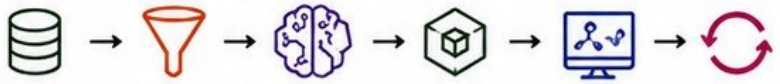
4. AI/ML Architecture Principles

- ☑ Start with high-quality, reliable data.
- ☑ Build reproducible and versioned pipelines.
- ☑ Separate training and inference environments.
- ☑ Automate with CI/CD and MLOps practices.
- ☑ Monitor models and data in production.
- ☑ Secure data, models, and endpoints.
- ☑ Optimize for performance, cost, and scalability.

5. Core AWS Services for AI/ML

Amazon SageMaker	AWS Glue	Amazon S3	Amazon Redshift	Amazon Bedrock	SageMaker Feature Store
Build, train, and deploy ML models at scale with fully managed infrastructure.	Data integration and ETL for preparing clean and reliable datasets.	Durable storage for datasets, models, artifacts, and logs.	Data warehouse for large-scale analytics and model training.	Access foundation models (LLMs) and build generative AI applications.	Store, manage, and serve ML features for training and inference.

6. ML Workflow Lifecycle



1. Collect	2. Prepare	3. Train	4. Deploy	5. Monitor	6. Improve
Gather data from various sources.	Clean, transform, and engineer features.	Train and tune models using SageMaker.	Deploy models as endpoints or serverless.	Monitor data, performance, and drift.	Retrain models and optimize continuously.

Automation and monitoring across the lifecycle ensure reliable AI in production.

7. Common AI/ML Architecture Patterns

	Data Lake + ML	Store raw data in S3, process with Glue/EMR, train models with SageMaker.
	Real-time Inference	Use SageMaker Endpoints, Lambda, and API Gateway for low-latency predictions.
	Streaming ML	Process streaming data with Kinesis, run inference in real-time, and store results.
	MLOps Pipeline	Automate training, testing, deployment, and monitoring with SageMaker Pipelines + CI/CD.
	Generative AI	Use Amazon Bedrock or custom LLMs for chatbots, summarization, and content generation.

8. Best Practices

Security First	Data Quality	Cost Optimization	Model Monitoring	Automation
Encrypt data, control access with IAM, and secure endpoints with VPC.	Validate, clean, and monitor data quality before training models.	Use Spot training, right-size instances, and limit endpoint payload sizes.	Monitor accuracy, data drift, latency, and retrain proactively.	Automate pipelines, testing, deployment, and rollback workflows.

Key Takeaway

A well-architected AI/ML system is data-driven, secure, scalable, and continuously improving.
Combine the right AWS services with MLOps to deliver intelligent solutions at scale.

VERIQTA

Senior Engineer Tip: Focus on data quality, monitoring, and automation. Models are only as good as the data and processes behind them.

VERIQTA

VERIQTA API ARCHITECTURE

AWS Solutions Architect Production Handbook

ENGINEERING VERIQTA EXCELLENCE

Design secure, scalable, and maintainable APIs that power modern applications, integrations, and digital products.

1. Overview

API architecture defines how APIs are designed, exposed, secured, deployed, and scaled. It ensures reliability, security, performance, and excellent developer experience across all integrations.

2. Key Benefits

- ☑ Loose coupling and reusability
- ☑ Scalable and independently deployable
- ☑ Faster integration and time to market
- ☑ Secure access and controlled exposure
- ☑ Better developer experience
- ☑ Enables microservices and automation

3. API Architecture Overview

Clients	API Gateway	Authentication	Services	Data
Web, Mobile, Third-party Applications	Routing, Rate Limiting, Caching, Throttling	Validate, Authorize and Issue Tokens	Business Logic Microservices or Lambda	Databases, Caches, Storage

4. API Styles

REST

- Resource based
- Stateless
- Uses HTTP methods (GET, POST, PUT, DELETE)
- JSON / XML
- Widely adopted

Use Case: General CRUD operations

GraphQL

- Query language
- Client requests exact data
- Single endpoint
- Strongly typed schema
- Real-time subscriptions

Use Case: Complex queries, mobile apps, efficiency

gRPC

- High performance
- Binary protocol (Protocol Buffers)
- Strongly typed
- Streaming support
- Low latency

Use Case: Internal services, high performance

5. Core API Components

API Gateway	Authentication	Authorization	Rate Limiting	Monitoring
Single entry point for all API requests. Handles routing, auth, throttling, and more.	Verifies identity using tokens or API keys. Integrates with Cognito, OAuth2, or IAM.	Checks permissions and scopes. Enforce least privilege access to resources.	Protect APIs from abuse. Control traffic and ensure fair usage.	Track usage, latency, errors, and performance. Use CloudWatch and X-Ray.

6. API Request Flow

1. Client	2. API Gateway	3. Authenticate	4. Authorize	5. Process	6. Respond
Sends request to API Gateway	Routes, validates, and applies policies	Validate token and authenticate request	Check permissions and scopes	Route to backend services / Lambda	Return response to client

Keep APIs stateless, secure, versioned, and observable.
Design for scale, reliability, and developer experience.

7. Authentication & Authorization Options

Method	Description	Use Case
IAM Auth	AWS IAM signs requests (SigV4) for secure access.	AWS internal services, machine to machine
Cognito User Pools	User sign-up, sign-in, token issuance (JWT).	Web and mobile applications
OAuth 2.0 / OpenID Connect	Standard protocol for delegated authorization.	Third-party login, enterprise SSO
API Keys	Simple API key validation for client identity.	Public APIs, partner integrations
JWT Tokens	Stateless tokens with claims and expiration.	Microservices, serverless APIs

8. Best Practices

Security First	Version Your APIs	Rate Limit	Document	Monitor & Log	Cache Smartly
Enforce TLS, auth, authorization, and least privilege access.	Use URI or header versioning. Maintain backward compatibility.	Set limits, throttle traffic, and protect backend services.	Provide clear API docs using OpenAPI / Swagger.	Monitor usage, errors, latency, and set alerts proactively.	Cache responses and use CDN for better performance.

9. API Design Best Practices

- ☑ Use nouns for resources, not verbs.
- ☑ Use proper HTTP methods and status codes.
- ☑ Keep requests and responses consistent.
- ☑ Use pagination, filtering, and sorting.
- ☑ Handle errors with meaningful messages and codes.
- ☑ Keep payloads small and optimized.
- ☑ Design for idempotency where appropriate.

Key Takeaway

A well-architected API is secure, scalable, maintainable, and developer-friendly.
It is the contract that powers integrations, automation, and digital innovation.

VERIQTA

Senior Engineer Tip: Treat your APIs as products: version, monitor, secure, measure, and continuously improve.

VERIQTA
Migration Strategies

Move your workloads to the cloud the right way.
Choose the best strategy for each application.

1. Overview

Migration strategies help you choose the best approach for each workload based on business goals, technical requirements, cost, and timeline. Not every application needs the same strategy.

2. Benefits

- ☑ Lower cost and risk
- ☑ Faster time to cloud value
- ☑ Improved performance and security
- ☑ Business agility and innovation
- ☑ Application modernization at scale
- ☑ Better resource utilization

3. Migration Strategy Overview

Evaluate each application and select the strategy that delivers the most business value with the least risk.

4. Strategy Selection Factors

- ☑ Business objectives and desired outcomes
- ☑ Application dependencies and complexity
- ☑ Cost and total cost of ownership (TCO)
- ☑ Performance and reliability requirements
- ☑ Security, compliance, and data sensitivity
- ☑ Timeline and organizational readiness
- ☑ Skills, tooling, and operational model

5. The 7 R's of Cloud Migration

Strategy	Description	Use Case	Example
1. Rehost (Lift and Shift)	Move applications to the cloud with minimal or no changes. Fastest and lowest risk approach.	• Legacy applications • Urgent cloud adoption • Rehosting for DR • Cost reduction first step	Move a legacy web app from on-prem VM to EC2 as-is.
2. Replatform (Lift, Tinker, and Shift)	Make some optimizations to take advantage of cloud features without changing core architecture.	• Improve scalability • Reduce operational overhead • Enhance performance	Move database to Amazon RDS, use Auto Scaling for web tier.
3. Refactor (Rearchitect)	Redesign or rewrite parts of the application to take full advantage of cloud-native services.	• Modernize applications • Improve agility • Enable innovation • Long-term value	Rewrite monolith into microservices using ECS, Lambda, and DynamoDB.
4. Retire	Decommission and remove applications that are no longer needed.	• Obsolete applications • Low or no business value • Redundant systems	Retire an old reporting tool no longer in use.
5. Repurchase (Replace)	Replace the application with a SaaS solution that meets business requirements.	• Replace existing software • Reduce management overhead • Faster innovation	Replace on-prem CRM with Salesforce.
6. Retain	Keep the application on-premises or in the current environment.	• Compliance constraints • Low cloud readiness • High refactor cost • Data residency	Keep mainframe system due to compliance and regulatory needs.
7. Relocate (Move)	Move applications to a different environment (e.g., another data center or cloud provider).	• Data center exit • Mergers & acquisitions • Better location or connectivity	Move workloads to AWS Europe region for lower latency.

6. Migration Phases

1. **Assess** — Discover and assess workloads
2. **Plan** — Define strategy, roadmap, and business case
3. **Prepare** — Build landing zone, prepare cloud environment
4. **Migrate** — Execute migration using chosen strategy
5. **Validate** — Test, validate, and optimize
6. **Optimize** — Continuously improve performance, cost, and operations

7. Best Practices

- **Start Small** — Begin with low-risk workloads and build confidence.
- **Automate** — Use automation and IaC to reduce risk and speed up.
- **Measure & Optimize** — Continuously monitor cost, performance, and reliability.
- **Involve Stakeholders** — Engage business, security, and operations early and often.
- **Security First** — Apply security controls throughout the migration journey.
- **Plan for the Future** — Choose strategies that enable long-term modernization.

Key Takeaway
There is no one-size-fits-all approach. Use the 7 R's to choose the right migration strategy for each workload and maximize value in the cloud.

Senior Engineer Tip: Assess, prioritize, and migrate iteratively. Combine strategies across applications to achieve quick wins and long-term transformation.

VERIQTA
Production Failure Analysis

Detect issues fast. Find the real root cause.
Fix permanently. Prevent recurrence.

1. Overview

Production systems fail. The goal is not to avoid failures completely, but to detect quickly, analyze correctly, and restore reliably. **A structured failure analysis process helps you find the real root cause and prevent repeat incidents.**

2. Common AWS Failures

Category	Common Failure	Impact	Possible Cause
Compute	EC2 instance high CPU / OOM / Crash	Slow or unavailable apps	Code issues, memory leak, improper sizing
Storage	S3 high 5xx errors or timeouts	Data access failures	Request rate limits, network issues, S3 internal events
Database	RDS failover or connection errors	Application downtime	Failover, network, exhausted connections, query performance
Networking	VPC / NACL / SG blocking traffic	Service inaccessible	Misconfiguration, routing, security rules
Load Balancing	ALB / NLB unhealthy targets	Requests failing	Health check failure, target crash, deployment issues
DNS	Route 53 resolution errors or latency	Users cannot reach service	DNS misconfig, propagation delay, external DNS issues
Lambda	Lambda errors, timeouts, throttling	Function failures	Code error, timeout, concurrency limits, cold starts
Infrastructure	AZ / Region outage or service event	Widespread outage	AWS service event, region disruption, dependency failure

3. Failure Impact Areas

- **User Experience** — Slow response, errors, downtime
- **Business Impact** — Revenue loss, SLA breach, customer churn
- **Operational Impact** — On-call stress, manual work, incident escalation
- **Technical Impact** — Data loss risk, cascading failures, instability

4. Root Cause Methodology

1. **Detect** — Alert triggers. Identify that something is wrong.
2. **Triage** — Validate alert. Check impact. Gather initial information.
3. **Investigate** — Analyze metrics, logs, traces, and events. Form hypotheses.
4. **Identify Root Cause** — Find the primary cause that led to the incident (not just symptoms).
5. **Resolve & Recover** — Fix the issue. Restore service. Validate fully.
6. **Post-Incident Review** — Document timeline, root cause, actions, and lessons learned.
7. **Prevent & Improve** — Implement preventive controls, automation, monitoring, and improvements to avoid recurrence.

5. Investigation Checklist

- [x] Check CloudWatch Alarms and dashboards.
- [x] Review recent deployments and changes.
- [x] Analyze CloudWatch Metrics (CPU, Memory, Latency, Errors).
- [x] Check CloudTrail for API errors or throttling.
- [x] Review VPC Flow Logs and Security Group rules.
- [x] Analyze application logs and stack traces.
- [x] Check AWS Health Dashboard for service events.
- [x] Validate dependencies (DB, Cache, External APIs).
- [x] Reproduce issue in lower environment (if possible).

6. Useful AWS Tools

- **CloudWatch** — Metrics, logs, alarms
- **CloudTrail** — API activity and errors
- **AWS X-Ray** — Distributed tracing
- **VPC Flow Logs** — Network traffic analysis
- **AWS Health** — Service health events
- **Trusted Advisor** — Best practice checks

7. Incident Severity Guide

Severity	Description	Example	Response Time
SEV 1	Major outage. Service down for most users.	Website down, payments failing	< 15 minutes
SEV 2	Partial outage or major performance impact.	Slow performance, some users affected	< 30 minutes
SEV 3	Minor impact with workaround available.	Non-critical feature not working	< 2 hours
SEV 4	Minimal impact or cosmetic issues.	UI glitch, typo, small bug	< 1 business day

Key Takeaway
Fast detection + accurate analysis + permanent fix = reliable systems.
Your goal is not just to restore, but to prevent the next incident.

VERIQTA

Senior Engineer Tip: Blame the system, not people. Your infrastructure and processes should make the right path the easy path.

VERIQTA
Architecture Decision Records

Documenting Trade-offs. Building Clarity.
Ensuring Long-term Maintainability.

1. What is an ADR?

An Architecture Decision Record (ADR) is a lightweight document that captures an important architectural decision, the context, options considered, the chosen solution, and the reasoning behind it.

Purpose: Create a historical record of why decisions were made, not just what was built.

2. Why use ADRs?

- ☑ Preserve decision context over time
- ☑ Improve team alignment and transparency
- ☑ Speed up onboarding for new engineers
- ☑ Avoid repeating discussions
- ☑ Support audits and compliance
- ☑ Improve long-term maintainability
- ☑ Make future changes easier and safer

3. ADR Structure Template

Title	Short, descriptive name of the decision.
Context	What problem are we trying to solve? What is the current situation?
Decision	What decision are we making? What is the chosen solution?
Options Considered	What alternatives were evaluated? Why were they considered?
Decision Drivers	What factors influenced the decision? (Cost, performance, security, etc.)
Consequences	What are the positive and negative impacts of this decision?
Status	Proposed | Accepted | Deprecated | Superseded
Date & Authors	When was this decision made? Who made or contributed to it?
References	Links, docs, diagrams, tickets, ADRs related to this decision.

4. Example ADR

Title:	Use Amazon Aurora PostgreSQL Instead of MySQL on EC2
Context	Our application requires a highly available, scalable, and managed database with minimal operational overhead.
Decision	We will use Amazon Aurora PostgreSQL as our primary database.
Options Considered	• MySQL on EC2 • Amazon RDS for MySQL • Amazon Aurora PostgreSQL
Decision Drivers	• High availability and durability • Automated backups and failover • Better performance at scale • Reduced operational overhead • Cost efficiency over self-managed solutions
Consequences	Positive: High availability, automatic scaling, reduced admin work. Negative: Higher cost than EC2 MySQL, Aurora-specific features.
Status	Accepted
Date & Authors	2024-05-15 | Platform Team
References	Aurora Docs, Cost Comparison Sheet, Architecture Diagram, Ticket #DB-1024

5. Documenting Trade-Offs

Trade-Off Factors	Questions to Ask
Cost — Short-term vs long-term cost?	☑ What are we solving and why?
Performance — Latency, throughput, scalability impact?	☑ What are the must-have vs nice-to-have requirements?
Security — Data protection, compliance, attack surface?	☑ What are the risks of each option?
Operational — Ease of operation, monitoring, team skillset?	☑ What happens if this decision is wrong?
Flexibility — Can it adapt to future requirements?	☑ How hard will it be to change this later?
Maintainability — How easy is it to maintain and evolve?	☑ What is the impact on users, costs, and operations?

6. ADR Best Practices

- **Keep It Lightweight** — One page is enough. Focus on the decision, not the design.
- **Write for the Future** — Assume future engineers will not know the context.
- **Link Everything** — Link related ADRs, diagrams, tickets, and docs.
- **Review Regularly** — Revisit decisions during architecture reviews.
- **Record the Thinking** — Document the reasoning, not just the outcome.
- **Version Your ADRs** — Keep them in version control with your code.

7. ADR Lifecycle

Proposed (New idea or decision raised.) → **Under Review** (Team reviews and discusses.) → **Accepted** (Decision is approved.) → **Implemented** (Decision is put into action.) → **Revisited** (Context changes, decision reviewed.) → **Deprecated** (No longer valid, superseded.)

Key Takeaway

ADRs build institutional memory. They capture why decisions were made, the trade-offs involved, and help your architecture stay maintainable and adaptable for the long term.

VERIQTA

Senior Engineer Tip: If a decision is not documented, it will be rediscovered, re-litigated, or regretted. Write it down once, and save time forever.

VERIQTA

ARCHITECTURE REVIEW CHECKLIST

AWS SOLUTIONS ARCHITECT PRODUCTION HANDBOOK

ENGINEERING VERIQTA EXCELLENCE

Review early. Find issues before they become incidents.
Build secure, scalable, reliable, and maintainable systems.

1. OVERVIEW

This checklist helps architects and engineers validate designs before implementation. Use it in design reviews, pre-production reviews, or periodic architecture assessments.

2. PURPOSE

- ☑ Validate design quality and completeness
- ☑ Identify risks and gaps early
- ☑ Ensure alignment with business goals
- ☑ Promote security, reliability, and scalability
- ☑ Improve long-term maintainability
- ☑ Reduce cost and operational overhead

3. ARCHITECTURE PRINCIPLES CHECK

Operational Excellence	Security	Reliability	Performance Efficiency	Cost Optimization
☑ Monitoring enabled	☑ Least privilege access	☑ Multi-AZ / Region design	☑ Auto scaling configured	☑ Right-sizing applied
☑ Logging enabled	☑ Encryption in transit & at rest	☑ Fault tolerance built-in	☑ Caching strategy used	☑ Cost estimates reviewed
☑ Runbooks available	☑ Network security defined	☑ Backups and DR plan	☑ Performance tested	☑ Unused resources removed
☑ Alarms configured	☑ Secrets managed securely	☑ Health checks enabled	☑ Bottlenecks identified	☑ Budget alarms set
☑ Incident handling defined	☑ Compliance considered	☑ SLA / SLO defined	☑ Quotas and limits considered	☑ Reserved options considered

4. DESIGN AREA CHECKLIST

Area	Checkpoints	Ok
Business & Requirements	• Business objectives are clear • Requirements are documented • Stakeholders are aligned • Success metrics are defined	☐
Architecture Design	• High-level design is complete • Components and interactions are clear • Data flow is well defined • Technology choices justified	☐
Security	• Threat model reviewed • Access control defined • Data classification done • Security controls in place	☐
Reliability Resilience	• Failure scenarios identified • Resiliency patterns applied • DR strategy documented • Backup & restore tested	☐
Performance	• Expected load defined • Scalability tested • Performance benchmarks set • Caching / CDN used	☐
Cost	• Cost estimation completed • Cost drivers identified • Optimization opportunities considered • Budgets & alerts configured	☐
Operations	• Deployment strategy defined • Monitoring & logging planned • Runbooks documented • Support model defined	☐

5. ARCHITECTURE REVIEW CHECKLIST

Category	Checkpoint	Yes	No	N/A
Business	Does the architecture meet the business objectives?	☐	☐	☐
	Are requirements complete and non-functional needs addressed?	☐	☐	☐
Design	Is the architecture simple and easy to understand?	☐	☐	☐
	Are components loosely coupled?	☐	☐	☐
	Is the data flow and dependencies clearly defined?	☐	☐	☐
Security	Is least privilege access enforced?	☐	☐	☐
	Is data encrypted in transit and at rest?	☐	☐	☐
	Are security controls and compliance requirements met?	☐	☐	☐
Reliability	Is the architecture highly available?	☐	☐	☐
	Can the system recover from component failures?	☐	☐	☐
	Are backups, DR, and failover tested?	☐	☐	☐
Performance	Can the architecture handle expected load?	☐	☐	☐
	Is auto scaling configured where needed?	☐	☐	☐
	Are performance bottlenecks identified and mitigated?	☐	☐	☐
Cost	Is the architecture cost-effective?	☐	☐	☐
	Are resources right-sized?	☐	☐	☐
	Are cost optimization strategies applied?	☐	☐	☐
Operations	Is monitoring, logging, and alerting defined?	☐	☐	☐
	Are runbooks and incident procedures documented?	☐	☐	☐
	Is the deployment pipeline automated?	☐	☐	☐
Maintainability	Is the architecture modular and maintainable?	☐	☐	☐
	Is infrastructure as code used?	☐	☐	☐
	Can the system evolve with minimal rework?	☐	☐	☐

6. REVIEW OUTCOME

APPROVED
Architecture is sound.
Proceed to implementation.

CONDITIONAL APPROVAL
Address the issues listed.
Re-review before production.

NOT APPROVED
Major issues found.
Do not proceed.

REVIEW DETAILS

Reviewed By: _____

Date: _____

Notes / Issues: _____

_____

KEY TAKEAWAY
Great architectures are reviewed, challenged, and improved.
A good review today prevents a major incident tomorrow.

VERIQTA

SENIOR ENGINEER TIP: Challenge assumptions. Ask "what could go wrong?". Review with production in mind, not just happy path.

VERIQTA
Production Readiness Review

Before we go live, we review everything.
Building systems that are secure, scalable, reliable, and supportable.

1. Overview

The Production Readiness Review (PRR) validates that the system, team, and processes are ready for reliable operation in a live environment. It reduces risk, prevents incidents, and builds operational confidence.

2. Readiness Principles

Security First	Secure by design. Least privilege, encryption, and compliance.
Reliability by Design	High availability, resilience, and failure recovery.
Performance Matters	Meets performance, scalability, and capacity requirements.
Cost Awareness	Resources are right-sized and costs are optimized.
Operationally Excellent	Observable, supportable, and easy to operate.
Team Prepared	Runbooks, on-call, and clear roles and responsibilities.

3. Production Readiness Areas

Architecture	Architecture is reviewed, documented, and approved.
Security	Security controls implemented and validated.
Reliability	High availability, backup, and recovery validated.
Performance	Performance and scalability tests completed.
Operations	Monitoring, alerting, and runbooks are in place.
Data Protection	Backups, retention, and DR plans are validated.
Deployment	Deployment pipeline tested and releases strategy confirmed.
Documentation	Documentation complete and up to date.
Compliance	Regulatory and compliance requirements met.
Team Readiness	Team trained, roles defined, and communication ready.

4. Production Readiness Checklist

Category	Checkpoint	Yes	No	N/A	Notes
Architecture	Architecture diagram and components reviewed	☐	☐	☐	
	Dependencies and integrations documented	☐	☐	☐	
	Networking (VPC, subnets, routing) validated	☐	☐	☐	
	Capacity and scalability requirements met	☐	☐	☐	
Security	Least privilege access enforced	☐	☐	☐	
	Encryption in transit and at rest enabled	☐	☐	☐	
	Security groups, NACLs, and WAF configured	☐	☐	☐	
	Secrets managed securely (Secrets Manager)	☐	☐	☐	
	Vulnerability scan completed with no critical issues	☐	☐	☐	
Reliability	Multi-AZ / multi-region design validated	☐	☐	☐	
	Auto Scaling and failover tested	☐	☐	☐	
	Backups configured and restore tested	☐	☐	☐	
	Health checks and alarms validated	☐	☐	☐	
	Disaster recovery plan tested / documented	☐	☐	☐	
Performance	Load testing completed	☐	☐	☐	
	Auto Scaling thresholds validated	☐	☐	☐	
	Database performance optimized	☐	☐	☐	
	Caching implemented where needed	☐	☐	☐	
	CDN / Edge optimization enabled	☐	☐	☐	
Operations	CloudWatch dashboards created	☐	☐	☐	
	Alarms configured and tested	☐	☐	☐	
	Runbooks created and reviewed	☐	☐	☐	
	On-call rotation and escalation defined	☐	☐	☐	
	Logging centralized and searchable	☐	☐	☐	
Data Protection	Backup strategy documented and tested	☐	☐	☐	
	Data retention and lifecycle policies set	☐	☐	☐	
	Cross-region backup (if required) configured	☐	☐	☐	
	RPO and RTO targets defined and met	☐	☐	☐	
Deployment	CI/CD pipeline tested end-to-end	☐	☐	☐	
	Blue/Green or Canary strategy defined	☐	☐	☐	
	Rollback plan tested and documented	☐	☐	☐	
	Release checklist created and reviewed	☐	☐	☐	
Documentation	Architecture documentation is up to date	☐	☐	☐	
	API documentation and schemas available	☐	☐	☐	
	Operational runbooks and SOPs available	☐	☐	☐	
	System diagrams and data flows documented	☐	☐	☐	
Compliance	Compliance requirements identified	☐	☐	☐	
	Audit logging enabled	☐	☐	☐	
	Data privacy requirements met	☐	☐	☐	
	Third-party licenses validated	☐	☐	☐	
Team Readiness	Team training completed	☐	☐	☐	
	Roles and responsibilities clearly defined	☐	☐	☐	
	Communication plan established	☐	☐	☐	
	All stakeholders sign-off completed	☐	☐	☐	

5. Readiness Outcome

Approved
System is ready for production. All critical areas have been reviewed and accepted.

Conditional Approval
Minor issues exist. Address listed items before final approval or go-live.

Not Approved
Major gaps exist. Resolve critical issues before considering production.

6. Sign-Off

Reviewed By: _____
Role / Title: _____
Date: _____
Notes / Action Items: _____

Key Takeaway
Production readiness is not a single task. It is the result of good design, strong processes, prepared teams, and continuous validation.

VERIQTA

Senior Engineer Tip: The best time to find problems is before your users do. A strong PRR today prevents late-night incidents tomorrow.

VERIQTA
FINANCIAL SERVICES REFERENCE ARCHITECTURE

AWS SOLUTIONS ARCHITECT PRODUCTION HANDBOOK

ENGINEERING VERIQTA EXCELLENCE

Secure. Compliant. Highly Available. Cost Optimized.
Built for Banking, Payments, Insurance, and Capital Markets.

1. ARCHITECTURE OVERVIEW

This reference architecture provides a secure, scalable, and compliant cloud foundation for financial services workloads. It supports high availability, data protection, regulatory compliance, and real-time processing.

2. BUSINESS USE CASES

- ☑ Digital Banking & Mobile Apps
- ☑ Payment Processing & Gateways
- ☑ Trading Platforms & Market Data
- ☑ Insurance Policy & Claims Systems
- ☑ Fraud Detection & Risk Analytics
- ☑ Customer Onboarding (KYC/AML)
- ☑ Regulatory Reporting & Data Lakes

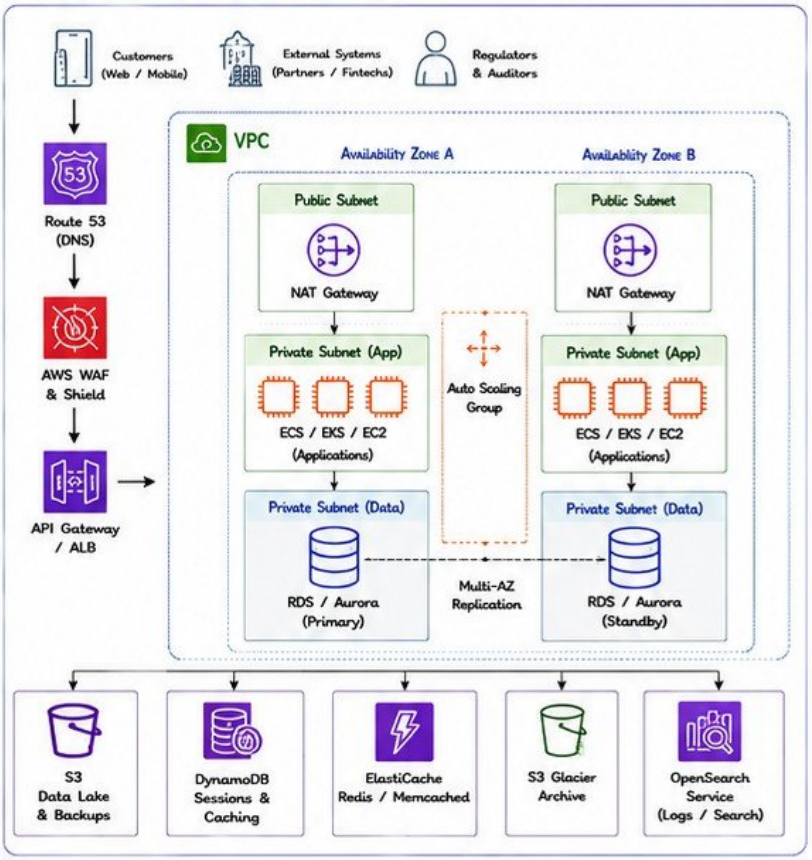
3. ARCHITECTURE PRINCIPLES

SECURITY FIRST	Zero Trust, least privilege, encryption, and strong identity controls.
COMPLIANCE BY DESIGN	Built to meet regulatory standards (PCI DSS, SOC 2, ISO 27001, GDPR, etc.).
HIGH AVAILABILITY	Multi-AZ, auto scaling, and resilient services.
PERFORMANCE & SCALE	Elastic services and caching for low latency and high throughput.
COST OPTIMIZED	Right-sizing, reserved capacity, automation, and monitoring.
OBSERVABILITY	Centralized logging, monitoring, tracing, and alerting.

4. COMPLIANCE & REGULATIONS

REGULATION	AWS SERVICES / FEATURES
PCI DSS	VPC, Security Groups, KMS, WAF, AWS Shield
GDPR	S3 Encryption, Data Residency, Macie, IAM
SOC 2	CloudTrail, Config, GuardDuty, AWS Artifact
ISO 27001	IAM, KMS, Security Hub, Organizations SCPs
FINRA / SEC	CloudTrail, S3 Immutable Storage, Glacier
AML / KYC	Amazon Comprehend, SageMaker, Rekognition

5. HIGH-LEVEL ARCHITECTURE DIAGRAM



Shared Services (Organization Level)

- IAM Identity Center (SSO)
- AWS Organizations & SCPs
- CloudTrail (Logging)
- AWS Config (Compliance)
- Security Hub (Posture Mgmt)
- CloudWatch (Monitoring)

6. DATA FLOW

1. Users access via web / mobile apps.
2. Traffic routed through Route 53 → WAF/Shield.
3. Requests forwarded to API Gateway / ALB.
4. Applications run in private subnets (ECS/EKS/EC2).
5. Data stored in RDS (Multi-AZ) and DynamoDB.
6. Files and analytics data stored in S3 Data Lake.
7. Logs, metrics, and alerts sent to CloudWatch.

7. KEY SERVICES

CATEGORY	KEY AWS SERVICES
NETWORKING	VPC, Route 53, ALB, NAT Gateway, AWS WAF, Shield
COMPUTE	ECS, EKS, EC2 Auto Scaling, Lambda
DATABASE	RDS (Aurora), DynamoDB, ElastiCache
STORAGE	S3, S3 Glacier, EFS, FSx
SECURITY	IAM, KMS, Secrets Manager, GuardDuty, Macie
OBSERVABILITY	CloudWatch, CloudTrail, X-Ray, OpenSearch
INTEGRATION	SNS, SQS, EventBridge, API Gateway

8. BENEFITS

- **Secure by Design** — Built with zero trust, encryption, and strong controls.
- **Regulatory Ready** — Meets global financial regulations and audit requirements.
- **High Availability** — Multi-AZ and resilient architecture for critical workloads.
- **Performance at Scale** — Low latency, elastic, and optimized for growth.
- **Cost Optimized** — Right services, automation, and continuous optimization.

KEY TAKEAWAY

Financial services systems demand security, compliance, availability, and performance.
This reference architecture provides a proven foundation for modern, cloud-native financial solutions.

VERIQTA

SENIOR ENGINEER TIP: Always align architecture with business risk, regulatory requirements, and operational resilience.

VERIQTA
HEALTHCARE REFERENCE ARCHITECTURE

AWS Solutions Architect Production Handbook

Secure. Compliant. Highly Available. Patient-Centric.
Built for Hospitals, Clinics, Telehealth, Labs, and Health-Tech.

1. Architecture Overview

This reference architecture enables healthcare organizations to deliver secure, compliant, and reliable applications. It supports patient data protection, interoperability (HL7/FHIR), telehealth, analytics, and AI/ML for better patient outcomes.

2. Business Use Cases

- [x] Electronic Health Records (EHR)
- [x] Patient Portal & Mobile Apps
- [x] Telehealth & Virtual Care
- [x] Medical Imaging & Diagnostics
- [x] Population Health Analytics
- [x] AI/ML for Clinical Insights
- [x] Billing, Claims & Revenue Cycle
- [x] Compliance & Reporting

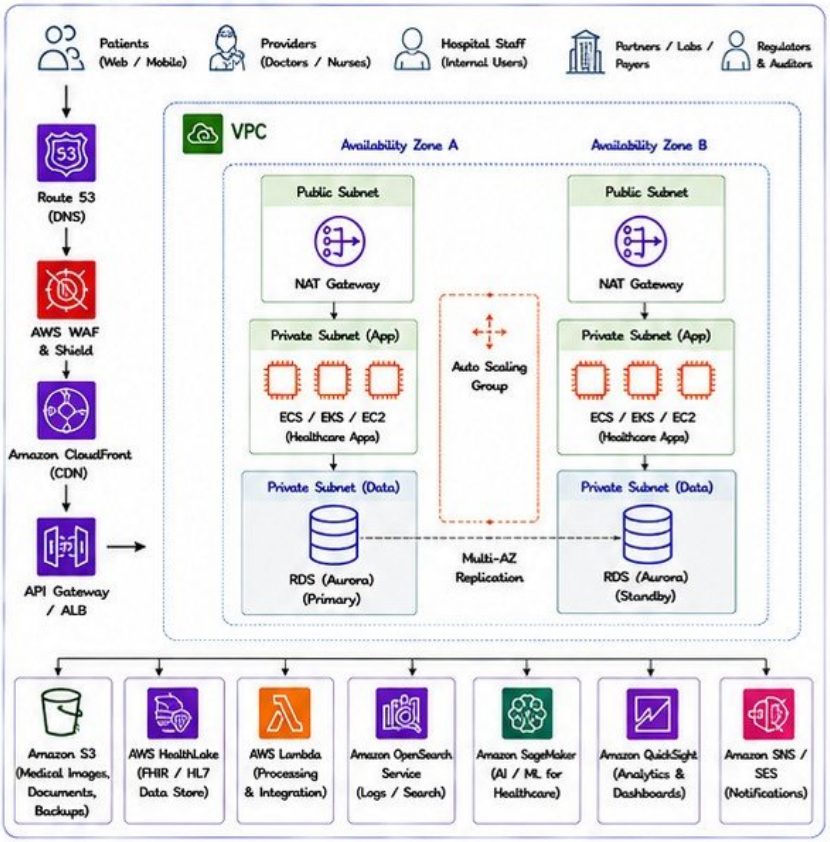
3. Architecture Principles

Security First	Protect PHI with encryption, least privilege, and zero trust.
Compliance by Design	HIPAA, HITRUST, PCI DSS, SOC 2, and GDPR aligned.
High Availability	Multi-AZ, auto scaling, and fault-tolerant services.
Performance & Scale	Elastic, serverless, and caching for high demand and real-time workloads.
Cost Optimized	Right-size, lifecycle policies, and managed services.
Observability	Centralized monitoring, logging, tracing, and alerting.

4. Compliance & Regulations

Regulation	AWS Services / Features
HIPAA	IAM, KMS, Secrets Manager, CloudTrail, AWS Config, S3 Encryption
HITRUST	AWS Artifact, Security Hub, GuardDuty, Inspector, Macie
SOC 2	CloudTrail, AWS Config, CloudWatch, Trusted Advisor
PCI DSS	VPC, Security Groups, WAF, Shield, KMS, ACM
GDPR	S3 Data Residency, Macie, KMS, Access Analyzer
HITECH	Encryption, Audit Logging, Access Controls
FHIR / HL7	API Gateway, Lambda, Amazon HealthLake

5. High-Level Architecture Diagram



6. Data Flow

1. Users access applications via web or mobile.
2. Traffic routed through Route 53, WAF/Shield, and CloudFront.
3. Requests handled by API Gateway / ALB.
4. Applications run on ECS / EKS / EC2 in private subnets.
5. Data stored in Aurora (Multi-AZ) and S3.
6. Files, images, and logs stored in S3 and processed by Lambda.
7. Logs, metrics, and alerts sent to CloudWatch.

7. Key AWS Services

Category	Key AWS Services
Networking	VPC, Route 53, CloudFront, WAF, Shield, NAT Gateway
Compute	ECS, EKS, EC2 Auto Scaling, Lambda
Database	RDS (Aurora), HealthLake, DynamoDB
Storage	S3, EFS, Glacier, FSx
Security	IAM, KMS, Secrets Manager, GuardDuty, Inspector
Compliance	AWS Config, Security Hub, CloudTrail, Artifact
Observability	CloudWatch, X-Ray, OpenSearch, CloudTrail
Integration	API Gateway, SNS, SQS, EventBridge, SES

8. Benefits

Secure by Design
Protect patient data (PHI) with encryption, access controls, and monitoring.

Compliant & Audit Ready
Meets HIPAA, HITRUST, SOC 2, PCI DSS, and other healthcare regulations.

Highly Available
Multi-AZ, auto scaling, and resilient architecture for critical healthcare workloads.

Patient-Centric
Enable better care with real-time data, analytics, and AI-powered insights.

Cost Optimized
Use managed services, right-sizing, and lifecycle policies to control costs.

Key Takeaway
This healthcare reference architecture ensures security, compliance, availability, and performance—empowering providers to focus on delivering better patient outcomes.

VERIQTA

Senior Engineer Tip: Always design for compliance, data protection, and interoperability. Healthcare systems are mission-critical—build with reliability and auditability from day one.

VERIQTA
E-COMMERCE REFERENCE ARCHITECTURE

AWS Solutions Architect Production Handbook

ENGINEERING VERIQTA EXCELLENCE

Secure. Scalable. Highly Available. Cost Optimized.
Built for Online Stores, Marketplaces, and Digital Commerce Platforms.

1. Architecture Overview

This reference architecture supports e-commerce platforms with high traffic, secure transactions, fast performance, and seamless scalability during peak events.

2. Business Use Cases

- ☑ Online Storefronts
- ☑ Product Catalog & Search
- ☑ Shopping Cart & Checkout
- ☑ User Accounts & Profiles
- ☑ Order Management
- ☑ Payments & Invoicing
- ☑ Inventory & Fulfillment
- ☑ Recommendations & Analytics

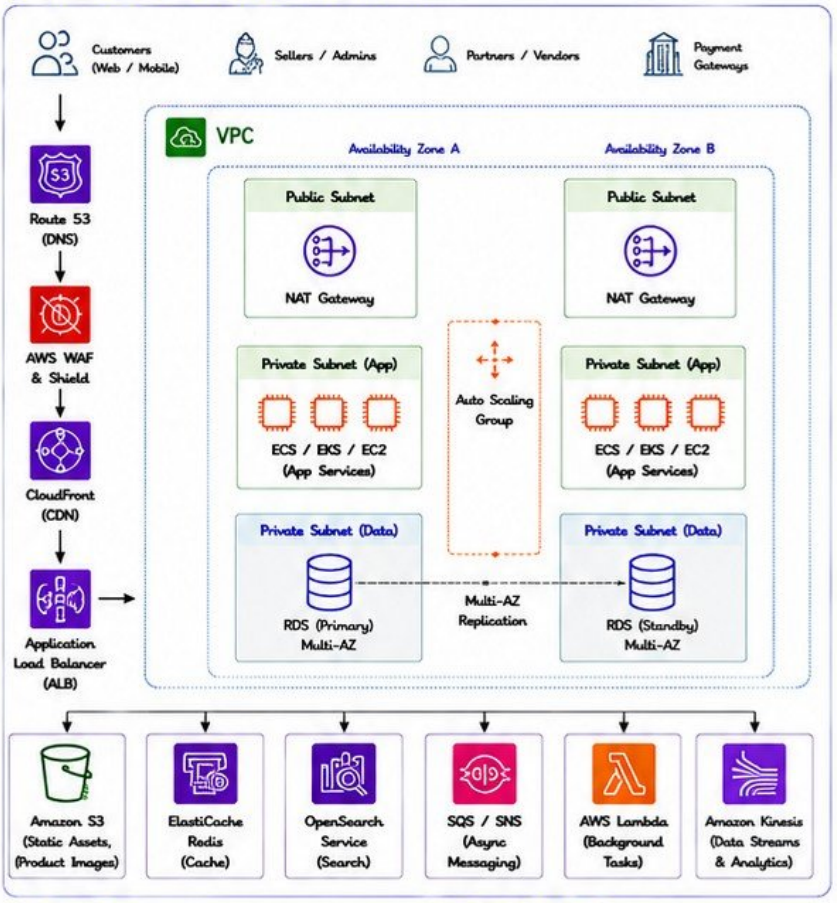
3. Architecture Principles

Security First	Zero trust, encryption, least privilege, and strong controls.
Compliance by Design	PCI DSS, GDPR, SOC 2, and data protection built-in.
High Availability	Multi-AZ, auto scaling, health checks, and fault tolerance.
Performance & Scale	Caching, CDN, read replicas, and async processing.
Cost Optimized	Right-sizing, reserved capacity, and managed services.
Observability	Centralized logging, metrics, tracing, and alerting.

4. Compliance & Regulations

Regulation	AWS Services / Features
PCI DSS	VPC, Security Groups, WAF, Shield, KMS, ACM, AWS Config
GDPR	S3 Encryption, Macie, IAM, Data Residency, Access Analyzer
SOC 2	CloudTrail, GuardDuty, AWS Config, Systems Manager
CCPA	AWS Glue, Lake Formation, Data Catalog, IAM
ISO 27001	IAM, KMS, CloudTrail, Security Hub
AML / KYC	Amazon Comprehend, Rekognition, SageMaker, Textract

5. High-Level Architecture Diagram



Shared Services (Account Level)

- IAM Identity Center (SSO)
- AWS Organizations & SCPs
- AWS Config (Compliance)
- AWS Security Hub (Posture Mgmt)
- CloudTrail (Logging)
- CloudWatch (Monitoring)
- AWS Backup & Recovery

6. Data Flow

1. Users request content via Route 53 and CloudFront.
2. Traffic filtered by WAF/Shield and routed to ALB.
3. ALB forwards requests to app containers (ECS/EKS/EC2).
4. App retrieves data from Cache or RDS (Multi-AZ).
5. Static assets served from S3 via CloudFront.
6. Background tasks processed via SQS/SNS and Lambda.
7. Logs, metrics, and events streamed to CloudWatch.

7. Key AWS Services

Category	Key AWS Services
Networking	VPC, Route 53, CloudFront, ALB, NAT Gateway, WAF, Shield
Compute	ECS, EKS, EC2 Auto Scaling, Lambda
Database	RDS (Aurora), ElastiCache, DynamoDB (Sessions/Cart)
Storage	S3, EFS, Glacier, FSx
Integration	SQS, SNS, EventBridge, Step Functions, API Gateway
Analytics	Kinesis, Redshift (Analytics), QuickSight (BI)
Security	IAM, KMS, Secrets Manager, GuardDuty, Inspector
Observability	CloudWatch, CloudTrail, X-Ray, OpenSearch Service

8. Benefits

- **Secure by Design** — Protects data, payments, and customer privacy with strong controls.
- **Highly Available** — Multi-AZ, auto scaling, and fault tolerant for peak traffic.
- **High Performance** — CDN, caching, search, and optimized delivery for fast user experience.
- **Scalable & Elastic** — Handles growth, flash sales, and seasonal traffic effortlessly.
- **Cost Optimized** — Efficient resource usage and managed services reduce cost.

Key Takeaway
This e-commerce reference architecture delivers security, scalability, performance, and cost efficiency to provide an excellent online shopping experience.

VERIQTA

Senior Engineer Tip: Design for scale from day one. Automate everything, monitor deeply, and test for peak loads.

VERIQTA SaaS Multi-Tenant Architecture

AWS Solutions Architect Production Handbook

Secure. Scalable. Isolated. Highly Available.
Built for Modern SaaS Platforms with Multiple Customers.

1. Architecture Overview

This architecture supports SaaS applications with multiple tenants, providing secure isolation, elastic scalability, high availability, and tenant-aware operations at scale.

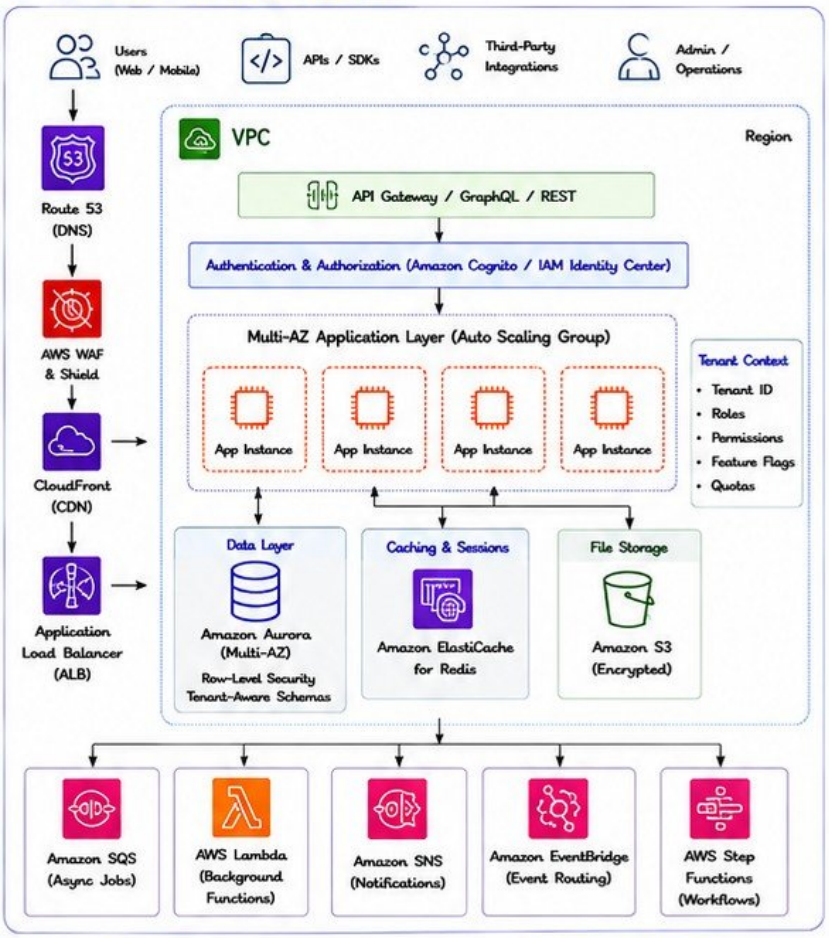
2. Multi-Tenancy Models

Single Tenant	Dedicated infrastructure per tenant. Maximum isolation.
Multi-Tenant (Shared)	Shared infrastructure with logical isolation at application layer.
Hybrid Model	Critical tenants get dedicated resources, others share.

3. Architecture Principles

Security First	Tenant isolation, least privilege, encryption, and strong identity controls.
Isolation	Logical isolation with tenant context, row-level security, and data segregation.
Scalability	Elastic compute, auto scaling, and serverless services for unpredictable workloads.
High Availability	Multi-AZ, fault tolerance, health checks, and automatic failover.
Cost Optimization	Right-sizing, reserved capacity, shared services, and usage monitoring.
Observability	Centralized logging, metrics, tracing, alerting, and tenant-aware dashboards.

4. High-Level Architecture Diagram



Shared Services (Account Level)

- IAM Identity Center (SSO)
- AWS Organizations & SCPs
- AWS Config (Compliance)
- CloudTrail (Audit Logs)
- CloudWatch (Monitoring)
- AWS Backup (Backup & DR)
- Amazon GuardDuty (Threat Detection)

5. Data Isolation Strategies

1. Users access the application via Route 53 and CloudFront.
2. Requests go through WAF / Shield and reach ALB.
3. ALB forwards traffic to the application layer (Auto Scaling).
4. Application accesses tenant data in Aurora with RLS.
5. Caching layer (Redis) improves performance.
6. Async jobs, events, and notifications handled by SQS, Lambda, SNS, and EventBridge.

7. Key AWS Services

Category	Key AWS Services
Networking	VPC, Route 53, CloudFront, ALB, WAF, Shield
Compute	ECS / EKS / EC2 Auto Scaling, Lambda
Database	Amazon Aurora (Multi-AZ), RDS Proxy
Cache	Amazon ElastiCache for Redis
Storage	Amazon S3, EFS
Identity	Cognito, IAM Identity Center, IAM
Observability	CloudWatch, X-Ray, CloudTrail, GuardDuty
Integration	API Gateway, SQS, SNS, EventBridge, Step Functions

8. Benefits

- **Secure by Design** — Tenant isolation, encryption, least privilege, and audit logging.
- **Highly Scalable** — Elastic resources handle growth without downtime.
- **High Performance** — Caching, CDN, and optimized data access for great UX.
- **Cost Efficient** — Shared services, right-sizing, and automation reduce operational cost.
- **Operational Excellence** — Observability, automation, and self-service empower teams and tenants.

Key Takeaway
A well-designed SaaS multi-tenant architecture balances isolation, security, performance, and cost to deliver a reliable and scalable platform that grows with your customers.

Senior Engineer Tip: Always design for tenant isolation, data protection, and observability from day one. Your future self (and your customers) will thank you.

VERIQTA STREAMING PLATFORM ARCHITECTURE

AWS Solutions Architect Production Handbook

High Performance. Highly Available. Globally Scalable.
Built for Live Streaming, Video-on-Demand, and Real-Time Experiences.

1. Architecture Overview

This architecture delivers live streaming, on-demand video, and real-time content at global scale. It supports millions of concurrent viewers with low latency, high availability, and cost efficiency.

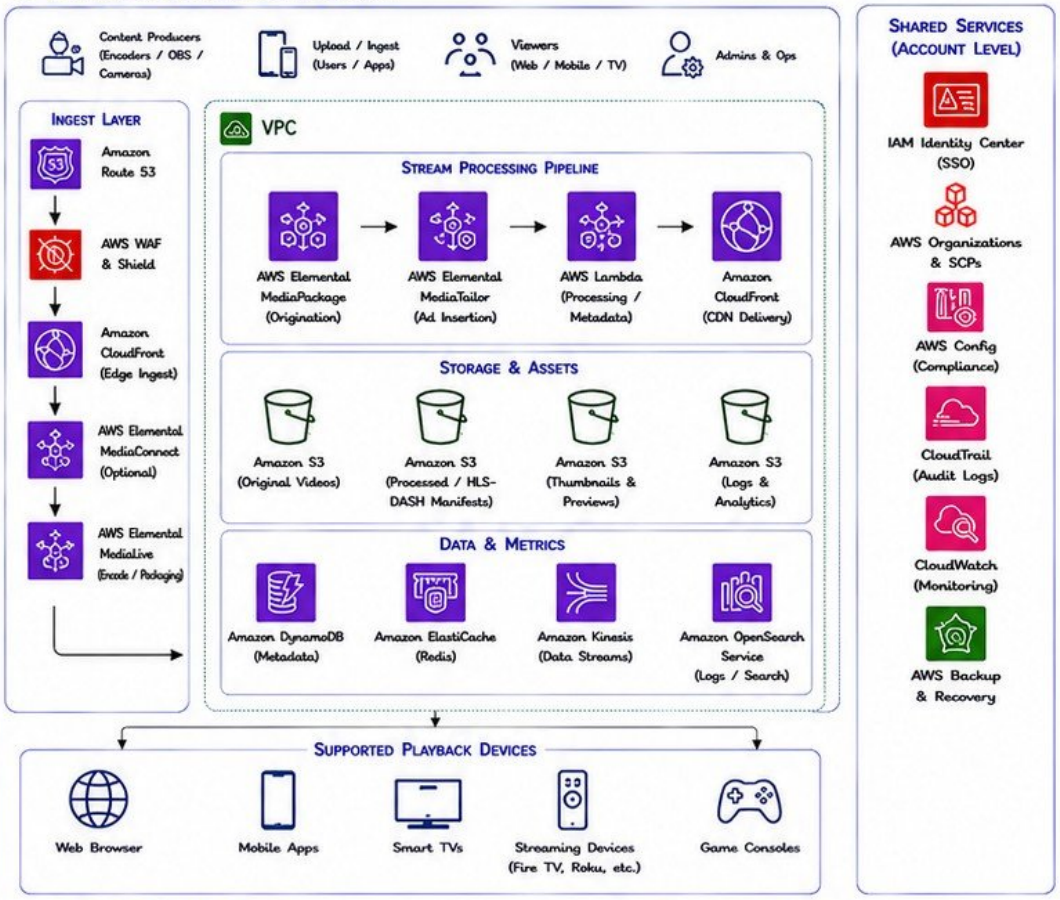
2. Use Cases

- ☑ Live Streaming Events
- ☑ Video-on-Demand (VOD)
- ☑ Online Courses & Webinars
- ☑ Real-Time Gaming Streams
- ☑ Live Sports & Broadcast
- ☑ User Generated Content
- ☑ Multi-Device Playback
- ☑ Global Content Delivery

3. Architecture Principles

Security First	Encrypt content, control access, and protect against abuse.
High Availability	Multi-AZ services and auto scaling for continuous delivery.
Low Latency	Optimized pipelines and CDN for minimal delay.
Global Scale	Edge delivery and replication for worldwide reach.
Cost Optimization	Use serverless, auto scaling, and intelligent storage tiers.
Observability	Monitor streams, quality, and user experience end-to-end.

4. High-Level Architecture Diagram



5. Data Flow

1. Content is captured and sent to MediaLive for encoding.
2. Encoded streams are packaged by MediaPackage.
3. (Optional) Ads inserted via MediaTailor for monetization.
4. Content stored in S3 and metadata in DynamoDB.
5. Content delivered globally through CloudFront CDN.
6. Viewers request content via optimal edge locations.
7. Logs, metrics, and quality data captured in Kinesis and OpenSearch for analytics.

6. Key AWS Services

Category	Key AWS Services
Networking	Route 53, CloudFront, WAF, Shield
Media Services	MediaLive, MediaPackage, MediaTailor, MediaConnect
Compute	Lambda
Storage	S3, EFS
Database	DynamoDB, ElastiCache (Redis)
Streaming Data	Kinesis Data Streams
Analytics	OpenSearch Service, CloudWatch
Security	IAM, KMS, Shield, WAF, Secrets Manager
Monitoring	CloudWatch, CloudTrail, AWS Config

7. Performance & Scale

- Auto scaling encoders and Lambda functions.
- Global CDN with edge locations for low latency.
- Segmented HLS / DASH for adaptive streaming.
- Horizontal scaling with serverless services.
- Supports millions of concurrent viewers.

8. Benefits

- **Reliable & Highly Available** — Multi-AZ services and CDN ensure uninterrupted streaming.
- **Global Reach** — Deliver content quickly to viewers anywhere in the world.
- **Scalable & Elastic** — Handle traffic spikes and large live events effortlessly.
- **Cost Optimized** — Pay for what you use. Use serverless and storage tiers.
- **Secure Content Delivery** — End-to-end encryption, access control, and anti-abuse protection.

Key Takeaway
This streaming architecture provides a robust, scalable, and secure foundation for delivering live and on-demand video experiences at global scale with excellent performance and cost efficiency.

VERIQTA

Senior Engineer Tip: Design for peaks, test for failures, and monitor user experience. Great streaming platforms are built with resilience, observability, and customer delight in mind.

VERIQTA

VERIQTA AI PLATFORM ARCHITECTURE

AWS SOLUTIONS ARCHITECT PRODUCTION HANDBOOK

ENGINEERING VERIQTA EXCELLENCE

Secure. Scalable. Trusted. Responsible.

Built for Enterprise AI/ML Workloads, MLOps, and Generative AI Applications.

1. ARCHITECTURE OVERVIEW

This architecture provides an end-to-end platform for building, training, deploying, and operating AI/ML and Generative AI solutions at scale with security, governance, observability, and cost efficiency.

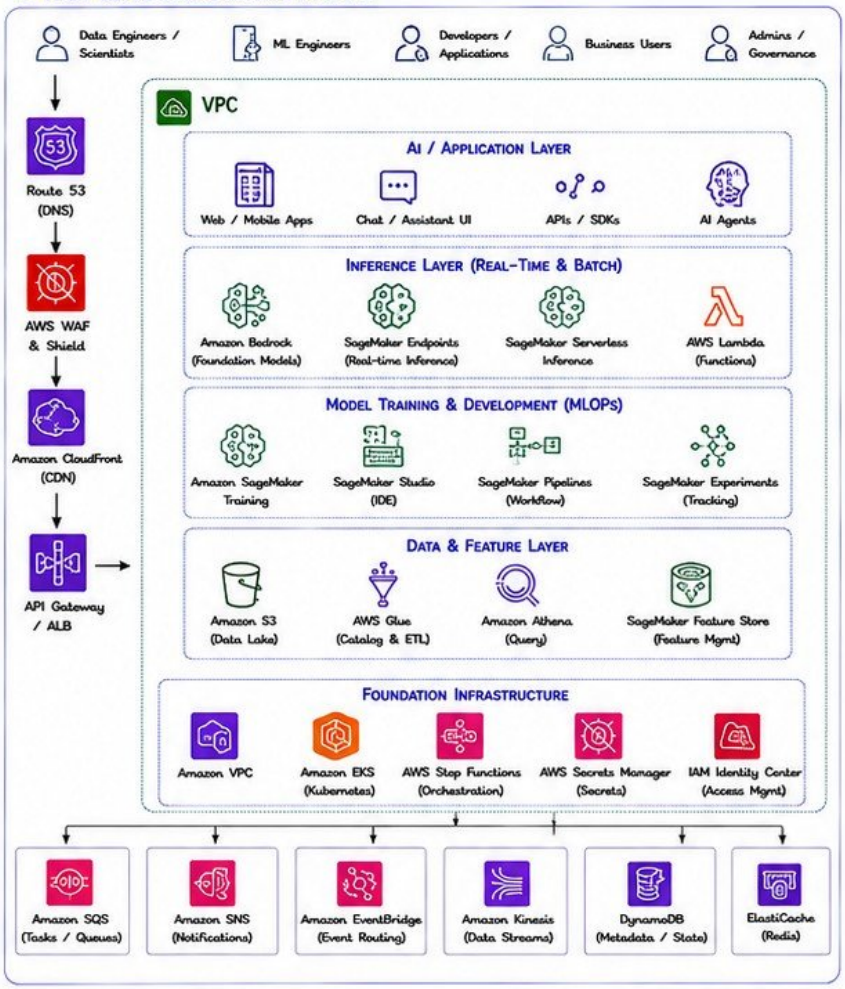
2. USE CASES

- ☑ Machine Learning (ML) Models
- ☑ Generative AI Applications
- ☑ Natural Language Processing
- ☑ Computer Vision
- ☑ Recommendation Engines
- ☑ AI-Powered Search
- ☑ Intelligent Automation (RPA + AI)
- ☑ Predictive Analytics & Forecasting

3. ARCHITECTURE PRINCIPLES

SECURITY FIRST	End-to-end encryption, least privilege, and data protection by design.
DATA PRIVACY	PII protection, data masking, consent management, and privacy compliance.
SCALABILITY	Elastic training & inference, auto scaling, and serverless where possible.
RELIABILITY	Multi-AZ, fault tolerant, retry logic, and resilient pipelines.
RESPONSIBLE AI	Bias detection, explainability, model monitoring, and ethical AI practices.
COST OPTIMIZATION	Right-sizing, spot training, caching, and efficient resource utilization.
OBSERVABILITY	End-to-end monitoring, tracing, metrics, logs, and AI model observability.

4. HIGH-LEVEL ARCHITECTURE DIAGRAM



SHARED SERVICES (ACCOUNT LEVEL)

- IAM Identity Center (SSO)
- AWS Organizations & SCPs
- AWS Config (Compliance)
- Amazon CloudTrail (Audit Logs)
- AWS Security Hub (Posture Mgmt)
- AWS Backup & Recovery
- Amazon GuardDuty (Threat Detection)

5. DATA FLOW

1. Data is ingested via APIs, apps, or streaming into the data lake.
2. ETL & cataloging with AWS Glue: data stored in S3 (encrypted).
3. Features engineered and stored in SageMaker Feature Store.
4. Models trained using SageMaker; experiments tracked.
5. Models deployed to endpoints (real-time or batch inference).
6. Applications call inference endpoints via APIs / SDKs.
7. Logs, metrics, and predictions captured for monitoring & analysis.

6. KEY AWS SERVICES

CATEGORY	KEY AWS SERVICES
NETWORKING	VPC, Route 53, CloudFront, API Gateway, WAF, Shield
COMPUTE	SageMaker, Lambda, EKS, EC2 (GPU instances)
DATA	S3, Glue, Athena, DynamoDB, Kinesis
AI / ML	SageMaker, Bedrock, Feature Store, Experiments
INTEGRATION	Step Functions, EventBridge, SQS, SNS
SECURITY	IAM, Secrets Manager, KMS, Security Hub, GuardDuty
MONITORING	CloudWatch, CloudTrail, X-Ray, Config

7. MLOPS CAPABILITIES

- ☑ CI/CD pipelines for models using SageMaker Pipelines
- ☑ Model registry, versioning, and approval workflows
- ☑ Automated testing: data, model quality, and drift
- ☑ Canary / shadow deployments for safe rollouts
- ☑ Continuous monitoring of data & model performance
- ☑ Retraining triggers based on drift & performance

8. BENEFITS

Secure & Compliant
Built with enterprise-grade security, governance, and compliance controls.

Scalable & Elastic
Auto scaling training & inference for any workload size.

Fast & Reliable
Optimized pipelines, low-latency inference, and high availability end-to-end.

Innovate Faster
Modern MLOps platform accelerates AI delivery and experimentation.

Cost Efficient
Right-size compute, use managed services, and optimize costs continuously.

KEY TAKEAWAY
This AI platform architecture provides a secure, scalable, and end-to-end foundation for building, deploying, and operating intelligent applications with modern MLOps and responsible AI practices.

VERIQTA

SENIOR ENGINEER TIP: Invest in data quality, observability, and model monitoring. The best AI systems in production are measured by reliability, trust, and business impact, not just model accuracy.

VERIQTA ENTERPRISE DATA PLATFORM

Secure. Scalable. Governed. Trusted.

Unified data platform for analytics, AI, and business intelligence at scale.

1. Architecture Overview

This architecture provides a centralized, secure, and governed data platform for collecting, storing, processing, and serving enterprise data. It enables advanced analytics, BI, machine learning, and real-time insights.

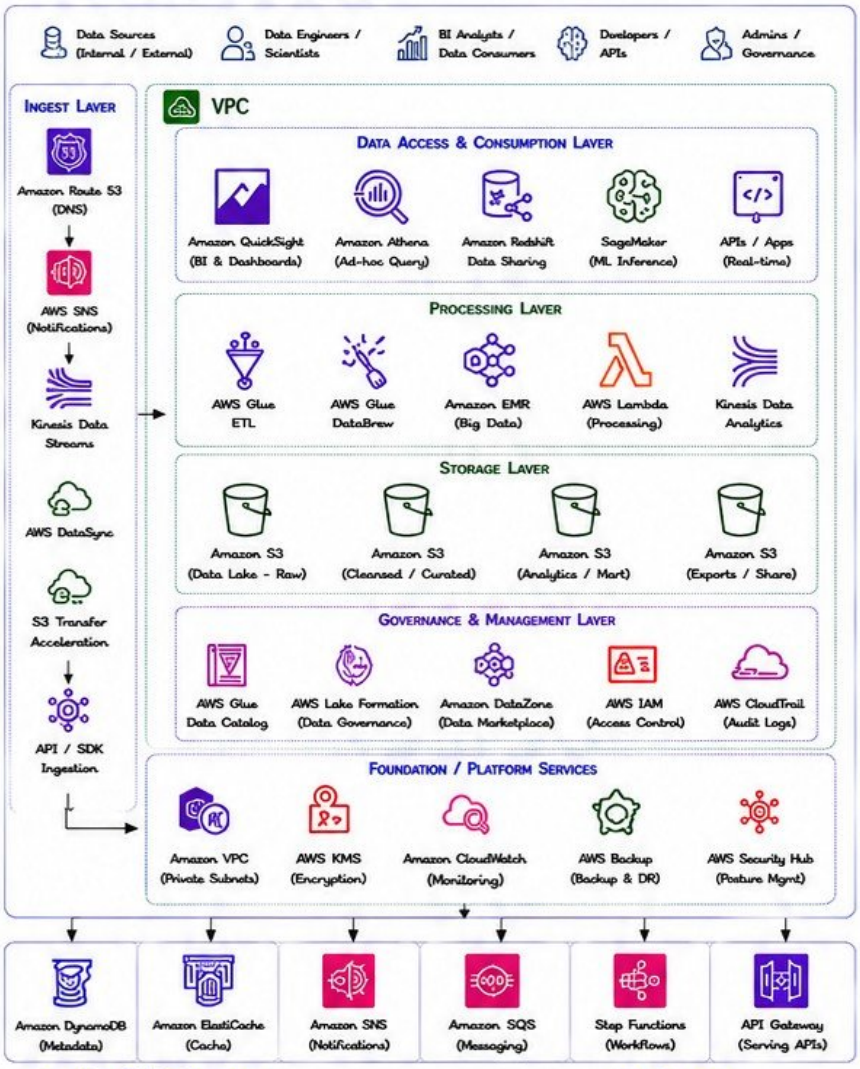
2. Use Cases

- ☑ Enterprise Reporting & BI
- ☑ Self-Service Analytics
- ☑ Real-Time Dashboards
- ☑ Customer 360 & Data Marts
- ☑ Machine Learning & AI
- ☑ Data Science Experiments
- ☑ Data Sharing & Monetization
- ☑ Regulatory & Compliance

3. Architecture Principles

Security First	Least privilege, encryption, network isolation, and continuous monitoring.
Data Governance	Data catalog, lineage, quality, ownership, and policy enforcement.
Scalability	Decoupled layers, auto scaling, and serverless where possible.
Reliability	Multi-AZ, backup, monitoring, and disaster recovery.
Performance	Optimized storage formats, caching, partitioning, and query acceleration.
Cost Optimization	Right-size resources, lifecycle policies, and usage monitoring.
Observability	Centralized logs, metrics, tracing, and alerts.

4. High-Level Architecture Diagram



5. Data Flow

1. Data is ingested from multiple sources via batch, streaming, APIs, or file transfers.
2. Raw data is stored in S3 (Data Lake - Raw).
3. Data is processed, cleansed, and transformed using Glue, EMR, Lambda, or Kinesis.
4. Curated data is stored in S3 (Cleansed / Curated).
5. Analytics-ready data is loaded into S3 (Mart) or Redshift for consumption.
6. Users access data via BI tools, SQL, APIs, or ML applications.
7. Data is governed, monitored, and secured across the platform.

6. Key AWS Services

Category	Key AWS Services
Ingestion	DMS, Kinesis, DataSync, S3 Transfer Acceleration
Processing	AWS Glue, EMR, Lambda, Kinesis Analytics
Storage	Amazon S3, Redshift
Catalog & Gov	AWS Glue Data Catalog, Lake Formation, DataZone
Analytics	Athena, Redshift, QuickSight, SageMaker
Integration	API Gateway, Step Functions, SQS, SNS
Database	DynamoBB, ElastiCache
Security	IAM, KMS, Security Hub, GuardDuty, CloudTrail
Monitoring	CloudWatch, Config, CloudTrail, Backup

7. Benefits

- ☑ Unified platform for all enterprise data.
- ☑ Secure, governed, and compliant by design.
- ☑ Scalable to petabytes of data.
- ☑ Supports real-time, batch, and ML workloads.
- ☑ Enables self-service analytics and data sharing.
- ☑ High availability, backup, and disaster recovery.
- ☑ Cost optimized with right storage and compute.

Secure & Governed — End-to-end security, encryption, governance, and compliance.

Scalable & Elastic — Handles growing data volumes and users seamlessly.

Performance Optimized — Fast queries, caching, partitioning, and query acceleration.

Data-Driven Insights — Power BI, AI/ML, and real-time decisions across the business.

Cost Effective — Optimize storage, compute, and data lifecycle for savings.

Key Takeaway

An enterprise data platform centralizes data, ensures governance, and delivers reliable, scalable, and high-performance insights for analytics, AI, and business operations.

Senior Engineer Tip: Design for governance first, automate everything, monitor continuously, and optimize for cost and performance. Great data platforms turn data into business impact.

VERIQTA
LARGE KUBERNETES PLATFORM ON AWS

AWS SOLUTIONS ARCHITECT PRODUCTION HANDBOOK

ENGINEERING VERIQTA EXCELLENCE

Secure. Scalable. Resilient. Highly Available.
Enterprise-grade Kubernetes platform for massive scale on AWS.

1. ARCHITECTURE OVERVIEW

This architecture shows a large, multi-account, multi-region Kubernetes platform on AWS using EKS, VPC CNI, ALB Ingress, AWS services, and observability at enterprise scale.

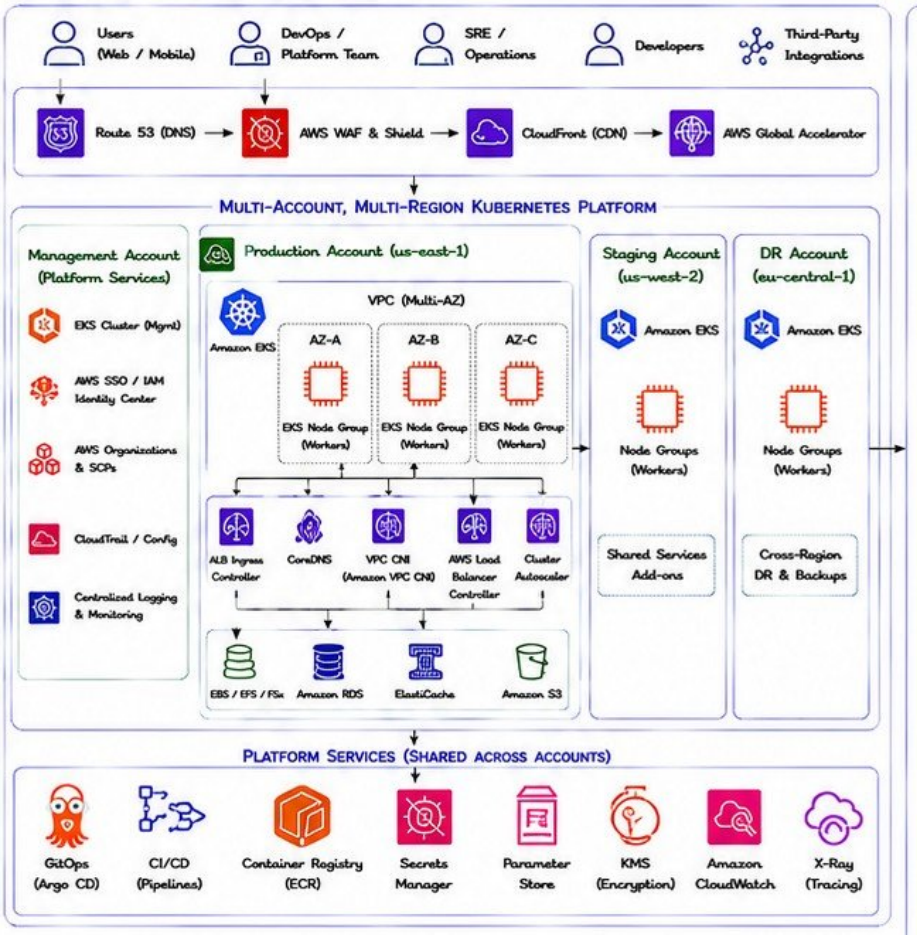
2. PLATFORM CAPABILITIES

- ☑ Multi-Account, Multi-Region
- ☑ Highly Available & Resilient
- ☑ Auto Scaling & Self-Healing
- ☑ GitOps & CI/CD Integration
- ☑ Security, Compliance & IAM
- ☑ Observability & Monitoring
- ☑ Network Segmentation
- ☑ Cost Optimization

3. ARCHITECTURE PRINCIPLES

SECURITY FIRST	Zero trust, least privilege, encryption in transit and at rest.
HIGH AVAILABILITY	Multi-AZ, multi-region, auto-healing, and fault tolerance.
SCALABILITY	Horizontal & vertical scaling for nodes and applications.
RELIABILITY	Health checks, rollbacks, backups, and disaster recovery.
OBSERVABILITY	Centralized logs, metrics, traces, and alerts.
COST OPTIMIZATION	Right-sizing, savings plans, spot instances, and lifecycle policies.

4. HIGH-LEVEL ARCHITECTURE DIAGRAM



5. TRAFFIC FLOW

1. Users resolve DNS via Route 53.
2. Traffic protected by WAF, Shield, and Global Accelerator.
3. Requests served via CloudFront (if static) or direct to ALB.
4. ALB Ingress routes traffic to Kubernetes services.
5. Services communicate internally via Kubernetes networking.
6. Data stored in AWS managed databases & storage.
7. Logs, metrics, and traces shipped to observability stack.
8. Backups replicated cross-region for disaster recovery.

6. KEY AWS SERVICES

CATEGORY	KEY AWS SERVICES
COMPUTE	Amazon EKS, EC2 (Worker Nodes), Fargate
NETWORKING	VPC, ALB, NLB, Route 53, CloudFront, VPC CNI
STORAGE	EBS, EFS, FSx, S3, ECR
DATABASE	RDS, Aurora, ElastiCache, DynamoDB
SECURITY	IAM, IAM Identity Center, KMS, Secrets Manager, WAF
OBSERVABILITY	CloudWatch, CloudTrail, X-Ray, Prometheus, Grafana
DEVOPS	CodePipeline, CodeBuild, GitHub/CodeCommit, Argo CD
GOVERNANCE	Organizations, SCPs, Config, Security Hub, GuardDuty

7. BENEFITS

- ☑ Massive scale with high availability and resilience.
- ☑ Centralized governance and security at enterprise level.
- ☑ Self-service platform for developers and teams.
- ☑ Automated deployments with GitOps and CI/CD.
- ☑ Full observability with metrics, logs, and traces.
- ☑ Cross-region disaster recovery and data protection.
- ☑ Cost optimized with savings plans and spot instances.
- ☑ Built for performance, reliability, and compliance.

GLOBAL SCALE — Multi-region, multi-account architecture built for enterprise scale.

HIGH RESILIENCE — Multi-AZ, auto-healing, and cross-region disaster recovery.

SECURE BY DESIGN — Zero trust, encryption, least privilege, and continuous compliance.

DEVELOPER EXPERIENCE — Self-service, GitOps, CI/CD, and reusable platform components.

OPERATIONAL EXCELLENCE — Observability, automation, and continuous improvement.

COST EFFICIENT — Right-size resources, optimize workloads, reduce waste.

KEY TAKEAWAY

A large Kubernetes platform on AWS requires a strong foundation of security, scalability, automation, and observability. Design once, automate everything, and scale with confidence.

VERIQTA

SENIOR ENGINEER TIP: Design for failure. Test failover, monitor everything, and automate recovery. The best platforms are boring because they just work.

VERIQTA GLOBAL MULTI-REGION DESIGN

AWS Solutions Architect Production Handbook

Resilient. Fast. Available Everywhere.

Design for failure. Deliver low latency. Operate globally.

1. Architecture Overview

This architecture provides a global, active-active or active-passive design across multiple AWS Regions for high availability, disaster recovery, low latency, and data sovereignty.

2. Design Principles

Resilience	Survive Region failures with minimal impact.
Low Latency	Route users to the closest Region.
High Availability	Active-active or active-passive across Regions.
Data Sovereignty	Store data in Regions that meet local laws.
Scalability	Scale horizontally across Regions on demand.
Cost Optimization	Right-size, automate, and use reserved capacity where possible.
Observability	Centralized visibility across all Regions.
Automation	Infrastructure as Code, policy as code, and automated recovery.

4. Traffic Management Modes

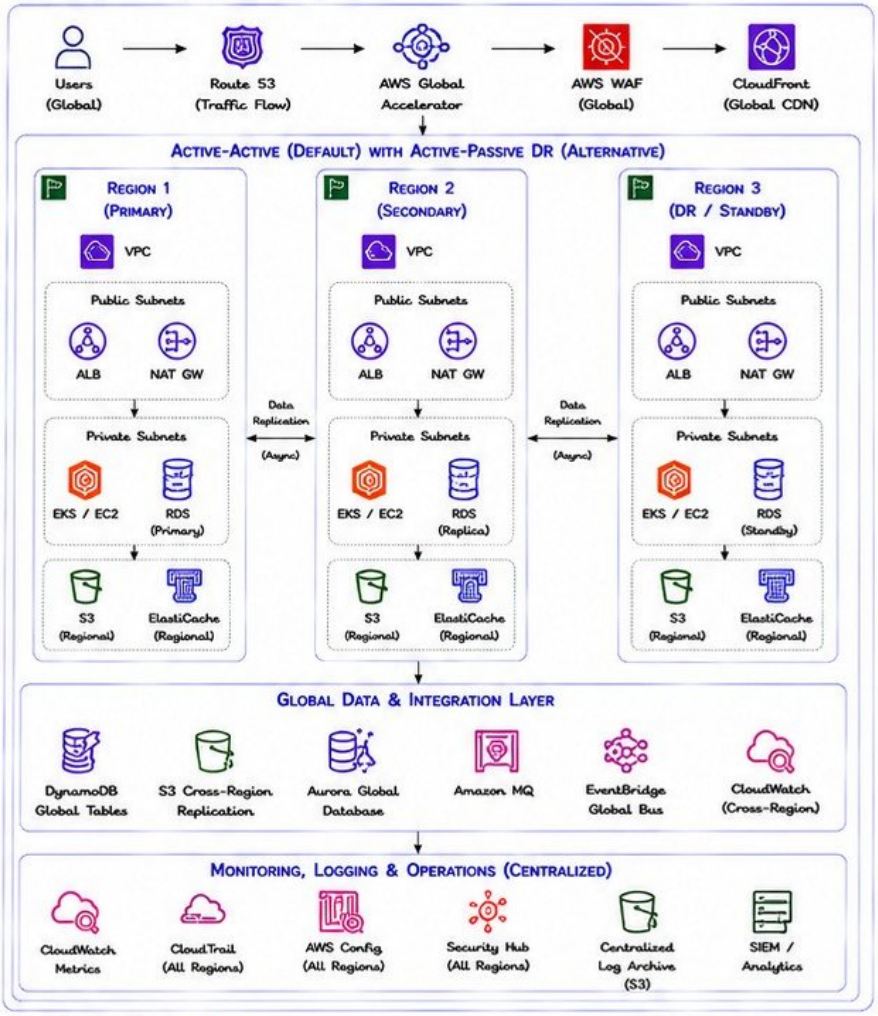
Active-Active (Recommended)

- ☑ Traffic distributed across Regions.
- ☑ Best for low latency & high availability.
- ☑ Requires multi-master or eventual consistency.

Active-Passive (DR)

- ☑ Primary Region handles production traffic.
- ☑ Standby Region used during failure.
- ☑ Simpler data architecture and lower cost.

3. Global Architecture Diagram



Shared Services (Centralized Account)

- IAM Identity Center (SSO)
- AWS Organizations & SCPs
- AWS Config (Compliance)
- CloudTrail (Audit Logs)
- Security Hub (Posture Mgmt)
- Amazon GuardDuty (Threat Detection)
- AWS Backup (Cross-Region)
- Central Log Archive (S3)
- Cost Explorer & Budgets

6. Key AWS Services

Category	Key Services
Networking	Route 53, Global Accelerator, CloudFront, VPC
Compute	Amazon EKS, EC2 Auto Scaling, Lambda
Storage	S3, EBS, EFS, EBS Snapshots, S3 CRR
Database	Aurora Global Database, DynamoDB Global Tables
Messaging	SQS, SNS, EventBridge Global Bus, Amazon MQ
Security	IAM, WAF, Shield, Security Hub, KMS
Monitoring	CloudWatch, CloudTrail, AWS Config
Governance	Organizations, SCPs, Control Tower, Config Rules

7. Benefits

- ☑ Low latency for global users.
- ☑ High availability with automatic failover.
- ☑ Protection against Region outages.
- ☑ Compliance with data residency laws.
- ☑ Improved performance with edge services.
- ☑ Centralized governance and visibility.
- ☑ Cost-efficient with flexible scaling.

Global Resilience — Built to survive Region failures without downtime.

User Experience — Route users to the closest healthy Region instantly.

Data Protection — Replicate and back up data across Regions.

Operational Excellence — Centralized monitoring, automation, and governance.

Cost Aware — Optimize resources globally and reduce waste.

Key Takeaway

A global multi-region design ensures your applications are fast, resilient, and compliant—no matter where your users are or what failures occur.

Senior Engineer Tip: Test your failover. Architect for failure. If you haven't tested it, it doesn't exist. Reliability is proven, not assumed.

VERIQTA PRODUCTION DEBUGGING WORKFLOW

AWS Solutions Architect Production Handbook

Observe. Isolate. Analyze. Fix. Verify.

A systematic workflow for fast root cause analysis and resolution.

1. Overview

This workflow helps you debug production issues faster by following a structured, repeatable process. It reduces MTTR, improves accuracy, and prevents guesswork.

2. Core Principles

Observe First	Gather facts and understand the blast radius.
Isolate	Narrow down the problem scope and eliminate noise.
Analyze Deeply	Find patterns, correlate data, and identify the root cause.
Fix Safely	Apply the right fix with minimal risk.
Verify & Learn	Confirm resolution and create knowledge for the future.

3. Debugging Habits

- [x] Stay calm and think clearly
- [x] Follow the workflow step-by-step
- [x] Collect evidence before making changes
- [x] Communicate status frequently
- [x] Document root cause and solution
- [x] Improve systems to prevent recurrence

4. Production Debugging Workflow (Step-by-Step)

1 Observe

Goal: Understand the issue and its impact.

Key Actions
- Check alerts, dashboards, and on-call notes
- Confirm affected users, services, and regions
- Identify when the issue started
- Assess severity and business impact
- Create incident and communicate

Tools: CloudWatch, Datadog, PagerDuty, Grafana, Status pages, Slack / Teams

2 Isolate

Goal: Narrow down the problem scope.

Key Actions
- Identify which layer is failing (DNS, Network, Load Balancer, App, DB, etc.)
- Check recent changes / deployments
- Compare with unaffected components
- Reproduce the issue (if safe)

Tools: VPC Flow Logs, Route53, ELB Access Logs, Kubernetes (kubectl), Git history, CI/CD logs

3 Analyze

Goal: Find the root cause using data and correlation.

Key Actions
- Check logs around the time of failure
- Correlate across services and tiers
- Look for errors, exceptions, timeouts
- Analyze metrics and resource usage
- Identify the root cause (not the symptom)

Tools: CloudWatch Logs Insights, X-Ray, Prometheus, Kibana / OpenSearch, Application Logs

4 Fix

Goal: Apply the correct fix with minimal risk.

Key Actions
- Validate the fix in a safe way
- Apply change (rollback, patch, scale, config)
- Monitor closely after applying the fix
- Document what was changed
- Communicate resolution to stakeholders

Tools: AWS CLI / Console, kubectl, Helm, CI/CD Pipelines, Feature Flags

5 Verify

Goal: Confirm the issue is fully resolved.

Key Actions
- Verify metrics return to normal
- Check logs for new errors
- Confirm user experience is back to normal
- Close the incident
- Monitor for a period (watch for regression)

Tools: Dashboards, Alerts, Synthetic Checks, APM Tools, User Monitoring

6 Learn

Goal: Capture knowledge and prevent recurrence.

Key Actions
- Write postmortem (blameless)
- Document root cause and timeline
- Identify improvements and action items
- Update runbooks and alerts
- Share learnings with the team

Tools: Confluence / Notion, Runbooks, Postmortem Template, Knowledge Base

5. Checklist Summary

Before You Start
- [x] Read the alert carefully
- [x] Check impact & severity
- [x] Gather initial context
- [x] Create incident

During Debugging
- [x] Don't guess
- [x] Follow the workflow
- [x] Collect evidence
- [x] Narrow the scope
- [x] Communicate updates

After Resolution
- [x] Verify completely
- [x] Monitor for regression
- [x] Close the incident
- [x] Document the RCA
- [x] Improve & prevent

6. Common Areas to Check

- DNS & Networking
- Load Balancer
- Application / Service
- Database
- Caches (Redis / ElastiCache)
- Queues (SQS / Kafka)
- Infrastructure (EC2 / EKS)
- IAM / Permissions
- Third-Party Services

7. Troubleshooting Layers (Top to Bottom)

User	DNS	Network	Edge	Load Balancer	Application	Data Layer	Third Party
Devices, Browsers, Mobile Apps	Route53, External DNS, TTL	VPC, Subnets, Security Groups, NACL, IGW	CloudFront, WAF, Shield, Global Accelerator	ALB / NLB / GWLB	EC2 / ECS / EKS / Lambda / App Services	RDS / DynamoDB / S3 / ElastiCache / SQS	APIs, SaaS, Payment Gateways, Email, etc.

8. Root Cause Checklist

- [x] What failed? (Symptom)
- [x] Why did it fail? (Immediate Cause)
- [x] What caused that? (Underlying Cause)
- [x] Why was it not detected earlier?
- [x] What will prevent this next time?

9. Incident Severity Guide

Sev 1	Critical Outage	Major impact, many users affected, business down.
Sev 2	Major Degradation	Partial impact, key features affected.
Sev 3	Minor Impact	Small impact, workaround available.
Sev 4	Minimal / No Impact	Low impact or informational.

10. Golden Rules

- ☆ Don't guess, verify.
- ☆ Check the latest change.
- ☆ Isolate before you fix.
- ☆ Metrics + Logs = Truth.
- ☆ Communicate early and often.
- ☆ Document and improve.

Key Takeaway: A consistent debugging workflow reduces MTTR, improves accuracy, and turns incidents into opportunities to build more reliable systems.

Senior Engineer Tip: The best engineers are not the fastest fixers; they are the best investigators, communicators, and problem solvers. Follow the process, not panic.

VERIQTA

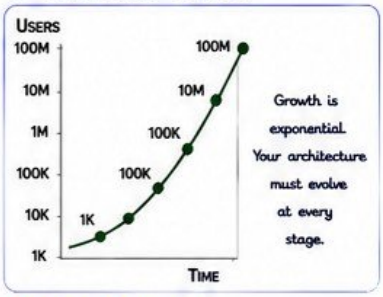
VERIQTA
SCALING FROM 1,000 TO 100 MILLION USERS

AWS SOLUTIONS ARCHITECT PRODUCTION HANDBOOK

ENGINEERING EXCELLENCE

Design for scale. Build for growth. Operate at scale.
A practical blueprint for taking your application from startup to global scale.

1. THE JOURNEY OVERVIEW



Growth is exponential. Your architecture must evolve at every stage.

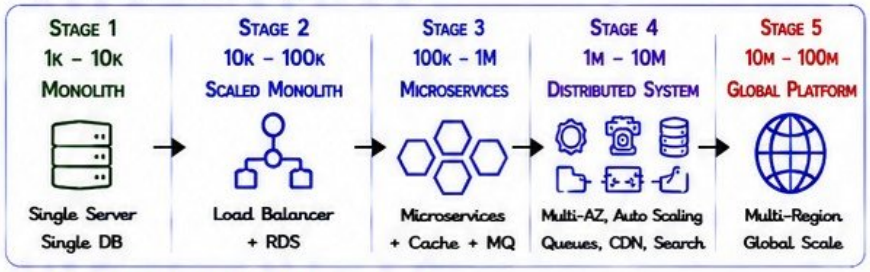
2. SCALING PRINCIPLES

- **DESIGN FOR SCALE** — Plan for 100x from day one.
- **AUTOMATE EVERYTHING** — Humans don't scale, systems do.
- **DECOUPLE & DISTRIBUTE** — Break into small, independent services.
- **DATA IS EVERYTHING** — Model, partition, cache, replicate.
- **OBSERVABILITY FIRST** — You can't fix what you can't see.
- **FAILURE IS NORMAL** — Design for resilience and graceful degradation.
- **COST OPTIMIZATION** — Scale efficiently, not blindly.

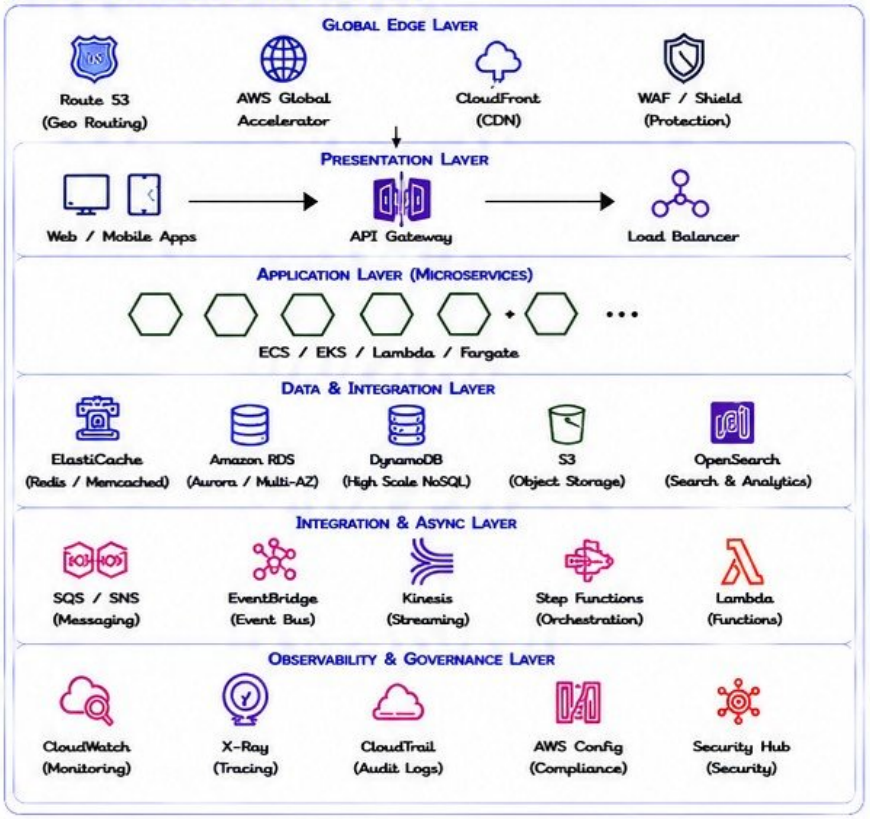
3. EVOLUTION STAGES

STAGE	USERS	FOCUS
STAGE 1	1K – 10K	Validate product, Keep it simple.
STAGE 2	10K – 100K	Stability, Monitoring, Automate deploys.
STAGE 3	100K – 1M	Decouple services, Database scaling, Caching.
STAGE 4	1M – 10M	Multi-AZ, DR, Queue based architecture.
STAGE 5	10M – 100M	Multi-Region, Global Scale, Advanced Data Architecture.

5. ARCHITECTURE EVOLUTION (AT A GLANCE)



6. TARGET ARCHITECTURE (100 MILLION USERS)



7. SCALE ENABLERS

- ☑ **Horizontal Scaling** — Every layer scales out independently.
- ☑ **Stateless Services** — No sessions on servers.
- ☑ **Caching Strategy** — Cache at multiple layers.
- ☑ **Database Scaling** — Read Replicas, Sharding, Partitioning.
- ☑ **Async Processing** — Queues and streams for decoupling.
- ☑ **CDN Everywhere** — Static + dynamic content delivery.
- ☑ **Global Deployment** — Multi-Region active-active architecture.
- ☑ **Automation & IaC** — Everything as code, repeatable at scale.
- ☑ **Security at Scale** — Zero trust, least privilege, automation.

4. KEY METRICS TO WATCH

1. Active Users (DAU / MAU)
2. Requests Per Second (RPS)
3. Latency (p50 / p95 / p99)
4. Error Rate (%)
5. Availability (Uptime %)
6. Infrastructure Cost / User

8. CAPACITY PLANNING EXAMPLE

COMPONENT	1K USERS (PEAK)	10K USERS (PEAK)	1M USERS (PEAK)	10M USERS (PEAK)	100M USERS (PEAK)
RPS (PEAK)	10	1,000	10,000	100,000	1,000,000+
WEB SERVERS	1 – 2	5 – 20	20 – 100	100 – 500	1000+ (Auto)
DATABASE	db.t3.small	db.r6g.large (RDS)	Aurora Multi-AZ	Aurora Cluster	Aurora Global Database
CACHE	N/A	ElastiCache	ElastiCache Cluster	Multi-Node Cluster	Global Cache
STORAGE	S3 Standard	S3 + Lifecycle	S3 + IA	S3 + Glacier	S3 + Glacier Deep Archive

9. DATA STRATEGY AT SCALE

- **Relational Data** — Aurora (Multi-AZ, Read Replicas, Partitioning)
- **Key-Value / NoSQL** — DynamoDB for massive scale and low latency
- **Caching Layer** — ElastiCache for sessions, counters, hot data
- **Analytics & Logs** — S3 + Athena + OpenSearch for insights
- **Data Pipeline** — Kinesis + Glue + Redshift for real-time analytics

10. COST OPTIMIZATION AT SCALE

- ☑ Use Auto Scaling & Right-Sizing
- ☑ Leverage Savings Plans & Reserved Instances
- ☑ Use Spot Instances for Fault-Tolerant Workloads
- ☑ Lifecycle Policies for S3 & Logs
- ☑ Cache Aggressively to Reduce DB Load
- ☑ Serverless for Spiky / Event-Driven Workloads
- ☑ Monitor & Optimize Continuously (FinOps)

EXAMPLE COST DRIVERS

Compute · Data Transfer · Storage · Database · Third Party

RELIABILITY BY DESIGN — Build resilient systems that can self-heal and recover automatically.

PERFORMANCE BY DESIGN — Optimize for latency, throughput, and user experience.

SECURITY BY DESIGN — Secure every layer and protect data at all times.

OBSERVABILITY BY DESIGN — Logs, metrics, traces built-in from day one.

GROWTH BY DESIGN — Architect today for the scale of tomorrow.

KEY TAKEAWAY
Scaling is not a single decision, it is a continuous evolution of people, process, and technology.
Design for where you are, but build for where you are going.

SENIOR ENGINEER TIP
The difference between a toy system and a platform is not code quality, it is the ability to scale reliably under real-world pressure.

VERIQTA

VERIQTA
ARCHITECTURE TRADE-OFF CASE STUDIES

AWS SOLUTIONS ARCHITECT PRODUCTION HANDBOOK

ENGINEERING VERIQTA EXCELLENCE

All architectures are about trade-offs.
Great engineers understand the business, evaluate the options, and choose the right balance for the right problem.

1. How to Read This Page

Each case shows a real architectural decision. We compare options across key factors and explain the decision and why.

2. Trade-Off Factors Explained

- **Cost** — Infrastructure and operational cost.
- **Performance** — Latency, throughput, and user experience.
- **Reliability** — Availability, durability, and fault tolerance.
- **Scalability** — Ability to grow with demand.
- **Complexity** — Operational and development complexity.
- **Compliance** — Security, governance, and regulatory needs.

3. Architecture Trade-Off Case Studies

Case & Context	Option A	Option B	Comparison (A vs B)						Recommended & Why
			Lower Cost	Better Latency	Higher Reliability	Auto Scale	Lower Complexity	Higher Security	
1 Hosting Static Content Serve images, CSS, JS for a global e-commerce app.	S3 + CloudFront	EC2 + Nginx							**Choose Option A (S3 + CloudFront)** Cheaper, globally distributed, highly available, auto scales and lower ops.
2 Database for Application High write workload, needs transactions and consistency.	Amazon RDS (Multi-AZ)	Amazon DynamoDB	Higher Cost	Good Performance	High Reliability (AZ)	Limited Scale (Vertical)	Lower Complexity	Easy Compliance	**Choose Depends** RDS for relational ACID workloads. DynamoDB for massive scale and single-digit ms.
3 Compute for Microservices Containerized microservices with variable traffic.	ECS on EC2	ECS on Fargate	Higher Cost	Same Performance	High Reliability	Better Auto Scale	Lower Complexity	Good Security	**Choose Option B (Fargate)** Less ops, no server management, scales to zero, faster delivery.
4 Caching Layer Reduce database load and improve response time.	ElastiCache (Redis)	DynamoDB DAX	Lower Cost	Faster Reads	High Reliability	Good Scale	Medium Complexity	Good Security	**Choose Depends** Redis for flexible caching. DAX for DynamoDB workloads only.
5 Data Analytics Platform Analyze TBs of logs and user behavior data.	Redshift	Athena + S3	Higher Cost	Faster Queries	High Reliability	Good Scale	Medium Complexity	Good Security	**Choose Depends** Redshift for complex analytics & BI. Athena + S3 for serverless, pay-per-query, simple analytics.

4. Decision Framework

- [x] 1 Understand the business requirement.
- [x] 2 Identify and list viable options.
- [x] 3 Compare using trade-off factors.
- [x] 4 Score each option (1 = low, 5 = high).
- [x] 5 Consider future growth and change.
- [x] 6 Make a decision and document why.
- [x] 7 Review and revisit as needed.

5. Common Patterns

- **Start Simple, Then Scale** — Begin with the simplest option, optimize when needed.
- **Measure, Don't Guess** — Use data to validate assumptions before scaling.
- **Automate Everything** — Automation reduces complexity and human errors.

6. Questions to Ask

- What is the real business goal?
- What are the growth projections?
- What happens if this fails?
- What is the cost impact?
- How easy is it to operate?
- Can we change this later?
- Are we optimizing prematurely?

7. Anti-Patterns To Avoid

- **Premature Optimization** — Solving problems you don't have.
- **Over-Engineering** — Adding complexity without value.
- **Ignoring Ops Cost** — Operational cost is real cost.
- **Vendor Lock-In** — Design that limits future choices.

8. Real-World Example Summary

Decision	Winning Option	Key Reason
Static Content	S3 + CloudFront	Lower cost, global scale
DB for Transactions	RDS (Multi-AZ)	ACID, joins, consistency
Microservices Compute	Fargate	No servers, auto scale
Caching Layer	ElastiCache (Redis)	Flexible, low latency
Analytics Platform	Athena + S3	Serverless, pay-per-use

9. Scoring Example (1 - 5 Scale)

Factor	Option A	Option B
Cost	2	5
Performance	4	3
Reliability	5	4
Scalability	3	5
Complexity	4	2
Compliance	4	4
Total	22	**23**

10. Key Takeaways

- [x] There is no perfect architecture.
- [x] Every choice is a trade-off.
- [x] The best architecture is the one that meets business goals with acceptable trade-offs.
- [x] Document your decisions.
- [x] Revisit and refactor when needed.
- [x] Great architects balance today's needs and tomorrow's scale.

Key Takeaway

Architecture is not about technology. It is about making the right trade-offs to deliver value, scale, and resilience.

Senior Engineer Tip

Good architects know when to build, when to buy, and when to wait. Context is everything.

AWS Solutions Architect Production Handbook

VERIQTA TOP 50 PRODUCTION DESIGN MISTAKES

Real mistakes. Real impact. Real lessons.
Avoid these and you'll save downtime, money, and your reputation.

1. Why These Mistakes Matter

- Outages & Downtime
- Data Loss
- Security Breaches
- High Costs
- Poor Performance
- Lost Customer Trust

Great design prevents problems.
Bad design creates incidents.

2. Mistake Categories

Architecture & Design
Structure, patterns, and high-level design.

Networking
VPC, routing, DNS, connectivity.

Security & IAM
Access control, identity, encryption, secrets.

Data & Storage
Databases, S3, backups, durability.

Scalability & Performance
Scaling, caching, capacity, and performance.

Reliability & HA
Availability, resilience, and fault tolerance.

Operations & Observability
Monitoring, logging, alerting, runbooks.

Cost Management
Waste, over-provisioning, inefficient design.

Deployment & Change
CI/CD, IaC, releases, config management.

Governance & Compliance
Policies, compliance, standards, auditing.

Remember
Design for failure.
Plan for scale.
Automate everything.
Monitor always.
Review continuously.

3. The Top 50 Mistakes

1-10 Architecture & Design

1. No clear architecture diagram
2. Single point of failure
3. No separation of concerns
4. Tight coupling between services
5. No environment separation
6. No multi-AZ design
7. Ignoring regional design
8. Over-reliance on one service
9. No disaster recovery plan
10. Not designing for failure

11-20 Networking

11. Flat network design
12. Overlapping CIDR ranges
13. No NAT gateway for private subnets
14. Exposing services to the internet
15. Security groups too open
16. NACLs misconfigured
17. DNS misconfiguration
18. Ignoring VPC flow logs
19. No VPC endpoints for AWS services
20. Incorrect route table association

21-30 Security & IAM

21. Using root account
22. Hardcoded credentials
23. Overly permissive IAM policies
24. No MFA on critical accounts
25. Secrets in code or images
26. No encryption at rest
27. No encryption in transit
28. Public S3 buckets
29. Missing key rotation
30. Ignoring least privilege

31-40 Data & Storage

31. No backup strategy
32. Backups not tested
33. Storing data in wrong AZ/Region
34. No versioning on S3
35. No lifecycle policies on S3
36. Using gp2 volumes
37. RDS in single AZ
38. No read replicas when needed
39. Large databases without indexing
40. No data retention policy

41-50 Scalability & Performance

41. No auto scaling
42. Wrong instance types
43. No load balancing
44. No caching strategy
45. Synchronous everything
46. Ignoring performance testing
47. No CDN for static content
48. Large monolith applications
49. Not monitoring resource usage
50. Over-provisioning or under-provisioning

4. Quick Impact Guide

Severity Level

- Critical – Causes outage, breach or data loss
- High – Major impact on performance or cost
- Medium – Noticeable impact or risk
- Low – Minor impact or improvement

5. How to Avoid Them

- [x] Design with best practices in mind
- [x] Use well-architected framework
- [x] Review and refine regularly
- [x] Automate and standardize
- [x] Test failure scenarios
- [x] Monitor and alert
- [x] Document everything
- [x] Keep learning

6. Golden Rules

- ☆ Simplicity before complexity
- ☆ Secure by default
- ☆ Automate for consistency
- ☆ Design for scale
- ☆ Plan for failure
- ☆ Measure what matters
- ☆ Improve continuously

7. Real-World Examples

Single Point of Failure
One server dies, the whole app goes down.
Impact: Outage
Solution: Multi-AZ + Auto Scaling

Public S3 Bucket
Sensitive data exposed to the world.
Impact: Breach
Solution: Block Public Access

No Backups
Data deleted or corrupted permanently.
Impact: Data Loss
Solution: Automated Backups + DR Plan

No Monitoring
Issues detected too late.
Impact: Downtime
Solution: CloudWatch + Alarms

Hardcoded Secrets
Secrets leaked in code repos or logs.
Impact: Breach
Solution: Secrets Manager

Senior Engineer Tip

Most production incidents are not random.
They are the result of avoidable design mistakes.

Good architecture is good engineering.

8. Production Design Checklist (Quick Scan)

Architecture
- [x] Multi-AZ / Multi-Region
- [x] No single Point of Failure
- [x] Environment Separation
- [x] DR / Backup Plan
- [x] Well-Documented

Networking
- [x] Proper Subnet Design
- [x] Route Tables Correct
- [x] Security Groups Tight
- [x] NAT / Endpoints Used
- [x] DNS Working

Security
- [x] Least Privilege IAM
- [x] MFA Enabled
- [x] Encryption At Rest
- [x] Encryption In Transit
- [x] Secrets Managed

Data
- [x] Backups Enabled
- [x] Backups Tested
- [x] Lifecycle Policies
- [x] Versioning Enabled
- [x] Right Storage Classes

Scalability
- [x] Auto Scaling Enabled
- [x] Load Balancers Used
- [x] Caching Implemented
- [x] Performance Tested
- [x] Right Instance Types

Operations
- [x] Monitoring in Place
- [x] Alarms Configured
- [x] Logging Centralized
- [x] Dashboards Built
- [x] Runbooks Ready

Cost
- [x] Right Sizing Done
- [x] Unused Resources Cleaned
- [x] Budgets & Alerts
- [x] Reserved Instances
- [x] Cost Reviewed

Key Takeaway

Avoiding these 50 mistakes will save you thousands of hours, prevent major incidents, and help you build systems that are secure, scalable, reliable, and cost-effective.

VERIQTA

VERIQTA

VERIQTA SENIOR ARCHITECT DECISION TREES

AWS Solutions Architect Production Handbook

ENGINEERING VERIQTA EXCELLENCE

Great architects don't guess. They evaluate, decide, and design with purpose.
Use these decision trees to make the right calls, every time.

1. Workload Type Decision

What type of workload are you building?

Stateless Web / API	Stateful Application	Batch / ETL Processing	Event-Driven Processing
Recommend: Auto Scaling Group, ALB, ECS or EKS	Recommend: RDS / Aurora or StatefulSet on EKS	Recommend: AWS Batch or ECS Fargate	Recommend: SQS + Lambda or EventBridge + Lambda

2. Database Choice

What is your primary data access pattern?

Relational ACID	Key-Value High Scale	Document Flexible Schema	Time-Series Metrics / Logs
Recommend: Amazon Aurora (PostgreSQL /MySQL)	Recommend: DynamoDB	Recommend: DocumentDB (MongoDB) or Amazon OpenSearch	Recommend: Amazon Timestream or OpenSearch

3. Scaling Strategy

How does your traffic behave?

Predictable Steady Growth	Spiky / Unpredictable	Very Large Scale (Global)
Recommend: Target Tracking Scaling (CPU / RPS)	Recommend: Step Scaling + Auto Scaling Policies	Recommend: Global Accelerator + Multi-Region Auto Scaling

4. Deployment Strategy

What is your deployment priority?

Zero Downtime	Fast Rollback	Simple & Low Risk	High Change Frequency
Recommend: Blue/Green Deployment	Recommend: Canary Deployment	Recommend: Rolling Deployment	Recommend: Feature Flags + CI/CD Pipelines

5. Network Architecture

What is your primary network requirement?

High Security & Isolation	High Performance Low Latency	Multi-Account Environment	Hybrid Connectivity
Recommend: Private Subnets, NACL, SG, VPC Endpoints	Recommend: Placement Groups, Enhanced Networking	Recommend: AWS Organizations + Transit Gateway	Recommend: Direct Connect + Transit Gateway or VPN

9. Observability & Monitoring

What do you need to observe?

Infrastructure Health	Application Performance	Logs & Troubleshooting	User Experience
Recommend: CloudWatch Alarms & Dashboards	Recommend: X-Ray + APM Tools	Recommend: CloudWatch Logs + OpenSearch	Recommend: CloudWatch Synthetics + RUM

6. Availability Requirement

What level of availability do you need?

99.99%+ (4 9's)	Multi-AZ Active / Passive
99.999%+ (5 9's)	Multi-AZ Active / Active
Global HA	Multi-Region Active / Active
Disaster Recovery	Pilot Light or Warm Standby

7. Security Focus

Protect data at rest	Encrypt with KMS
Secure access & identity	IAM Least Privilege + MFA
Network protection	VPC, SG, NACL, Private Subnets
Detect & respond to threats	GuardDuty, Security Hub, CloudTrail
Compliance & audit	AWS Config, CloudTrail, Artifact

8. Cost Optimization Focus

What is your main cost goal?

Reduce steady state cost	Right-size + Reserved Instances
Handle variable workloads	Auto Scaling + Spot Instances
Reduce idle resources	Schedule stop, Lambda, Fargate
Optimize storage cost	S3 IA / Glacier, Lifecycle Policies
Improve cost visibility	AWS Budgets + Cost Explorer

10. Architectural Pattern Decision (Quick Guide)

Pattern	Use When	Key Services	Benefits	Trade-Offs
Microservices	Independent services, frequent deployments	EKS, ECS, RDS, SQS, API GW	Scalability, resilience, team autonomy	Higher complexity, operational overhead
Monolith	Simple system, small team	EC2, RDS, ALB	Simple, faster delivery, lower overhead	Harder to scale, slower to evolve
Serverless	Event-driven, variable workloads	Lambda, API GW, DynamoDB, S3	No servers, auto scale, pay-per-use	Cold starts, vendor lock-in risk
Service-Oriented (SOA)	Enterprise integration, shared services	SQS, SNS, MQ, ESB	Reusability, loose coupling	Complex, heavy infrastructure
Event-Driven	Real-time processing, decoupled systems	EventBridge, SQS, Kinesis, Lambda	Real-time, scalable, resilient	Event ordering, complex debugging

11. Decision Framework (Simple Model)

1. Understand the business goal.
2. Define functional & non-functional requirements.
3. Identify constraints (budget, team, timeline).
4. Evaluate options using decision trees.
5. Choose the best fit for today and tomorrow.
6. Revisit and evolve as needs change.

Key Takeaway

There is no perfect architecture.
Only the right architecture for the right problem at the right time.
Use these trees, learn from outcomes, and keep improving.

Senior Engineer Tip

Good architects ask better questions, explore trade-offs, and document their decisions for the team and the future.

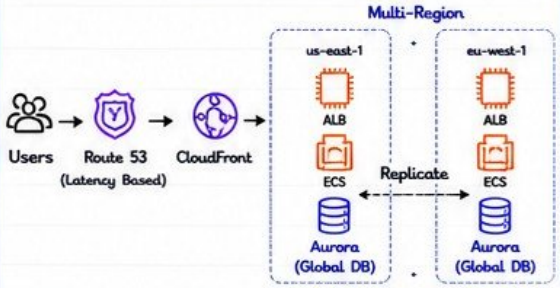
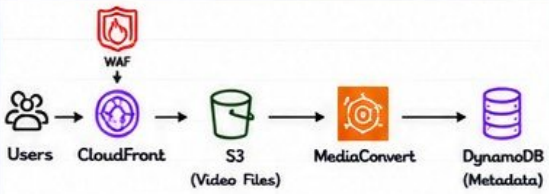
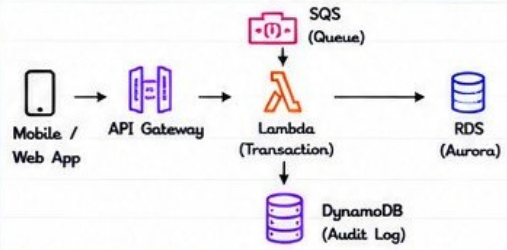
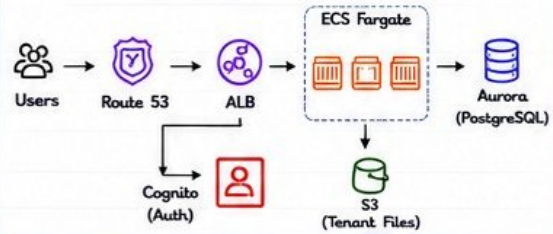
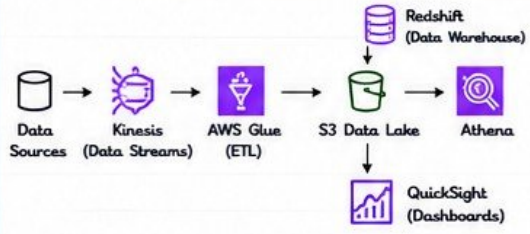
VERIQTA

VERIQTA
REAL-WORLD ARCHITECTURE SCENARIOS

AWS SOLUTIONS ARCHITECT PRODUCTION HANDBOOK

ENGINEERING VERIQTA EXCELLENCE

Real problems. Real architectures. Real lessons.
Understand how senior architects solve complex challenges in production.

#	Scenario & Context	High Level Architecture	Key Architecture Decisions	Outcome & Benefits
1	**Global E-Commerce Platform** **Context:** Need to support users worldwide with low latency, high availability, and ability to handle traffic spikes during sales.		✓ Multi-region active-active ✓ Route 53 latency based routing ✓ CloudFront for global content delivery ✓ Aurora Global Database for low latency writes ✓ ECS for containerized microservices	**Outcome:** • < 100ms latency globally • Zero downtime deployments • Handles 10x traffic spikes • High availability (99.99%+)
2	**Media Streaming Service** **Context:** Deliver video content at scale with high availability and efficient cost.		✓ S3 + CloudFront for cost-effective delivery ✓ MediaConvert for on-the-fly transcoding ✓ DynamoDB for metadata (high throughput) ✓ WAF for protection	**Outcome:** • High quality streaming at low cost • Scales to millions of concurrent viewers • Secure and resilient architecture
3	**Fintech Payment System** **Context:** Process millions of transactions securely with high consistency and auditability.		✓ API Gateway + Lambda for secure APIs ✓ SQS for decoupling and resilience ✓ Aurora (Multi-AZ) for ACID transactions ✓ DynamoDB for immutable audit logs ✓ KMS for encryption	**Outcome:** • 99.99% availability • Strong consistency for financial data • Full audit trail and compliance • Secure by design
4	**SaaS Application (Multi-Tenant)** **Context:** Build a SaaS platform that securely isolates tenants and scales cost-effectively.		✓ Shared compute (ECS Fargate) ✓ Database per tenant (Aurora Serverless v2) ✓ Cognito for authentication ✓ S3 for tenant file storage ✓ Tag-based isolation and IAM policies	**Outcome:** • Secure tenant isolation • Pay-per-use scaling • Easy onboarding of new tenants • High availability
5	**Data Analytics Platform** **Context:** Ingest, process, and analyze large volumes of data for business insights.		✓ Kinesis for real-time data ingestion ✓ Glue for serverless ETL ✓ S3 as data lake ✓ Redshift for analytics ✓ QuickSight for dashboards	**Outcome:** • Real-time + batch analytics • Scalable to PBs of data • Lower cost with serverless services • Actionable insights at scale

Architect's Checklist (Apply to any Scenario)

- ☑ Understand the business problem and constraints
- ☑ Define success metrics (performance, availability, cost)
- ☑ Choose the right services for the job
- ☑ Design for failure (HA, DR, resilience)
- ☑ Optimize for cost without sacrificing quality
- ☑ Plan for security, compliance, and governance
- ☑ Monitor, measure, and iterate

Key Principles

- ☆ Start with the business problem
- ☆ Design for scale and failure
- ☆ Decouple and automate
- ☆ Security is everyone's responsibility
- ☆ Use managed services when possible
- ☆ Measure everything
- ☆ Keep it simple, then evolve

Senior Architect Tip

There is no perfect architecture.
The best architecture today is the one that meets business goals, is secure, scalable, reliable, and can evolve as needs change.

VERIQTA

VERIQTA

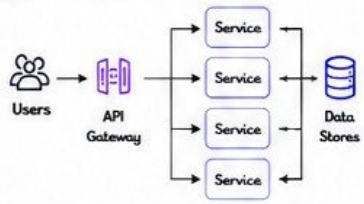
VERIQTA ENTERPRISE DESIGN PATTERNS CHEAT SHEET

AWS SOLUTIONS ARCHITECT PRODUCTION HANDBOOK

ENGINEERING VERIQTA EXCELLENCE

Reusable architecture patterns for scalable, secure, reliable, and cost-effective enterprise systems.

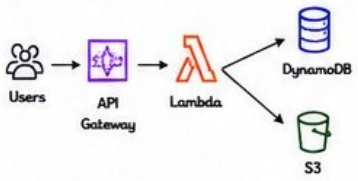
1 MICROSERVICES PATTERN



USE CASES
- ☑ Large applications
- ☑ Independent teams
- ☑ Frequent releases

BENEFITS
- ☑ Scalability
- ☑ Resilience
- ☑ Lambda
- ☑ RDS / DynamoDB
- ☑ SQS / SNS

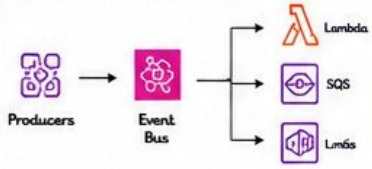
2 SERVERLESS PATTERN



USE CASES
- ☑ Event-driven apps
- ☑ APIs & backends
- ☑ Batch processing

KEY AWS SERVICES
- ☑ Lambda
- ☑ API Gateway
- ☑ DynamoDB
- ☑ S3
- ☑ EventBridge

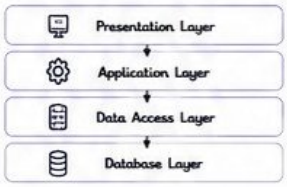
3 EVENT-DRIVEN ARCHITECTURE



USE CASES
- ☑ Real-time processing
- ☑ Decoupled systems
- ☑ Notification systems

BENEFITS
- ☑ Loose coupling
- ☑ Scalability
- ☑ Resilience

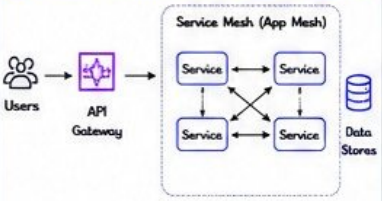
4 LAYERED ARCHITECTURE (N-TIER)



USE CASES
- ☑ Enterprise apps
- ☑ Complex systems
- ☑ Clear separation of concerns

KEY AWS SERVICES
- ☑ ALB / CloudFront
- ☑ ECS / EC2 / Lambda
- ☑ RDS / DynamoDB
- ☑ ElastiCache

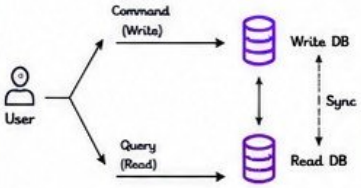
5 MICROSERVICES WITH SERVICE MESH



USE CASES
- ☑ Microservices at scale
- ☑ Traffic management
- ☑ Observability

BENEFITS
- ☑ Secure traffic
- ☑ Traffic control
- ☑ Visibility

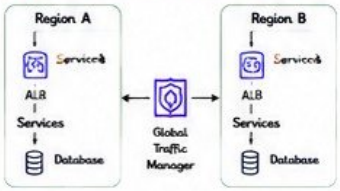
6 CQRS (COMMAND QUERY RESPONSIBILITY SEGREGATION)



USE CASES
- ☑ High read/write loads
- ☑ Complex domains
- ☑ Reporting systems

KEY AWS SERVICES
- ☑ DynamoDB
- ☑ RDS (Read Replicas)
- ☑ Lambda
- ☑ EventBridge

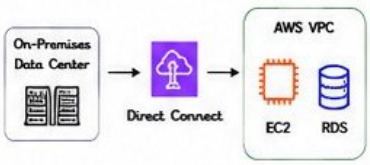
7 MULTI-REGION ACTIVE-ACTIVE



USE CASES
- ☑ Global apps
- ☑ High availability
- ☑ Low latency

BENEFITS
- ☑ No single region failure
- ☑ Better latency
- ☑ High availability

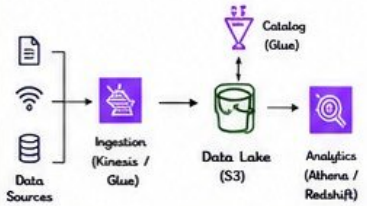
8 HYBRID CLOUD PATTERN



USE CASES
- ☑ Legacy integration
- ☑ Data residency
- ☑ Gradual migration

KEY AWS SERVICES
- ☑ Direct Connect
- ☑ AWS VPN
- ☑ VPC
- ☑ Storage Gateway

9 DATA LAKE ARCHITECTURE



USE CASES
- ☑ Big data analytics
- ☑ Machine learning
- ☑ Data consolidation

BENEFITS
- ☑ Scalable storage
- ☑ Flexible schema
- ☑ Cost-effective

10 SECURE LANDING ZONE

USE CASES
- ☑ Enterprise setup
- ☑ Governance
- ☑ Security baseline

KEY AWS SERVICES
- ☑ Organizations
- ☑ Control Tower
- ☑ CloudTrail
- ☑ AWS Config
- ☑ GuardDuty

ARCHITECT'S CHECKLIST

- ☑ Define requirements and constraints clearly.
- ☑ Choose the right pattern for the use case.
- ☑ Consider scalability, resilience, and security.
- ☑ Evaluate cost and operational complexity.
- ☑ Design for failure and automate recovery.
- ☑ Monitor, measure, and iterate.

KEY PRINCIPLES

- ☆ Design for scale and change.
- ☆ Decouple and automate everything.
- ☆ Security is built in, not added on.
- ☆ Use managed services when possible.
- ☆ Measure what matters.
- ☆ Keep it simple, then evolve.

SENIOR ARCHITECT TIP

Patterns are starting points, not one-size-fits-all solutions. Great architects understand the business context, the trade-offs, and adapt patterns to build systems that are reliable, scalable, secure, and cost-effective.

VERIQTA

VERIQTA
PRODUCTION ARCHITECTURE CHECKLIST

AWS SOLUTIONS ARCHITECT PRODUCTION HANDBOOK

ENGINEERING VERIQTA EXCELLENCE

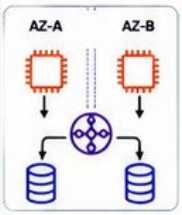
Use this checklist before going live. Great architectures are tested, validated, and reviewed – not assumed.

1 BUSINESS & ARCHITECTURE ALIGNMENT

- ☐ Business goals and success metrics defined
- ☐ Stakeholders and owners identified
- ☐ Requirements and constraints documented
- ☐ Architecture reviewed and approved
- ☐ Cost, security, and compliance aligned
- ☐ Risks identified and mitigated

Confirm the architecture solves the right problem in the right way.

2 RELIABILITY & HIGH AVAILABILITY



- ☐ Multi-AZ / Multi-Region strategy
- ☐ No single points of failure
- ☐ Health checks configured
- ☐ Auto Scaling configured
- ☐ Backups tested and automated
- ☐ DR plan documented and tested
- ☐ RTO / RPO defined and met

Assume failure. Design for continuous availability and fast recovery.

3 SECURITY & COMPLIANCE

- ☐ Least privilege IAM enforced
- ☐ MFA enabled for privileged access
- ☐ Encryption at rest (KMS)
- ☐ Encryption in transit (TLS)
- ☐ Secrets stored securely (Secrets Manager)
- ☐ Security Groups / NACLs reviewed
- ☐ WAF / Shield enabled
- ☐ Compliance requirements validated
- ☐ Audit logging enabled (CloudTrail)

Security is not a feature, it's a foundation.

4 NETWORK & CONNECTIVITY

- ☐ VPC design follows best practices
- ☐ Subnets across multiple AZs
- ☐ Route tables correct and isolated
- ☐ Internet access controlled
- ☐ NAT Gateways configured
- ☐ VPN / Direct Connect (if needed)
- ☐ DNS resolution validated
- ☐ Endpoint services used for AWS access
- ☐ Network ACLs reviewed

A solid network is the foundation of everything.

5 PERFORMANCE & SCALABILITY

- ☐ Capacity planning completed
- ☐ Auto Scaling policies tested
- ☐ Load balancers configured
- ☐ Caching strategy implemented
- ☐ Database performance optimized
- ☐ Connection pooling enabled
- ☐ CDN used for static / global content
- ☐ Performance tested under load
- ☐ Bottlenecks identified and resolved

Design to scale. Test to prove. Monitor to sustain.

6 DATA & STORAGE

- ☐ Data classification completed
- ☐ Storage class optimized
- ☐ Lifecycle policies configured
- ☐ Backups automated and tested
- ☐ Cross-Region replication (if needed)
- ☐ Database encryption enabled
- ☐ Data retention policies defined
- ☐ Data integrity checks enabled
- ☐ Disaster recovery validated

Protect data, ensure durability, and plan for recovery.

7 OPERATIONS & OBSERVABILITY

- ☐ Centralized logging enabled
- ☐ Metrics and dashboards configured
- ☐ Alarms and notifications set
- ☐ Tracing enabled (X-Ray / OTEL)
- ☐ Runbooks and SOPs documented
- ☐ On-call and escalation defined
- ☐ SLOs / SLIs defined and tracked
- ☐ Synthetic monitoring configured
- ☐ Monitoring tested end-to-end

You can't improve what you don't observe.

8 DEPLOYMENT & AUTOMATION

- ☐ IaC used for all resources
- ☐ CI/CD pipeline tested
- ☐ Blue/Green or Canary strategy
- ☐ Rollback strategy validated
- ☐ Configuration management in place
- ☐ Secrets not hardcoded
- ☐ Artifact repositories secured
- ☐ Environment parity validated
- ☐ Change management followed

Automate everything. Manual steps create risk.

9 COST MANAGEMENT

- ☐ Cost estimation completed
- ☐ Budgets and alerts configured
- ☐ Right-sizing done for all services
- ☐ Reserved Instances / Savings Plans
- ☐ Unused resources removed
- ☐ Lifecycle policies for cost efficiency
- ☐ Cost allocation tags applied
- ☐ Regular cost reviews scheduled
- ☐ FinOps best practices followed

Optimize cost without sacrificing performance or reliability.

10 TESTING & VALIDATION

- ☐ Unit, integration, and system tests passed
- ☐ Load / stress tests completed
- ☐ Security testing completed
- ☐ Failover testing completed
- ☐ DR testing completed
- ☐ User acceptance testing passed
- ☐ Performance benchmarks met
- ☐ Regression tests passed
- ☐ Go/No-Go decision documented

Test in production-like conditions. Trust is earned, not assumed.

11 DOCUMENTATION & GOVERNANCE

- ☐ Architecture diagrams up to date
- ☐ Runbooks and guides documented
- ☐ Ownership and contacts defined
- ☐ Governance and policies followed
- ☐ Compliance evidence collected
- ☐ Change history documented
- ☐ Decision records maintained
- ☐ Knowledge base updated
- ☐ Training completed for teams

Good documentation reduces risk and speeds up recovery.

12 CO-LIVE READINESS

- ☐ All critical checks completed
- ☐ Stakeholder sign-off obtained
- ☐ Communication plan ready
- ☐ Rollback plan ready and tested
- ☐ Monitoring and alerts verified
- ☐ Support and escalation ready
- ☐ Go-Live window approved
- ☐ Post-launch review scheduled
- ☐ Confident? If yes, GO LIVE!

Ready. Set. Launch! Build it right. Run it right.

FINAL REVIEW SUMMARY

Review Date: _____

Reviewed By: _____

Environment: ☐ Dev ☐ Staging ☐ Production

Overall Readiness: ☐ Not Ready ☐ Ready

Notes / Risks: _____

SENIOR ARCHITECT TIP

A great architecture is not about using the most services. It's about designing simple, resilient, scalable, secure, and cost-effective systems that deliver real business value.

SIMPLICITY + AUTOMATION + OBSERVABILITY = PRODUCTION EXCELLENCE

VERIQTA

LEGEND

- Reliability & Data
- Security & Operations
- Performance & Delivery
- Architecture & Design
- Readiness & Governance

VERIQTA

VERIQTA FINAL ENTERPRISE SOLUTIONS ARCHITECT PLAYBOOK

AWS Solutions Architect Production Handbook

The complete guide to design, validate, build, and run world-class enterprise architectures.

Your Mission

Deliver enterprise solutions that are secure, scalable, reliable, cost-efficient, and aligned with business goals. Always solve the right problem the right way.

Secure | Scalable | Reliable | Cost-Efficient | Aligned

1. The Architect Mindset

- [x] Start with the business. Technology is the means, not the goal.
- [x] Solve the right problem, not just the one in front of you.
- [x] Think in trade-offs. Every decision has a cost.
- [x] Design for change. Systems evolve, architectures endure.
- [x] Automate everything. Manual steps create risk.
- [x] Security is everyone's responsibility.
- [x] Measure what matters. Data drives architecture.
- [x] Document decisions. Future you will thank you.
- [x] Keep it simple, then evolve.

2. Architecture Excellence Framework

3. Enterprise Architecture Lifecycle

Discover	Design	Validate	Build	Operate
Understand business, users, data, and constraints	Create options, evaluate, select best design	Test assumptions, review trade-offs, validate	Implement with automation & standards	Run, monitor, optimize, evolve

Continuous feedback loops across all phases.

4. Core Architect Responsibilities

- [x] Understand business deeply
- [x] Define architecture vision & principles
- [x] Design scalable & secure solutions
- [x] Evaluate trade-offs & risks
- [x] Guide teams & influence decisions
- [x] Establish standards & guardrails
- [x] Validate, test & iterate designs
- [x] Ensure operational readiness
- [x] Drive cost efficiency & sustainability
- [x] Mentor and grow engineering teams

Remember

You are not just designing systems.
You are designing outcomes, experiences, and the future.
– VERIQTA

5. Top Architecture Trade-Offs

Goal		Common Trade-Offs
High Availability	↔	Increased cost, operational complexity
Performance	↔	Higher cost, more infrastructure
Security	↔	Usability, development speed
Scalability	↔	Complexity, cost, operational overhead
Consistency	↔	Availability, partition tolerance (CAP)
Cost Efficiency	↔	Performance, redundancy, features
Flexibility	↔	Governance, standardization
Speed to Market	↔	Technical debt, long-term maintainability

6. Architecture Decision Checklist

- [ ] Does this solve the right business problem?
- [ ] Have we considered multiple options?
- [ ] Is the design secure by default?
- [ ] Is it scalable for future growth?
- [ ] Is it highly available and fault tolerant?
- [ ] Have we minimized single points of failure?
- [ ] Is it cost-effective over the long term?
- [ ] Can it be operated and monitored easily?
- [ ] Are compliance and governance needs met?
- [ ] Have we documented decisions and trade-offs?
- [ ] Do we have a plan for testing and validation?
- [x] Is the design simple, maintainable, and evolvable?

If any answer is NO – Revisit the design.

7. Operational Readiness Checklist

- [ ] Monitoring & alerting configured
- [ ] Logging & tracing enabled
- [ ] Runbooks & SOPs documented
- [ ] Incident response plan ready
- [ ] Capacity planning completed
- [ ] Auto scaling & self-healing enabled
- [ ] Backup & disaster recovery tested
- [ ] Security monitoring active
- [ ] Cost dashboards & budgets set
- [ ] Access reviews & audits scheduled
- [ ] Performance baselines established
- [ ] Change management process ready

Ready to run. Ready to scale. Ready to win.

8. Architect's Toolkit

Design & Modeling	draw.io, Lucidchart, Excalidraw
IaC & Automation	Terraform, CDK, CloudFormation
CI/CD	GitHub Actions, GitLab CI, Jenkins
Observability	CloudWatch, Prometheus, Grafana
Security	IAM, GuardDuty, Security Hub
Cost Management	Cost Explorer, Budgets, CUR
Collaboration	Confluence, Jira, Notion
Testing	k6, Locust, Integration Tests

9. Architect Principles (VERIQTA)

Customer First Design for real business value.	**Simplicity** Simple today, evolvable tomorrow.	**Security by Design** Build security in, not bolted on.	**Operational Excellence** Run it like you mean it.	**Data Driven** Measure, learn, and decide with data.
Resilience Expect failure. Design to recover.	**Automation First** Automate everything repeatable.	**Cost Conscious** Maximize value. Eliminate waste.	**Collaboration** Great solutions come from great teams.	**Continuous Improvement** Always learn. Always improve.

10. Final Architect Playbook

1. Understand the business
2. Define the vision & principles
3. Explore options & evaluate
4. Design the best solution
5. Validate assumptions
6. Build with automation
7. Operate, monitor & optimize
8. Evolve and improve

You don't just build architectures.
You build the future.

Final Thought

Architecture is a journey, not a destination.
Keep learning. Keep designing. Keep delivering.
Make impact. Leave a legacy.

Senior Architect Tip

Leave every system better than you found it.
Document your decisions.
Mentor others. Build for the long term.

VERIQTA